

The Art of Grassroots Logic Programming

Ehud Shapiro

London School of Economics
and
Weizmann Institute of Science

2025

Preface

This book presents GLP, a secure, multiagent, concurrent logic programming language designed for implementing grassroots platforms—distributed applications run by cryptographically-identified people on their networked personal devices.

A Personal Journey

The journey to GLP began decades ago with Concurrent Prolog and the Fifth Generation Computer Systems project. The vision then was to harness the power of logic programming for parallel computation. That vision evolved, and the core ideas matured through Flat Concurrent Prolog (FCP) and its theoretical foundations.

GLP represents the culmination of this journey, addressing a new challenge: how can ordinary people, using only their personal devices, create and operate distributed platforms without relying on centralized servers or trusted third parties?

Relationship to Earlier Works

Readers familiar with *The Art of Prolog* will recognize the pedagogical approach: we build understanding progressively, from simple concepts to sophisticated applications. However, where that book focused on sequential logic programming, this book embraces concurrency as fundamental.

Readers of *Concurrent Prolog: Collected Papers* will find the theoretical foundations familiar, but the focus here is practical: how to write programs that actually work in grassroots settings, with all the challenges of security, identity, and distributed coordination.

The Grassroots Vision

The grassroots vision is simple but radical: platforms owned and operated by their users, not by corporations. Social networks without surveillance capital-

ism. Currencies without banks. Democratic institutions without centralized control.

GLP provides the programming foundation for this vision. Its single-reader/single-writer discipline ensures that distributed computations remain secure. Its cryptographic integration enables identity verification without trusted authorities. Its concurrent semantics naturally express the parallelism inherent in grassroots systems.

Acknowledgments

[To be written]

Rehovot, 2025

How to Read This Book

This book has two parallel tracks: an *informal track* and a *formal track*.

The Informal Track

The informal track provides intuitive explanations, examples, and programming techniques. It is comprehensive and self-contained. Most readers should follow this track.

The informal presentation uses:

- Plain English explanations of concepts
- Numerous program examples with detailed walkthroughs
- Diagrams illustrating data flow and communication
- Analogies connecting GLP concepts to familiar ideas

The Formal Track

The formal track provides precise mathematical definitions and proofs. It appears in shaded boxes labeled “Formal X.Y” throughout the text.

Formal 0.1: Example Formal Box

This is what a formal box looks like. It contains precise definitions, theorems, and proofs that complement the informal exposition.

Definition 0.1 (Example Definition). *A widget is a triple (A, B, C) where A is a set, B is a relation on A , and C is a distinguished element of A .*

Readers who want rigorous foundations should read the formal boxes. Readers who prefer intuition can skip them entirely without losing the thread of the exposition.

The formal material is drawn from the GLP specification paper and provides:

- Precise syntax definitions using BNF grammar
- Operational semantics via transition systems
- Safety properties (SRSW preservation, acyclicity, monotonicity)
- Proofs of key theorems

Prerequisites

This book assumes familiarity with:

- Basic programming concepts (variables, functions, recursion)
- Elementary logic (propositions, predicates, implication)
- Some exposure to functional or logic programming is helpful but not required

No prior knowledge of Prolog or concurrent programming is assumed, though readers with such background will progress faster through the early chapters.

Chapter Dependencies

The book is organized into three parts:

Part I: Foundations Chapters 1–4 must be read in order. They establish the core concepts of logic programs and GLP.

Part II: Concurrent Programming Chapters 5–7 build on Part I. They cover stream programming, recursive programming, and metaprogramming.

Part III: Simulating Multiagent Systems Chapters 8–12 require Parts I–II. They present grassroots protocols: the social graph, social networks, interlaced streams, cryptocurrencies, and constitutional consensus.

Exercises

Each chapter concludes with exercises of varying difficulty:

- ★ Basic exercises reinforcing chapter concepts
 - ★★ Intermediate exercises requiring synthesis
 - ★★★ Advanced exercises exploring extensions
- Solutions to selected exercises appear in Appendix 12.10.

Notation Conventions

- Program code appears in `typewriter` font
- Variables in programs start with uppercase: `X`, `Stream`, `Result`
- Constants and functors start with lowercase: `merge`, `person(alice)`
- Reader variables use suffix `?`: `X?`
- Writer variables have no suffix: `X`
- Mathematical variables appear in italics: x , σ , θ

Contents

Preface	iii
How to Read This Book	v
Contents	vii
I GLP Concurrent Programming	1
1 Introduction	3
1.1 The Grassroots Vision	3
1.2 The Programming Challenge	4
1.3 Logic Programming: A Natural Foundation	4
1.4 Concurrent Logic Programming	5
1.5 The Single-Reader/Single-Writer Insight	5
1.6 A First GLP Program	5
1.7 Security Through Cryptography	6
1.8 Book Overview	6
2 Logic Programs and Linear Logic	9
2.1 Logic Programs	9
2.2 Linear Logic	12
3 GLP Core	15
3.1 GLP	15

3.2	Guards	21
3.3	Exercises	24
4	Basic Concurrent Programming	25
4.1	Programming with Constants	25
4.2	Streams	30
4.3	Recursive Programming	37
4.4	Metaprogramming	41
II	Moded Types	45
5	Types and Modes	47
5.1	Type Definitions	47
5.2	Built-in Types	48
5.3	Moded Procedure Declarations	48
5.4	Mode Checking	49
5.5	Embedded Modes: Response Slots	50
5.6	Complete Example: Typed Quicksort	51
5.7	Type Errors and Mode Errors	51
5.8	Summary	52
6	Typed Programming	53
6.1	Difference Lists	53
6.2	Quicksort	53
6.3	Equators: Emergency Brake	53
6.4	Bidirectional Communication	53
6.5	Buffered Communication	53
7	Module System	55
7.1	Design Principles	55
7.2	Module Structure	56
7.3	Procedure Declarations	56
7.4	Cross-Module Type Checking	58
7.5	Project Compilation	58
7.6	Dynamic Linking	60
7.7	Validation: Child-Safe Social Graph	61
III	Simulating Multiagent Systems	63
	Introduction to Part III	65
8	The Grassroots Social Graph	69
8.1	Graph Structure	69

8.2	Agent Initialization	69
8.3	Cold-Call Befriending Protocol	70
8.4	Friend-Mediated Introduction	72
8.5	Channel Notation	74
8.6	Utility Predicates	74
8.7	Testing with Multiagent Plays	75
8.8	Exercises	78
9	Social Networks	81
9.1	Direct Messaging	81
9.2	Feeds and Followers	84
9.3	Groups	87
9.4	Child-Safe Social Networking	92
9.5	Exercises	95
10	Interlaced Streams	97
10.1	Blocklace Structure	97
10.2	The Interlaced Streams Program	97
10.3	Execution Trace	98
10.4	Multiagent Deployment	99
10.5	Security Properties	99
10.6	Applications	100
10.7	Exercises	100
11	Grassroots Cryptocurrencies	101
11.1	Introduction	101
11.2	The Blocklace	103
11.3	The Grassroots Flash Protocol	105
11.4	Implementation in GLP	106
11.5	A Complete Example	110
11.6	Security Properties	112
11.7	Exercises	113
12	Constitutional Consensus	115
12.1	Introduction	115
12.2	The Consensus Problem	117
12.3	The Blocklace for Consensus	118
12.4	Wave Structure	119
12.5	Dual-Mode Operation	120
12.6	The Ordering Function	121
12.7	Implementation in GLP	121
12.8	A Complete Example	124
12.9	Correctness Properties	126
12.10	Exercises	127

GLP Syntax Reference	129
Lexical Elements	129
Terms	129
Atoms and Goals	129
Clauses	130
Guards	130
Programs	130
SRSW Requirements	130
GLP Reference Manual	131
Lexical Syntax	131
Clause Syntax	131
Guard Predicates	132
System Predicates	134
Arithmetic	134
Channel Abstractions	135
Standard Library Predicates	136
Stream Security Properties	137
Execution Model Summary	137
Writer Unification Catalog	139
Notation	139
Atomic Term Unification	139
Compound Term Unification	140
List Structure Cases	141
Difference List Cases	142
Guard Evaluation Cases	143
Clause Selection Mechanisms	144
Suspension and Reactivation	145
Cross-Reference Index	146
Implementation-Ready GLP	147
Overview	147
Goal Scheduling	147
Suspension and Activation	147
Message Delivery	147
Deterministic Transition System	148
Smartphone Implementation	148
From Prolog to GLP	149
Key Differences	149
Converting Programs	149
Common Patterns	149
Pitfalls	150

Historical Notes	151
Logic Programming Origins	151
Concurrent Logic Programming	151
The Grassroots Vision	152
From FCP to GLP	152
Acknowledgments	152
Social Networks Code	153
Direct Messaging Play	153
Feeds and Followers Play	154
Manager-Based Group Play	155
Interlaced Streams Group Play	156
Child-Safe Social Networking Play	157
Selected Exercise Solutions	159
Chapter ??: Streams	159
Library Utilities	163
Channel Operations	163
Stream-Channel Relay	163
Broadcast	164
Lookup Utilities	164
Stream Tagging	164
Deferred Injection	165
Time Utilities	165
Guard Utilities	165
Bibliography	167

Part I

GLP Concurrent
Programming



Introduction

This book presents GLP, a programming language designed to enable ordinary people to build and operate distributed platforms using only their personal devices. Before diving into the technical details, let us understand why such a language is needed and what makes it different from conventional programming languages.

1.1 The Grassroots Vision

Imagine social networks without corporate surveillance. Currencies without banks. Democratic institutions without centralized control. These are not utopian dreams—they are technically achievable through what we call *grassroots platforms*.

Grassroots platforms are distributed applications run by cryptographically-identified people on their networked personal devices, where multiple disjoint platform instances emerge independently and coalesce when they interoperate.

In a grassroots platform:

- **People run the platform**, not corporations. Each participant contributes their smartphone or computer.
- **Identity is cryptographic**, not administrative. You prove who you are through digital signatures, not usernames and passwords managed by a company.
- **Trust is personal**, not institutional. You communicate with authenticated friends, building a web of trust from the ground up.

- **Instances grow organically.** Small groups form independently and merge when they connect.

The foundation of all grassroots platforms is the *grassroots social graph*—a network where nodes represent people, edges represent authenticated friendships, and connected components arise spontaneously through befriending. Upon this foundation, grassroots social networks, cryptocurrencies, and democratic federations can be built.

1.2 The Programming Challenge

Why do we need a new programming language? Can't we build grassroots platforms using existing languages?

The challenge is that grassroots platforms operate in an environment of *faulty and malicious participants*. Without proper language support:

- Correct participants cannot reliably identify each other
- Secure communication channels cannot be established
- Code integrity cannot be verified

Existing programming languages lack the necessary combination of:

1. Distributed execution with cryptographic security
2. Multiagent operational semantics with proven properties
3. Abstractions suitable for grassroots programming
4. Safety and liveness guarantees

GLP aims to close this gap.

1.3 Logic Programming: A Natural Foundation

GLP builds on the tradition of logic programming, which views computation as logical deduction. This is not merely an aesthetic choice—it provides:

- **Declarative semantics:** Programs describe *what* to compute, not just *how*
- **Formal foundations:** Properties can be stated and proven mathematically
- **Natural concurrency:** Multiple goals can be pursued simultaneously

The name “GLP” stands for **G**rassroots **L**ogic **P**rogramming. It extends classical logic programs with mechanisms for secure, concurrent, multiagent computation.

1.4 Concurrent Logic Programming

Standard logic programming, exemplified by Prolog, is sequential. Goals are pursued one at a time, with backtracking when choices fail. This model is unsuitable for distributed systems where multiple agents must operate concurrently.

Concurrent logic programming, pioneered in the 1980s with languages like Concurrent Prolog, PARLOG, and GHC, introduced parallelism through *committed choice*—once a clause is selected, there is no backtracking. Communication occurs through shared logical variables.

However, concurrent logic programming faced a fundamental problem: *distributed unification*. When multiple processes can instantiate the same variable, ensuring consistency requires complex distributed protocols.

1.5 The Single-Reader/Single-Writer Insight

GLP's key innovation is the *Single-Reader/Single-Writer (SRSW)* discipline:

- Each logical variable has exactly one writer (who can assign it a value)
- Each logical variable has exactly one reader (who can observe the value)
- The reader and writer form a pair, establishing a communication channel

This discipline eliminates distributed unification entirely. Assignment becomes simple point-to-point communication. Yet expressive power is retained—complex communication patterns emerge from sharing readers and writers in messages.

1.6 A First GLP Program

Let us see GLP in action with a classic example: merging two streams.

Program 1.1: Fair Stream Merger

```
merge([X|Xs],Ys,[X?|Zs?]) :- merge(Ys?,Xs?,Zs).
merge(Xs,[Y|Ys],[Y?|Zs?]) :- merge(Xs?,Ys?,Zs).
merge([],[],[]).
```

This program merges two input streams into one output stream. Let us understand each piece:

- `[XXs]`— and `[YYs]`— are input streams (lists that may grow)
- `[X?Zs?]`— is the output stream being constructed
- `X?` and `Zs?` are *readers*—they provide read-only access to values
- `X` and `Zs` are *writers*—they can be assigned values

The first clause says: “If the first stream has element X , output it and recursively merge, swapping the streams for fairness.” The swap ensures both streams are treated equally.

Notice that each variable occurs exactly once as a reader and once as a writer—this is the SRSW discipline in action.

Formal 1.1: The merge Program

The `merge` program demonstrates fundamental GLP properties:

1. **SRSW Compliance:** Each variable X , Xs , Ys , Zs occurs exactly once as a writer (in the head or body) and its paired reader ($X?$, $Xs?$, $Ys?$, $Zs?$) occurs exactly once elsewhere.
2. **Fairness:** The clause ordering combined with committed choice ensures that when both streams have elements, they are alternately selected due to the argument swap in the recursive call.
3. **Stream Processing:** The program handles infinite streams correctly—it never requires the entire input to be available before producing output.

1.7 Security Through Cryptography

GLP integrates cryptographic security at the language level:

- **Identity:** Participants are identified by public/private keypairs
- **Encryption:** Messages between agents are encrypted
- **Signatures:** Messages are signed for authenticity
- **Attestation:** Code integrity is verified through attestations

These mechanisms enable both cold calls (for bootstrap and connecting disconnected components) and friend-mediated introductions (the preferred trust propagation method).

1.8 Book Overview

This book presents GLP progressively:

Part I: Foundations We begin with logic programs, then extend them to concurrent GLP, establishing the core concepts of reader/writer pairs, SRSW, and committed choice. We conclude with programming using constants to illustrate fundamental dataflow.

Part II: Concurrent Programming We develop stream-based concurrent programming, recursive techniques adapted to SRSW, and metaprogramming—programs that interpret and transform other programs.

Part III: Simulating Multiagent Systems We present the culmination: protocols for the grassroots social graph, social networking, interlaced streams, cryptocurrencies, and constitutional consensus—all executing on a single GLP runtime simulating distributed agents.

Each chapter builds on previous material, with formal content in shaded boxes for readers who want rigorous foundations.

Let us begin our journey into the art of grassroots logic programming.

Logic Programs and Linear Logic

Here we introduce transition systems, providing the formal framework for the operational semantics of both Logic Programs and GLP. We recall standard Logic Programs (LP): syntax, most-general unifier (mgu), operational semantics via nondeterministic goal/clause reduction, compositional semantics, and a proof that LP computations are deductions.

2.1 Logic Programs

Transition Systems

We use $\subset$ to denote the strict subset relation, $\subseteq$ when equality is also possible, and $a \neq b \in S$ as a shorthand for $a \neq b \wedge a \in S \wedge b \in S$. The following definition uses ‘configuration’ rather than the more standard ‘state’ to avoid confusion with the ‘local state’ of agents in a multiagent transition system, Definition ??.

Definition 2.1 (Transition System). A *transition system* is a tuple $TS = (C, c_0, T)$ where:

- C is an arbitrary set of *configurations*
- $c_0 \in C$ is a designated *initial configuration*
- $T \subseteq C \times C$ is a *transition relation*. A transition $(c, c') \in T$ is also written as $c \rightarrow c' \in T$.

A transition $c \rightarrow c' \in T$ is *enabled* from configuration c . A configuration c is *terminal* if no transitions are enabled from c . A *computation* is a (finite or infinite) sequence of configurations where for each two consecutive configurations (c, c') in the sequence, $c \rightarrow c' \in T$. A *run* is a computation

starting from $c0$, which is **complete** if it is infinite or ends in a terminal configuration.

Logic Programs Syntax

Definition 2.2 (Terms). • A **term** is a variable, a number, or $f(T_1, T_2, \dots, T_n)$ where f is a string (called the **functor**), $n \geq 0$ is the **arity**, and each T_i is a term. A term with arity 0 is identified with its functor. Let $\mathcal{T}$ denote the set of all terms.

- A **variable** is an alphanumeric string beginning with an uppercase letter or underscore `_` (e.g., `X`, `X1`, `Xs`, `_`, `_Result`). Let V denote the set of all variables.
- A **number** is a numeric literal, optionally signed, which may include a decimal point (e.g., `0`, `-1`, `3.14`).
- A **string** is an alphanumeric string beginning with a lowercase letter (e.g., `a`, `foo`, `mergeSort`), or a quoted string (e.g., `'`, `'`, `'Hello World'`).
- A term is **ground** if it contains no variables.

Definition 2.3 (Logic Programs). • A **goal** is a term $f(T_1, T_2, \dots, T_n)$ where f is a string, $n \geq 0$, and each T_i is a term. A goal is **conjunctive** if its functor is `'`, `'` and arity is 2; otherwise it is a **unit goal**. Let $\mathcal{G}$ denote the set of all goals.

- A **clause** is a term of the form $A :- B$ (read “A if B”), where A is a goal called the clause’s **head**, and B is a goal called the clause’s **body**. If B is the unit goal `true`, the clause is called a **unit clause** and may be written simply as A . The underscore `_` is a **don’t-care variable** that stands for a distinct variable occurring only once.
- A **logic program** is a finite sequence of clauses. By convention, clauses with the same functor and arity in their heads are grouped together and called a **procedure**.

Definition 2.4 (Auxiliary Notation). • A **constant** is a string or a number.

- **Lists:** The term `[]` represents the empty list. The term `'.'` (X, Xs) , written `[X|Xs]`, represents a list with first element X and rest Xs . The notation `[X]` abbreviates `[X|[]]`, and `[X1, X2, ..., Xn]` abbreviates the nested term `[X1[X2—...[Xn—[]]...]]—`.

We recall the quintessential logic program for list concatenation as an example:

Example 2.1 (Append). `append([X|Xs], Ys, [X|Zs]) :- append(Xs, Ys, Zs).`
`append([], Ys, Ys).`

Logically, a logic program clause $A:-B$ is a universally-quantified implication in which B implies A , and a program is a conjunction of its clauses. By convention, we use plural variable names like $\mathbf{Xs}$ to denote a list of $\mathbf{X}$'s.

Logic Programs Operational Semantics

Definition 2.5 (Substitution, Instance, Unifier, Most-General Unifier). A **substitution** σ is an idempotent function $\sigma : V \rightarrow \mathcal{T}$, namely a mapping from variables to terms applied to a fixed point. By convention, $\sigma(x) = x\sigma$.

- Given a substitution σ , $V_\sigma := \{X \in V \mid X\sigma \neq X\}$.
- Given a term $T \in \mathcal{T}$ and a substitution σ , $T\sigma$ is the term obtained from T by replacing every variable $X \in T$ by the term $X\sigma$.
- The partial order on terms $\preceq \subset \mathcal{T} \times \mathcal{T}$ is defined by $T \preceq T'$, or T' is an **instance** of T , if there is a substitution σ for which $T\sigma = T'$. If $T \neq T\sigma$ we say that σ **instantiates** T .
- For substitutions σ and σ' , $\sigma \preceq \sigma'$ if for every $T \in \mathcal{T}$, $T\sigma \preceq T\sigma'$, σ is **as general as** σ' if $\sigma \preceq \sigma'$.
- A substitution σ is a **unifier** of two terms $T, T' \in \mathcal{T}$ if $T\sigma = T'\sigma$; it is a **most-general unifier (mgu)** of T, T' if in addition it is as general as any other unifier of T and T' .

Namely, a most general unifier is frugal in not instantiating variables more than necessary.

Remark 2.1 (Substitution as Assignment Set). We view a substitution σ equivalently as a set of assignments $\{X_1 := T_1, \dots, X_n := T_n\}$ where $X_i\sigma = T_i$ and $T_i = T_i\sigma = T$. Thus the singleton substitution mapping X to T is $\{X := T\}$, its application $T\sigma$ may be written $T\{X := T\}$, the empty substitution is $\emptyset$, and composition of commutative substitutions corresponds to set union.

Definition 2.6 (Renaming, Rename apart). A **renaming** is a substitution $\sigma : V \mapsto V$ that maps variables to variables. A renaming σ renames T' **apart from** T if $T'\sigma$ and T have no variable in common.

We assume a fixed renaming-apart function, so that the result of renaming T' apart from T is well defined. Next we define the operational semantics of Logic Programs via a transition system.

Definition 2.7 (LP Goal/Clause Reduction). Given LP goal A and clause C , with $H:-B$ being the result of renaming C apart from A , the **LP reduction** of A with C **succeeds with result** (B, σ) if A and H have an mgu σ , else **fails**.

Definition 2.8 (Logic Programs Transition System). *A transition system $LP = (C, c0, T)$ is a **Logic Programs transition system** for a logic program M and initial goal $G_0 \in \mathcal{G}(M)$ if $C = \mathcal{G}(M)$, $c0 = G_0$, and T is the set of all transitions $G \rightarrow G' \in \mathcal{G}(M)^2$ such that for some unit goal $A \in G$ and clause $C \in M$ the LP reduction of A with C succeeds with result (B, σ) , and $G' = (G \setminus \{A\} \cup B)\sigma$.*

We write $G \xrightarrow{\sigma} G'$ when we want to make the substitution of a reduction explicit. As a tribute to resolution theorem proving [8]—the intellectual ancestor of logic programming—a configuration of LP is also referred to as a *resolvent*.

Logic Programs have two forms of nondeterminism: the choice of $A \in G$, called *and-nondeterminism*, and then choice of $C \in M$, called *or-nondeterminism*. Thus, as an abstract model of computation, LP are closely-related to *Alternating Turing Machines*, a generalization of Nondeterministic Turing Machines [9].

Definition 2.9 (Proper Run and Outcome). *A run $\rho : G_0 \xrightarrow{\sigma_1} G_1 \xrightarrow{\sigma_2} \dots \xrightarrow{\sigma_n} G_n$ of LP is **proper** if for any $1 \leq i < n$, a variable that occurs in G_{i+1} but not in G_i also does not occur in any G_j , $j < i$. If proper, the **outcome** of ρ is $(G_0 :- G_n)\sigma$ where $\sigma = \sigma_1 \circ \sigma_2 \circ \dots \circ \sigma_n$.*

It so happens that the set of all outcomes of all proper runs of a logic program constitutes its fully-abstract compositional semantics [3]. Next we prove the key safety property of LP:

Proposition 2.10 (LP Computation is Deduction). *The outcome $(G_0 :- G_n)\sigma$ of a proper run $\rho : G_0 \xrightarrow{\sigma_1} G_1 \xrightarrow{\sigma_2} \dots \xrightarrow{\sigma_n} G_n$ of LP , where $\sigma = \sigma_1 \circ \sigma_2 \circ \dots \circ \sigma_n$, is a logical consequence of M .*

2.2 Linear Logic

Linear logic, introduced by Girard [4], refines classical and intuitionistic logic by restricting the *structural rules* of contraction and weakening.

Definition 2.11 (Structural Rules). *The **structural rules** of classical logic are:*

- **Contraction:** *From A one may derive $A \wedge A$. An assertion may be used multiple times.*
- **Weakening:** *From $A \wedge B$ one may derive A . An assertion may be discarded unused.*

Linear logic restricts both rules: each assertion must be used *exactly once*—no duplication (contraction) and no discarding (weakening). This yields

a *resource interpretation*: assertions are consumable resources rather than eternal truths [5].

Definition 2.12 (Linear Logic). *Linear logic* is a substructural logic in which:

1. *Contraction is not admissible: an assertion A cannot be duplicated.*
2. *Weakening is not admissible: an assertion A cannot be discarded.*

The **exponential modality** $!A$ (read “of course A ”) marks assertions that may be contracted and weakened, recovering classical expressiveness where needed [4].

Linear implication $A \multimap B$, read “consuming A yields B ,” captures the resource interpretation: after using A to derive B , the assertion A is no longer available.

Example 2.2 (Resource Interpretation). Let D represent “one dollar” and C represent “one coffee.” The linear implication $D \multimap C$ represents a vending machine: consuming one dollar yields one coffee. Unlike classical logic, $D \multimap C$ does not entail $D \multimap (C \otimes D)$ —one cannot obtain both the coffee and keep the dollar.

The connection between linear logic and type systems is well established [11, 12]. A **linear type** ensures that a value is used exactly once: it cannot be duplicated (no aliasing) and cannot be discarded (no resource leaks). Related notions include **uniqueness types** [1], which guarantee a value has no other references, and **affine types**, which permit discarding but not duplication.

GLP as Linear Logic Programming

GLP realizes the linear discipline in logic programming by applying it to *equality assertions*—the fundamental operation of unification.

In standard LP, unification may assert multiple equalities involving the same variable. For example, unifying $p(X, X)$ with $p(a, a)$ asserts both $X = a$ (from the first argument) and $X = a$ (from the second). The unification algorithm reconciles these assertions, which in general requires checking consistency across all occurrences. This corresponds to *contraction*: the variable X is used multiple times.

In GLP, the SO invariant restricts each variable to at most one occurrence in any resolvent. This eliminates contraction: each variable participates in exactly one equality assertion. The distinction between writer X and reader $X?$ makes this precise:

- A *writer* X is the target of an equality assertion $X := T$. By SO, it occurs exactly once, so there is exactly one such assertion.

- A *reader* $X?$ observes the value established by its paired writer. By SO, it also occurs exactly once, so there is exactly one observer.

This corresponds directly to the linear discipline:

- *No contraction*: A variable cannot be duplicated. Each writer and each reader occurs exactly once.
- *No weakening*: A variable cannot be discarded unused. Each writer must have a paired reader (enforced by SRSW), and each reader eventually receives a value or the computation suspends.

The $\mathbf{ground}(X)$ guard provides the equivalent of the exponential modality $!A$: when a term is known to be ground (containing no variables), it may be safely duplicated because it carries no linear resources.

Formal 2.1: Linear Equality Assertions

The correspondence between GLP and linear logic:

Linear Logic	GLP
Assertion used exactly once	Variable occurs exactly once (SO)
No contraction	Single writer, single reader (SRSW)
No weakening	Paired reader/writer required
$!A$ (of course)	$\mathbf{ground}(X)$ guard
Linear implication $A \multimap B$	Communication: writer $\rightarrow$ reader

The practical consequence is that GLP eliminates distributed unification. In LP, reconciling multiple equality assertions on the same variable requires coordination across all occurrences—in a distributed setting, this requires atomic transactions [6]. In GLP, each equality assertion involves a unique writer, and communication is point-to-point from writer to reader. No coordination is needed because there are no conflicts to resolve.

GLP Core

3.1 GLP

We present GLP as an extension and simplification of Logic Programs: The syntax is extended with reader variables $X?$, where X and $X?$ form a reader/writer pair, and with the Single-Reader/Single-Writer syntactic restriction on clauses. Term matching replaces unification. For example, here is the quintessential concurrent logic program for merging two streams (incrementally constructed, potentially unbounded lists), written in GLP. Its first two arguments are the input streams to be merged, the third is the merged output stream:

Program 3.1: GLP Fair Stream Merger

```
merge([X|Xs],Ys,[X?|Zs?]) :- merge(Ys?,Xs?,Zs). % output from first stream
merge(Xs,[Y|Ys],[Y?|Zs?]) :- merge(Xs?,Ys?,Zs). % output from second stream
merge([],[],[]). % terminate on empty streams
```

Note that in each clause, each reader or writer occurs exactly once.

The operational semantics of GLP extends and simplifies that of LP as follows:

1. **Term matching instead of unification:** In LP, unification computes a most-general unifier that may bind any variable on either side. In GLP, term matching is a simpler operation: it assigns values only to writers, suspends when a reader requires a value not yet available, and fails when terms are structurally incompatible or when two variables of the same kind meet. The occurs check is not performed, as it is infeasible in a multi-agent setting. Circular terms may form through cross-goal communication; implementations must handle them gracefully.
2. **Synchronization:** Term matching may only assign writers, so in addition to succeed/fail, it may suspend if it requires assigning readers.

3. **Communication:** When term matching assigns a writer X to a term T , the message $X? := T$ encoding its paired reader assignment is created and added to the configuration. Its application happens asynchronously, realizing a message T from the single occurrence of X to the single occurrence of $X?$.
4. **Deterministic clause selection:** The first applicable clause is chosen, not nondeterministically as in LP. This provides for the fairness of **merge** presented above: As long as the two input streams are available the output dovetails the two inputs, due to switching their order in the recursive call of the first clause; as long as only one stream is available, its elements are copied to the output; and when both streams are unavailable the goal suspends.

The remainder of this section presents GLP syntax, nondeterministic operational semantics, and safety properties. A deterministic ‘workstation implementation-ready’ transition-system specification for GLP is presented in Appendix ??.

GLP Syntax

Reader/Writer pairs GLP extends Logic Programs with paired reader/writer variables, where a *writer* X is a single-assignment variable (promise) and its *paired reader* $X?$ provides read-only access to the (future) value of X . We denote by V the set of all writers, $V?$ the set of all readers and, $\mathcal{V} = V \cup V?$ the set of all variables, where for each writer $X \in V$ there exists a paired reader $X? \in V?$. We view $?$ as an identity suffix operator on non-writers, namely $(X?)? = X?$ for $X? \in V?$ and $T? = T$ for $T \notin \mathcal{V}$. We use $\mathcal{G}_?$ to denote the set of all goals over $\mathcal{V}$, and for a GLP program M , $\mathcal{G}_?(M)$ denotes the subset restricted to the vocabulary of M .

Single-Occurrence (SO) Invariant The fundamental requirement in GLP is that any variable may occur at most once in any resolvent. This *single-occurrence (SO) invariant* ensures there can be no conflict when assigning a writer, and no duplication when reading from a reader.

SRSW Syntactic Restriction on Clauses A clause satisfies the *single-reader/single-writer (SRSW) syntactic restriction* if every writer occurs exactly once and every reader occurs exactly once in the clause.

The SO invariant is maintained by the SRSW syntactic restriction: given a resolvent satisfying SO, reducing a goal with a clause satisfying SRSW produces a new resolvent satisfying SO.

This restriction excludes programs like the equality definition $X=X$ as it has two occurrences of the writer X . At the same time it eliminates the need for distributed atomic unification [6]—replacing it with efficient point-to-point

communication of a single assignment from the single occurrence of a writer to the single occurrence of its paired reader.

GLP Operational Semantics

Definition 3.1 (Writers Assignment, Readers Counterpart). A **writers assignment** σ is a set of assignments $\{X_1 := T_1, \dots, X_k := T_k\}$ where each $X_i \in V$ is a writer.

A **readers assignment** σ is a set of assignments where each variable in the domain is a reader.

If σ is a writers assignment then its **readers counterpart** is the readers assignment $\sigma?$ defined by $X?\sigma? = X\sigma$ for every $X \in \text{dom}(\sigma)$.

Definition 3.2 (Term Matching). Given two terms T_1 and T_2 that jointly satisfy the SO assumption (every variable occurs at most once in T_1 and T_2 combined), **term matching** of T_1 with T_2 produces one of three outcomes: **succeed** with a writers assignment, **suspend**, or **fail**. The outcome is determined by the following table, where X_1, X_2 denote writers, $X_1?, X_2?$ denote readers, and $f/n, f'/n'$ denote terms with functor f, f' and arity n, n' :

$T_1 \setminus T_2$	Writer X_2	Reader $X_2?$	Term f'/n'
Writer X_1	fail	$X_1 := X_2?$	$X_1 := T_2$
Reader $X_1?$	$X_2 := X_1?$	fail	suspend
Term f/n	$X_2 := T_1$	suspend	see below

For matching term f/n with term f'/n' : if $f \neq f'$ or $n \neq n'$, the match fails. Otherwise, recursively match all n pairs of corresponding arguments.

Outcomes combine as follows: if any match fails, the overall match fails; otherwise, if any match suspends, the overall match suspends; otherwise, the overall match succeeds with the union of all writers assignments.

Remark 3.1. Writer-to-writer matching (X_1 vs X_2) fails because, by the SO invariant, no other occurrences of X_1 or X_2 exist to assign them values, leaving their paired readers $X_1?$ and $X_2?$ abandoned.

Reader-to-reader matching ($X_1?$ vs $X_2?$) fails because neither can be assigned, and they cannot be equated without violating SO.

Renaming (Definition 2.6) is extended to respect variable pairing:

Definition 3.3 (GLP Renaming). Two GLP terms T, T' have a variable in common if for some writer $X \in V$, either X or $X?$ occur in T and either X or $X?$ occur in T' . A **GLP renaming** is a renaming substitution $\rho : V \mapsto V$ such that for each $X \in V$: $X\rho \in V$ and $X?\rho = (X\rho)?$.

Definition 3.4 (GLP Goal/Clause Reduction). Given GLP unit goal A and clause C , with $H :- B$ being the result of the GLP renaming of C apart from A ,

the **GLP reduction** of A with C fails or suspends if the term matching of A and H fails or suspends, respectively, and succeeds with result $(B\sigma, \sigma?)$ if the term matching succeeds with writers assignment σ .

The GLP operational semantics is defined via the following transition system, which employs the notions defined above to extend LP (Definition 2.8). It abstracts-away goal suspension and failure; these are used in the implementation-ready specifications (Appendixes ?? and ??) for explicit goal scheduling, suspension and activation.

Definition 3.5 (GLP Transition System). *Given a GLP program M , an **asynchronous resolvent** over M is a pair (G, σ) where $G \in \mathcal{G}_?(M)$ and σ is a readers assignment.*

*A transition system $GLP = (\mathcal{C}, c0, \mathcal{T})$ is a **GLP transition system** over M and initial goal $G_0 \in \mathcal{G}_?(M)$ satisfying SO if:*

1. $\mathcal{C}$ is the set of all asynchronous resolvents over M
2. $c0 = (G_0, \emptyset)$
3. $\mathcal{T}$ is the set of all transitions $(G, \sigma) \rightarrow (G', \sigma')$ satisfying:
 - a) **Reduce:** there exists a unit goal $A \in G$ such that $C \in M$ is the first clause for which the GLP reduction of A with C succeeds with result $(B, \hat{\sigma})$, $G' = (G \setminus \{A\} \cup B)\hat{\sigma}$, and $\sigma' = \sigma \circ \hat{\sigma}$
 - b) **Communicate:** $\hat{\sigma} = \{X? := T\} \in \sigma$, $G' = G\hat{\sigma}$, and $\sigma' = \sigma \setminus \hat{\sigma}$

The monotonicity of GLP goal/clause reduction (Proposition 3.10) allows a simple *GLP fairness requirement*: A goal that can be reduced is eventually reduced.

Worked Examples

Example 1: Success Goal $g = p(X, a)$, Head $h = p(b, Y)$ (after renaming).

Term matching: X matches b , a matches Y . Both are writer-to-constant. Writers assignment: $\{X \mapsto b, Y \mapsto a\}$. Succeeds.

Example 2: Suspension Goal $g = p(X?, a)$, Head $h = p(b, Y)$.

First argument: $X?$ (goal reader) against b (constant). Readers cannot be assigned, so term matching suspends, waiting for the paired writer X to be assigned.

Example 3: Failure Goal $g = p(a, b)$, Head $h = p(a, c)$.

First argument: $a = a$, succeeds. Second argument: $b \neq c$, term matching fails.

Example 4: Writer-to-Writer Failure Goal $g = p(X)$, Head $h = p(Y)$.
Both X and Y are writers. Writer-to-writer matching fails (not suspends).

Guards and system predicates GLP also includes *guards*—predicates that test runtime conditions (e.g., $\text{ground}(X)$ tests if X contains no variables) without modifying state, appearing after clause heads separated by $|$ —and *system predicates* that provide access to the GLP runtime state and operating system and hardware capabilities (variable state and name, arithmetic evaluation, timestamps). Guards enable conditional clause selection. The $\text{ground}(X)$ guard allows relaxing the single-reader constraint for $X?$ for the clause it occurs in, as having multiple occurrences of $X?$ instantiated to a ground term does not violate the fundamental single-writer requirement. Their specification appears in Appendix ??.

GLP Safety

Here we prove that, like LP, GLP computations are deductions, but, unlike LP, a goal that can be reduced in a configuration can still be reduced in any subsequent configuration of the computation.

GLP computations are deductions First we show that the extensions of GLP over LP do not take it outside of the logic programming realm.

Definition 3.6 (Pure Logic Variant). *Given a GLP term or goal T , the **pure logic variant** $L(T)$ of T is defined by replacing every reader $X?$ in T with its paired writer X . Given a GLP computation r , its pure logic variant $L(r)$ is the result of replacing every configuration (G, σ) in r by $L(G)$, removing duplications and labelling the remaining transitions by the mgu of their respective reduction.*

Note that duplications as above result from Communicate transitions.

Proposition 3.7 (GLP Computations are Deductions). *For any finite GLP run r , let $L(r) = G_0 \xrightarrow{\sigma_1} G_1 \xrightarrow{\sigma_2} \dots G_n$, with $\sigma = \sigma_1 \cdot \dots \cdot \sigma_n$, then $(G_0 : -G_n)\sigma$ is a logical consequence of $L(M)$.*

Next, we establish essential safety properties for GLP that distinguish it from standard LP. The key is monotonicity—once a goal becomes reducible in GLP, it remains reducible.

SO Invariant

Proposition 3.8 (SO Invariant). *If the initial goal G_0 in a GLP run satisfies SO, then every goal in the run satisfies SO.*

Circular Terms Circular terms may form through cross-goal communication. Consider the clause $p(X?, X)$ and goals $p(X, f(Y?))$, $p(Y, f(X?))$. After both reductions, X is assigned $f(Y?)$ and Y is assigned $f(X?)$, yielding the circular term $X = f(f(X?))$. In a multiagent setting these goals may execute on different agents, making any occurs check infeasible. Implementations must handle circular terms gracefully.

Formal 3.1: Circular Term Semantics

Circular terms arise when a variable appears (via its reader) within its own assigned value. The following operations must handle circular terms:

Dereferencing. Following a chain of assignments must detect cycles and terminate. When a cycle is detected, dereferencing returns the circular structure itself rather than diverging.

Ground test. A term is **ground** if it contains no variables. A circular term such as $f(f(f(\dots)))$ contains no variables and is therefore ground. A term like $f(g(X?), f(f(\dots)))$ where $X?$ is unbound is not ground, regardless of the cycle in its second argument.

Equality test. Two terms are equal if they have identical structure. For circular terms, this requires comparing corresponding positions while tracking pairs already visited. If two positions in the respective terms both point back to previously compared positions, they are equal at that point.

Term copying. Copying a circular term produces a circular copy with the same structure. The cycle in the copy is independent of the original.

Term display. Displaying a circular term must detect the cycle and use a finite representation, such as indicating the cycle point.

Example 3.1 (Circular Term Formation). *Given clause $p(X?, X)$ and query $p(X, f(Y?))$, $p(Y, f(X?))$:*

1. *Goal $p(X, f(Y?))$ reduces with clause $p(X?, X)$. Term matching assigns $X := f(Y?)$.*
2. *Goal $p(Y, f(X?))$ reduces with clause $p(X1?, X1)$ (renamed). Term matching assigns $Y := f(X?)$.*
3. *Applying the readers counterpart: $Y?$ receives $f(X?)$ and $X?$ receives $f(Y?)$.*
4. *Result: $X = f(Y?) = f(f(X?))$ — a circular term.*

The term X is ground (contains no unbound variables) despite being circular.

Monotonicity Unlike LP where variable instantiation can cause a previously reducible goal to fail, GLP exhibits monotonicity. In a run, if a goal A can be reduced at some point, it remains reducible at all future points in that run, where “future” implies that readers in A (and only readers) have been further instantiated by other goal reductions.

Lemma 3.9 (Reader-Instance). *In any GLP run $G_0 \rightarrow G_1 \rightarrow \dots$, if $G_i \rightarrow G_{i+1}$ via reduction with readers assignment σ does not reduce $A \in G_i$, then $A\tau \in G_{i+1}$ where τ instantiates only readers.*

Proposition 3.10 (Monotonicity). *In any GLP run $G_0 \rightarrow G_1 \rightarrow \dots$, if unit goal $A \in G_i$ can reduce with clause C , then for any $j > i$, either A has been reduced by step j , or there exists $A' \in G_j$ where $A' = A\tau$ for some readers assignment τ , and A' can reduce with C .*

3.2 Guards

Guards are predicates that test conditions without side effects. A clause is applicable only if its guard succeeds. Guards appear between the clause head and the body, separated by $|$:

Head $:-$ Guard $|$ Body.

If the guard suspends (e.g., tests an unbound variable), the clause is not yet applicable. If the guard fails, the next clause is tried. If the guard succeeds, the clause commits and executes its body.

Built-in Guards

Type Guards Type guards test the structure of a value:

- **ground(X)** — succeeds if X contains no unbound variables
- **known(X)** — succeeds if X is bound (may contain unbound subterms)
- **unknown(X)** — succeeds if X is an unbound variable
- **integer(X), number(X), string(X), constant(X)** — test for specific types
- **compound(X), tuple(X)** — test for compound terms
- **is_list(X)** — tests for proper list structure

The **ground(X)** guard is special: it relaxes the single-reader constraint for X ? within the clause, permitting multiple reader occurrences since a ground term contains no writers.

Arithmetic Guards Arithmetic guards compare numeric values:

- $X ::= Y$ — numeric equality
- $X < Y, X > Y, X \leq Y, X \geq Y$ — comparisons

These suspend if either operand is unbound.

Equality Guard The guard $X == Y$ tests ground equality: succeeds if both are ground and equal, fails if both are ground and different, suspends if either contains unbound readers.

Control Guards The **otherwise** guard succeeds when all previous clauses for the predicate failed (not suspended). This enables default case handling.

Guard Negation

Any negatable guard G can be negated using $\sim G$. The negated guard succeeds when G fails, fails when G succeeds, and suspends when G suspends.

```
lookup(Key, [(K,V)|_], V?) :- Key? == K? | true.
lookup(Key, [(K,_)|Rest], V?) :- ~(Key? == K?) | lookup(Key?, Rest?, V).
```

Type guards and $==$ are negatable. Arithmetic comparisons and **otherwise** are not.

Defined Guards

A *defined guard* is a user-defined predicate that can be called in guard position. It must be a *unit clause*—a clause with no guards and no body (or body **true**).

Type Tests A unit clause can define a type test:

```
channel(ch(_, _)).
```

When called in guard position, the pattern is matched against the argument:

```
process(X, ok) :- channel(X?) | handle(X?).
process(_, error) :- otherwise | true.
```

The compiler transforms this by *partial evaluation*—unfolding the unit clause at compile time:

```
process(ch(_, _), ok) :- handle(ch(_, _)?).
process(_, error) :- otherwise | true.
```

Channel Operations Channel operations are defined as unit clauses:

```
send(X, ch(In, [X?|Out?]), ch(In?, Out)).
receive(X?, ch([X|In], Out?), ch(In?, Out)).
new_channel(ch(Xs?, Ys), ch(Ys?, Xs)).
```

These enable abstract channel manipulation in guards. Consider a relay that forwards messages from a stream to a channel:

```
relay([X|In], Out?, Ch) :- send(X?, Ch?, Ch1) | relay(In?, Out, Ch1?).
relay(In?, [X?|Out?], Ch) :- receive(X, Ch?, Ch1) | relay(In, Out, Ch1?).
relay([], [], ch([], [])).
```

The first clause sends stream element X to the channel. The second receives from the channel to the output stream. The compiler unfolds the defined guards:

```
relay([X|In], Out?, ch(ChIn, [X?|ChOut?])) :- relay(In?, Out, ch(ChIn?, ChOut)).
relay(In?, [X?|Out?], ch([X|ChIn], ChOut?)) :- relay(In, Out, ch(ChIn?, ChOut)).
relay([], [], ch([], [])).
```

The abstract form is clearer; the expanded form shows what actually executes.

Channel Allocation The `new_channel` guard allocates a cross-linked channel pair:

```
make_pair(C1?, C2?) :- new_channel(C1, C2) | true.
```

Variables `C1` and `C2` are writers in the guard (allocated there) and readers in the head (returned to caller). After partial evaluation:

```
make_pair(ch(X1?, X2), ch(X2?, X1)).
```

The channels are cross-linked: what is written to one is read from the other.

Response Binding The cold-call protocol uses `new_channel` to create a response:

```
bind_response(yes, accept(RemoteCh?), LocalCh?) :-
    new_channel(LocalCh, RemoteCh) | true.
bind_response(no, reject, none).
```

When accepting, cross-linked channels are allocated. `RemoteCh` is sent to the caller; `LocalCh` is kept locally. After partial evaluation:

```
bind_response(yes, accept(ch(X2?, X1)), ch(X1?, X2)).
bind_response(no, reject, none).
```

SRSW Rules for Defined Guards

When a guard allocates variables (like `new_channel`), those variables are writers in the guard and may be readers elsewhere:

Position	Form	Meaning
Head	<code>X</code> (writer)	Receives value from caller
Head	<code>X?</code> (reader)	Returns value to caller
Guard	<code>X</code> (writer)	Guard provides this value
Guard	<code>X?</code> (reader)	Reads from head or earlier guard
Body	<code>X</code> (writer)	Body provides this value
Body	<code>X?</code> (reader)	Reads from head or guard

For allocation guards like `new_channel(C1, C2)`:

- `C1` and `C2` are writers in the guard (allocated there)
- They appear as readers in the head (`C1?`, `C2?`) to return to caller

The complete reference for guards appears in Appendix 12.10.

3.3 Exercises

1. Trace the full computation of `merge([1], [a], Out)` showing all configurations.
2. What happens if we query `merge(X, Y, [1,2,3])` with X and Y uninstantiated? Explain why.
3. Explain why monotonicity fails in standard LP. Give a concrete example where a reducible goal becomes unreducible.
4. Design a goal that never terminates but is fair (every subgoal gets reduced infinitely often).
5. Prove that if all goals in a GLP computation eventually become reducible, the computation is deadlock-free.

Basic Concurrent Programming

This chapter introduces GLP programming through progressively richer examples.

4.1 Programming with Constants

Unit Clauses

A *unit clause* is a clause with an empty body. The simplest GLP programs consist of unit clauses with constant arguments.

`p(a).`

This clause defines `p/1` with a single solution: `p(a)` holds.

Goal with matching constant.

```
GLP> p(a).  
p(a) :- true  
-> succeeds
```

Goal with non-matching constant.

```
GLP> p(b).  
-> fails
```

Goal with writer argument.

```
GLP> p(X) .
p(X) :- true
X = a
-> succeeds
```

The writer *X* receives the value *a* from the clause head.

Goal with reader argument.

```
GLP> p(X?) .
p(X?) -> suspended
```

A reader cannot receive a value—it can only observe a value produced elsewhere. With no writer to bind *X*, the goal suspends.

Conjunctive Goals

When goals share a variable, execution order affects behavior.

Writer then reader.

```
GLP> p(X), p(X?) .
p(X) :- true
p(a) :- true
X = a
-> succeeds
```

The first goal binds *X = a*. The second goal, with reader *X?*, succeeds since *p(a)* matches.

Reader then writer.

```
GLP> p(X?), p(X) .
p(X?) -> suspended
p(X) :- true
p(a) :- true
X = a
-> succeeds
```

The first goal suspends on the unbound reader. The second goal binds *X = a*. The first goal reactivates and succeeds.

Multiple Clauses

When multiple clauses define a predicate, the first applicable clause is selected:

```
q(b).
q(a).
```

Goal with writer argument.

```
GLP> q(X).
q(X) :- true
X = b
-> succeeds
```

With a writer argument, $q(X)$ commits to the first clause. Since $q(b)$ is first, X is bound to b .

Conjunctive Goals with Different Predicates

Combining $p/1$ and $q/1$ reveals how bindings propagate.

Writer from p , reader to q .

```
GLP> p(X), q(X?).
p(X) :- true
q(a) :- true
X = a
-> succeeds
```

The binding $X = a$ from $p(X)$ flows to $q(X?)$, which succeeds because $q(a)$ matches.

Reader from p , writer from q .

```
GLP> p(X?), q(X).
p(X?) -> suspended
q(X) :- true
p(b) -> fails
X = b
-> fails
```

The goal $p(X?)$ suspends. Then $q(X)$ binds $X = b$. When $p(X?)$ reactivates, it becomes $p(b)$, which fails.

Binary Unit Clauses

```
p(a,b).
```

Goal with two writers.

```
GLP> p(X,Y) .  
p(X,Y) :- true  
X = a  
Y = b  
-> succeeds
```

Shared Variables in Goals

```
p(a,a) .
```

Writer-reader order.

```
GLP> p(X,X?) .  
p(X,X?) :- true  
X = a  
-> succeeds
```

When X appears as both writer and reader in the same goal, the writer occurrence binds X , and the reader occurrence observes that binding.

Logic Gates

Digital logic gates provide a natural example of dataflow computation:

```
and(1,1,1) .  
and(1,0,0) .  
and(0,1,0) .  
and(0,0,0) .
```

```
or(1,1,1) .  
or(1,0,1) .  
or(0,1,1) .  
or(0,0,0) .
```

```
not(1,0) .  
not(0,1) .
```

```
xor(1,1,0) .  
xor(1,0,1) .  
xor(0,1,1) .  
xor(0,0,0) .
```



```
GLP> and(1,1,X).
and(1,1,X) :- true
X = 1
-> succeeds
```

Clauses with Bodies

A NAND gate inverts the output of an AND gate:

```
nand(A,B,Z?) :- and(A?,B?,W), not(W?,Z).
```

```
GLP> nand(1,1,X).
nand(1,1,X) :- and(1,1,W), not(W?,Z)
and(1,1,W) :- true
not(1,Z) :- true
X = 0
-> succeeds
```

Formal 4.1: Produces and Consumes Parameters

In GLP, a clause *produces* a value through a reader in the head, and *consumes* a value through a writer in the head.

Consider `nand(A,B,Z?) :- and(A?,B?,W), not(W?,Z).`

Produces The third parameter `Z?` is a reader in the head. The body contains the paired writer `Z`, which produces the output value.

Consumes The first two parameters `A` and `B` are writers in the head. The body contains the paired readers `A?` and `B?`, which consume the input values.

Summary

Parameter role	Head	Body
Produces	reader	writer
Consumes	writer	reader

These conventions follow from term matching (Definition 3.2) and the SRSW constraint.

Guards for Multiple Reader Occurrences

A half-adder takes two input bits and produces a sum and carry:

```
half_adder(A,B,Sum?,Carry?) :-
    ground(A?), ground(B?) |
    xor(A?,B?,Sum), and(A?,B?,Carry).
```

The `ground(A?)` and `ground(B?)` guards are necessary because `A?` and `B?` each appear twice in the body. The guards ensure both inputs are fully bound before the body executes, permitting multiple reader occurrences (see Formal Box 4.3).

```
GLP> half_adder(1,1,S,C).
half_adder(1,1,S,C) :- xor(1,1,S), and(1,1,C)
xor(1,1,S) :- true
and(1,1,C) :- true
S = 0
C = 1
-> succeeds
```

Compound Circuits

A full-adder adds three bits:

```
full_adder(A,B,Cin,Sum?,Cout?) :-
    half_adder(A?,B?,S1,C1),
    half_adder(S1?,Cin?,Sum,C2),
    or(C1?,C2?,Cout).
```

Each reader appears only once in the body, so no guards are needed.

```
GLP> full_adder(1,1,1,S,C).
...
S = 1
C = 1
-> succeeds
```

4.2 Streams

A *stream* is an incrementally-constructed list. Unlike a complete list that exists all at once, a stream is built element by element during computation.

- If a stream is *bounded* (will eventually close), the consumer should have a clause for `[]`.
- If a stream is *indefinite*, the consumer should *not* have a clause for `[]`—it should fail if the stream unexpectedly terminates.

Producers and Consumers

A producer that counts down from N :

```
producer([], 0).
producer([N?|Xs?], N) :- N? > 0 | N1 := N? - 1, producer(Xs, N1?).
```

A consumer that sums stream elements:

```
consumer([], Sum, Sum?).
consumer([X|Xs], Sum, Result?) :- ground(X?) |
    Sum1 := Sum? + X?,
    consumer(Xs?, Sum1?, Result?).
```

Formal 4.2: SRSW in Continuation Calls

The recursive call passes readers $Xs?$, $Sum1?$ —not writers.

Why Readers in Continuations? When the goal matches the head, the writer Xs receives a value. It cannot appear as writer again in the body. Its paired reader $Xs?$ must be used instead.

Principle: In continuation calls, pass readers for values already received.

Concurrent Execution

```
GLP> producer(H, 5), consumer(H?, 0, R).
producer(X1, 5) :- producer(X4, 4)
consumer([5 | X4?], 0, X2) :- consumer(X4?, 5, X6)
...
producer(X20, 0) :- true
consumer([], 15, X22) :- true
R = 15
-> succeeds
```

List Reversal

Naive Reverse

```
reverse([], []).
reverse([X|Xs], Ys?) :-
    reverse(Xs?, Zs?),
    append(Zs?, [X?], Ys?).

append([], Ys, Ys?).
```

```
append([X|Xs], Ys, [X?|Zs?]) :- append(Xs?, Ys?, Zs).
```

This is $O(n^2)$.

Reverse with Accumulator

```
reverse(Xs, Ys?) :- reverse_acc(Xs?, [], Ys).
```

```
reverse_acc([], Acc, Acc?).
```

```
reverse_acc([X|Xs], Acc, Ys?) :- reverse_acc(Xs?, [X?|Acc?], Ys).
```

Total work: $O(n)$.

```
GLP> reverse([a,b,c], Ys).
reverse([a,b,c], Ys) :- reverse_acc([a,b,c], [], Ys)
reverse_acc([a,b,c], [], Ys) :- reverse_acc([b,c], [a], Ys)
reverse_acc([b,c], [a], Ys) :- reverse_acc([c], [b,a], Ys)
reverse_acc([c], [b,a], Ys) :- reverse_acc([], [c,b,a], Ys)
reverse_acc([], [c,b,a], Ys) :- true
Ys = [c,b,a]
-> succeeds
```

Stream Merging

Simple Fair Merge

```
merge([X|Xs], Ys, [X?|Zs?]) :- merge(Ys?, Xs?, Zs).
merge(Xs, [Y|Ys], [Y?|Zs?]) :- merge(Xs?, Ys?, Zs).
merge([], Ys, Ys?).
merge(Xs, [], Xs?).
```

The key to fairness is swapping inputs: `merge(Ys?, Xs?, Zs)`.

The two base cases `merge([], Ys, Ys?)` and `merge(Xs, [], Xs?)` have two advantages over a single `merge([], [], [])` base case. First, merge terminates as soon as either stream closes, rather than requiring both to terminate—important when one stream may be indefinite. Second, when one stream closes, the entire remaining stream is copied to the output in a single step (`Ys?` or `Xs?`), rather than element-by-element through continued recursive calls.

```
GLP> merge([1,2,3], [a,b], Xs).
...
Xs = [1, a, 2, b, 3]
-> succeeds
```

Dynamic Merge

```

dmerge([merge(Ws)|Xs], Ys, Zs?) :-
    dmerger(Ws?, Xs?, Xs1), dmerge(Xs1?, Ys?, Zs).
dmerge(Xs, [merge(Ws)|Ys], Zs?) :-
    dmerger(Ws?, Ys?, Ys1), dmerge(Xs?, Ys1?, Zs).
dmerge([X|Xs], Ys, [X?|Zs?]) :-
    tuple(X?), ~(X? == merge(_)) | dmerge(Ys?, Xs?, Zs).
dmerge([X|Xs], Ys, [X?|Zs?]) :-
    constant(X?) | dmerge(Ys?, Xs?, Zs).
dmerge(Xs, [Y|Ys], [Y?|Zs?]) :-
    tuple(Y?), ~(Y? == merge(_)) | dmerge(Xs?, Ys?, Zs).
dmerge(Xs, [Y|Ys], [Y?|Zs?]) :-
    constant(Y?) | dmerge(Xs?, Ys?, Zs).
dmerge([], [], []).

dmerger(Ws, Xs, Out?) :- dmerge(Ws?, Xs?, Out).

```

Static Balanced Merge Tree

```

merge_tree([Xs], Xs?).
merge_tree([X,Y|Rest], Out?) :-
    merge_layer([X?,Y?|Rest?], Layer),
    merge_tree(Layer?, Out).

merge_layer([], []).
merge_layer([Xs], [Xs?]).
merge_layer([Xs,Ys|Rest], [Zs?|Layer?]) :-
    merge(Xs?, Ys?, Zs),
    merge_layer(Rest?, Layer).

```

Stream Distribution**Broadcast Distribution**

```

distribute([X|Xs], [X?|Ys?], [X?|Zs?]) :- ground(X?) | distribute(Xs?, Ys, Zs).
distribute([], [], []).

```

The `ground(X?)` guard permits replication.

Indexed Distribution

```

distribute_indexed([send(1,X)|In], [X?|Out1?], Out2) :-
    distribute_indexed(In?, Out1, Out2).
distribute_indexed([send(2,X)|In], Out1, [X?|Out2?]) :-
    distribute_indexed(In?, Out1, Out2).

```

```
distribute_indexed([], [], []).
```

Stream Observers

```
observer([X|Xs], [X?|Ys?], [X?|Zs?]) :- ground(X?) | observer(Xs?, Ys, Zs).
observer([], [], []).
```

Ripple-Carry Adder

An n-bit ripple-carry adder chains full-adders:

```
adder([], [], Cin, [Cin?]).
adder([A|As], [B|Bs], Cin, [S?|Ss?]) :-
    full_adder(A?, B?, Cin?, S, Cout),
    adder(As?, Bs?, Cout?, Ss?).
```

```
GLP> adder([1,0,1], [1,1,0], 0, R).
```

```
...
```

```
R = [0,0,0,1]
```

```
-> succeeds
```

The result `[0,0,0,1]` is 1011 in binary (LSB first), which is 11: $5+6 = 11$.

Buffered Communication

Sliding Window Buffer

```
bb :- consumer([X1?, X2? | Xs]), producer(1, [X1, X2 | Xs?]).
```

```
consumer([X1, X2, X3 | Xs?]) :- known(X1?) | consumer([X2?, X3? | Xs]).
```

```
producer(N, [N? | Xs]) :- number(N?) | N1 := N? + 1, producer(N1?, Xs?).
```

The buffer size (2) is the number of pre-allocated slots. The following trace uses a terminating version:

```
bb_test :- consumer([X1?, X2? | Xs], 10), producer(1, [X1, X2 | Xs?]).
```

```
consumer([X1, X2, X3 | Xs?], N) :- known(X1?), N? > 0 |
    N1 := N? - 1, consumer([X2?, X3? | Xs], N1?).
consumer(_, 0).
```

```
producer(N, [N? | Xs]) :- number(N?) | N1 := N? + 1, producer(N1?, Xs?).
```

```

GLP> bb_test.
consumer([X1? | [X2? | X3]], 10) -> suspended
producer(1, [X1 | [X2 | X3?]]) :- producer(2, [X2 | X3?])
consumer([1 | [X2? | X3]], 10) :- consumer([X2? | [X6? | X7]], 9)
producer(2, [X2 | [X6 | X7?]]) :- producer(3, [X6 | X7?])
consumer([2 | [X6? | X7]], 9) :- consumer([X6? | [X10? | X11]], 8)
producer(3, [X6 | [X10 | X11?]]) :- producer(4, [X10 | X11?])
...
consumer([11 | [X42? | X43]], 0) :- true
producer(12, [X42 | X43?]) :- producer(13, X43?)
producer(13, X43?) -> suspended
-> succeeds

```

The consumer suspends until the producer fills the first slot. Once running, they alternate: consumer processes N , producer writes $N + 2$. When the consumer terminates, the producer suspends—no demand for element 13.

Objects and Monitors

Processes maintaining state in recursive parameters are the GLP realization of *objects*. A *monitor* is an object accessed by multiple clients through a merge.

Counter

```

counter(In) :- counter_loop(In?, 0).

counter_loop([clear|In], _) :- counter_loop(In?, 0).
counter_loop([add|In], C) :- C1 := C? + 1, counter_loop(In?, C1?).
counter_loop([read(C?)|In], C) :- number(C?) | counter_loop(In?, C?).
counter_loop([], _).

GLP> counter([add, add, add, read(X), clear, add, read(Y), []]).
...
X = 3
Y = 1
-> succeeds

```

Formal 4.3: Which Guards Enable Multiple Reader Occurrences?

The counter's `read` clause uses `number(C?)` to enable replication. Here `C?` appears twice in the body.

The Danger of Replicating Readers If `C?`'s value contains an unbound writer, both occurrences would have access to it—violating single-writer.

Ground Guard The guard `ground(C?)` succeeds only if `C?` contains no unbound variables.

Type Guards Type guards like `constant(X?)`, `number(X?)`, and `integer(X?)` also enable multiple reader occurrences—these types cannot contain variables.

The known Guard Does Not Enable Multiple Reads The guard `known(X?)` only tests that `X?` is bound to *some* value—which may contain unbound variables.

Summary:

Guard	Multiple reads?
<code>ground(X?)</code>	Yes
<code>constant(X?)</code>	Yes
<code>number(X?)</code>	Yes
<code>integer(X?)</code>	Yes
<code>tuple(X?)</code>	No
<code>known(X?)</code>	No

Accumulator

```
accumulator(In) :- acc_loop(In?, 0).
```

```
acc_loop([add(N)|In], Sum) :- number(N?) |
    Sum1 := Sum? + N?, acc_loop(In?, Sum1?).
acc_loop([read(Sum?)|In], Sum) :- number(Sum?) | acc_loop(In?, Sum?).
acc_loop([], _).
```

Multiple clients through a merge:

```
test_acc :-
    merge(Client1?, Client2?, In),
    accumulator(In?),
    client1(Client1),
    client2(Client2).

client1([add(10), add(20), read(X) | []]) :- number(X?) | true.
client2([add(5), read(Y) | []]) :- number(Y?) | true.
```


4.3 Recursive Programming

We distinguish *tail recursion* (iteration by a single process) from *true recursion* (concurrent processes forming a computation tree).

Peano Arithmetic

Peano arithmetic represents natural numbers using zero and successor:

```
plus(0, Y, Y?).
plus(s(X), Y, s(Z?)) :- plus(X?, Y?, Z?).

times(0, _, 0).
times(s(X), Y, Z?) :- tuple(Y?) | times(X?, Y?, XY), plus(XY?, Y?, Z?).

lesseq(0, X) :- natural_number(X?) | true.
lesseq(s(X), s(Y)) :- lesseq(X?, Y?).

natural_number(0).
natural_number(s(X)) :- natural_number(X?).
```

Integer Arithmetic

```
double(X, Y?) :- Y := X? * 2.
average(X, Y, Avg?) :- Avg := (X? + Y?) / 2.

abs(X, X?) :- X? >= 0 | true.
abs(X, Y?) :- X? < 0 | Y := 0 - X?.

max(X, Y, X?) :- X? >= Y? | true.
max(X, Y, Y?) :- X? < Y? | true.
```

Recursive Numeric Functions

Factorial.

```
factorial(0, 1).
factorial(1, 1).
factorial(N, F?) :-
    N? > 1 |
    N1 := N? - 1,
    factorial(N1?, F1),
    F := N? * F1?.
```

Tail-Recursive Factorial.

```
factorial(N, F?) :- fact_acc(N?, 1, F).
```

```
fact_acc(0, Acc, Acc?).
fact_acc(N, Acc, F?) :-
    N? > 0 |
    Acc1 := Acc? * N?,
    N1 := N? - 1,
    fact_acc(N1?, Acc1?, F).
```

Fibonacci.

```
fib(0, 0).
fib(1, 1).
fib(N, F?) :-
    N? > 1 |
    N1 := N? - 1,
    N2 := N? - 2,
    fib(N1?, F1),
    fib(N2?, F2),
    F := F1? + F2?.
```

This spawns $O(2^N)$ processes.

Linear Fibonacci.

```
fib_linear(N, F?) :- fib_acc(N?, 0, 1, F).
```

```
fib_acc(0, A, _, A?).
fib_acc(N, A, B, F?) :-
    N? > 0 |
    N1 := N? - 1,
    AB := A? + B?,
    fib_acc(N1?, B?, AB?, F).
```

Flattening Nested Lists

```
flatten(Xs, Ys?) :- flatten_acc(Xs?, [], Ys).
```

```
flatten_acc([], Acc, Acc?).
flatten_acc([X|Xs], Acc, Ys?) :-
    ground(X?), is_list(X?) |
    flatten_acc(X?, Acc?, Acc1),
    flatten_acc(Xs?, Acc1?, Ys).
```

```
flatten_acc([X|Xs], Acc, Ys?) :-
    otherwise |
    flatten_acc(Xs?, [X?|Acc?], Ys).
```

Binary Trees

```
tree_sum(void, 0).
tree_sum(tree(V, L, R), S?) :-
    tree_sum(L?, SL),
    tree_sum(R?, SR),
    S := V? + SL? + SR?.
```

The two recursive calls spawn concurrently.

Insertion Sort

```
insertion_sort([], []).
insertion_sort([X|Xs], Sorted?) :-
    insertion_sort(Xs?, SortedTail),
    insert(X?, SortedTail?, Sorted).

insert(X, [], [X?]).
insert(X, [Y|Ys], [X?|Y?|Ys?]) :- X? < Y? | true.
insert(X, [Y|Ys], [Y?|Zs?]) :- X? >= Y? | insert(X?, Ys?, Zs?).
```

Merge Sort

```
mergesort([], []).
mergesort([X], [X?]).
mergesort([A,B|Xs], Sorted?) :-
    split2(A?, B?, Xs?, Left, Right),
    mergesort(Left?, SortedL),
    mergesort(Right?, SortedR),
    merge_sorted(SortedL?, SortedR?, Sorted).

split2(A, B, [], [A?], [B?]).
split2(A, B, [X], [A?,X?], [B?]).
split2(A, B, [X,Y|Xs], [A?|Left?], [B?|Right?]) :-
    split2(X?, Y?, Xs?, Left, Right).

merge_sorted([], Ys, Ys?).
merge_sorted(Xs, [], Xs?).
merge_sorted([X|Xs], [Y|Ys], [X?|Zs?]) :-
    number(X?), number(Y?), X? <= Y? |
    merge_sorted(Xs?, [Y?|Ys?], Zs?).
```

```

merge_sorted([X|Xs], [Y|Ys], [Y?|Zs?]) :-
    number(X?), number(Y?), X? > Y? |
    merge_sorted([X?|Xs?], Ys?, Zs?).

GLP> mergesort([3,1,4,1,5,9,2,6], S).
...
S = [1,1,2,3,4,5,6,9]
-> succeeds

```

Non-Ground Stream Distributor

For messages containing unbound subterms:

```

distribute_ng([X|Xs], [Y?|Ys?], [Z?|Zs?]) :-
    copy(X?, Y, Z),
    distribute_ng(Xs?, Ys, Zs).
distribute_ng([], [], []).

copy(X, Y, Z) :- constant(X?) | Y = X?, Z = X?.
copy(X, Y, Z) :- number(X?) | Y = X?, Z = X?.
copy(X, Y, Z) :- compound(X?) |
    X? =.. [F|Args],
    copy_list(Args?, Args1, Args2),
    Y =.. [F?|Args1?],
    Z =.. [F?|Args2?].

copy_list([], [], []).
copy_list([H|T], [H1?|T1?], [H2?|T2?]) :-
    copy(H?, H1, H2),
    copy_list(T?, T1, T2).

```

If a subterm is unbound, none of the guards succeed, so the goal suspends. When the producer later binds the subterm, copying proceeds.

Tree Substitution

```

substitute(_, _, void, void).
substitute(X, Y, tree(Leaf, Left, Right), tree(Leaf1?, Left1?, Right1?)) :-
    ground(X?), ground(Y?) |
    replace(X?, Y?, Leaf?, Leaf1),
    substitute(X?, Y?, Left?, Left1),
    substitute(X?, Y?, Right?, Right1).

replace(X, Y, Z, Y?) :- ground(X?), X? == Z? | true.
replace(X, _, Z, Z?) :- ground(X?), ~(X? == Z?) | true.

```

4.4 Metaprogramming

Programs that manipulate, interpret, and transform other programs. In logic programming, programs are data: clauses are terms, goals are terms.

Programs as Data

A GLP clause:

```
merge([X|Xs], Ys, [X?|Zs?]) :- merge(Xs?, Ys?, Zs).
```

can be represented using a `reduce/2` encoding:

```
reduce(merge([X|Xs], Ys, [X?|Zs?]), merge(Xs?, Ys?, Zs)).
reduce(merge(Xs, [Y|Ys], [Y?|Zs?]), merge(Xs?, Ys?, Zs)).
reduce(merge([], [], []), true).
```

Trust Mode Meta-Interpreter

```
run(M, true). % halt
run(M, (A,B)) :- run(M?, A?), run(M?, B?). % fork
run(M, M1 # G) :- run(M1?, G?). % cross-module
run(M, A) :- tuple(A?) | M # reduce(A?, B), run(M?, B?). % reduce
```

- **Halt:** `true` terminates.
- **Fork:** Conjunction spawns two processes.
- **Cross-module:** `M1 # G` switches modules.
- **Reduce:** Look up clause, continue with body.

```
GLP> run(merge, merge([1,2], [3,4], Zs)).
...
Zs = [1, 3, 2, 4]
-> succeeds
```

Fail-safe Meta-Interpreter

Reports failures using short-circuit:

```
run(M, true, L, L?). % halt - close circuit
run(M, (A,B), L, R?) :- % fork - split chain
    run(M?, A?, L?, Mid),
    run(M?, B?, Mid?, R).
run(M, M1 # G, L, R?) :- run(M1?, G?, L?, R). % cross-module
run(M, failed(A), L, [failed(A?)|L?]). % report failure
run(M, A, L, R?) :- % reduce
```

```

tuple(A?) |
M # reduce(A?, B),
run(M?, B?, L?, R).

```

The object program must include:

```

reduce(A, failed(A?)) :- otherwise | true.

```

Control Meta-Interpreter

Adds control stream for suspend/resume/abort:

```

run(M, true, _, L, L?).                % halt
run(M, (A,B), Cs, L, R?) :-             % fork
    ground(Cs?) |
    run(M?, A?, Cs?, L?, Mid),
    run(M?, B?, Cs?, Mid?, R).
run(M, M1 # G, Cs, L, R?) :-             % cross-module
    run(M1?, G?, Cs?, L?, R).
run(M, A, [suspend|Cs], L, R?) :-        % suspend
    suspended_run(M?, A?, Cs?, L?, R).
run(M, A, Cs, L, R?) :-                 % reduce
    tuple(A?) |
    M # reduce(A?, B),
    run(M?, B?, Cs?, L?, R).

suspended_run(M, A, [resume|Cs], L, R?) :- % resume
    run(M?, A?, Cs?, L?, R).
suspended_run(_, A, [abort|_], L, [A?|L?]). % abort + dump

```

The `ground(Cs?)` guard allows the control stream to be shared among all forked processes.

Tracing Meta-Interpreter

Builds an execution tree:

```

run(M, true, true).                    % halt
run(M, (A,B), fork(TA?, TB?)) :-      % fork
    run(M?, A?, TA),
    run(M?, B?, TB).
run(M, A, reduction(A_copy?, I?, TB?)) :- % reduce
    tuple(A?) |
    copy(A?, A_exec, A_copy),
    M # reduce(A_exec?, B, I),
    run(M?, B?, TB).

```

The `copy/3` predicate (Section 4.3) creates independent copies: `A_exec` drives execution, `A_copy` is frozen for the trace.

The object program provides clause indices:

```
reduce(merge([X|Xs], Ys, [X?|Zs?]), merge(Xs?, Ys?, Zs), 1).
reduce(merge(Xs, [Y|Ys], [Y?|Zs?]), merge(Xs?, Ys?, Zs), 2).
reduce(merge([], [], []), true, 3).
```

```
GLP> run(merge, merge([1,2], [a,b], Zs), Tree).
Zs = [1, a, 2, b]
Tree = reduction(merge([1,2], [a,b], [1,a,2,b]), 1,
    reduction(merge([2], [a,b], [a,2,b]), 2,
        ...))
-> succeeds
```

Given a trace, `replay` executes deterministically:

```
replay(M, true, true).
replay(M, fork(T1, T2), (A?, B?)) :-
    replay(M?, T1?, A),
    replay(M?, T2?, B).
replay(M, reduction(_, I, SubTree), Goal?) :-
    M # reduce_by_index(I?, Goal, Body),
    replay(M?, SubTree?, Body?).
```


Part II

Moded Types

Types and Modes

Types describe the structure of terms; modes describe the direction of data flow. Together, *moded types* specify both what values can appear at each argument position and whether those values flow from caller to callee or from callee to caller. Moded type declarations enable static verification of the SRSW restriction.

5.1 Type Definitions

A type definition introduces a type name and specifies its valid constructors. The syntax uses `::=` to introduce a type and `;` to separate alternatives:

```
Bit ::= 0 ; 1.
```

This defines `Bit` as a type with exactly two values: 0 and 1.

Recursive types define structures containing instances of themselves:

```
Nat ::= 0 ; s(Nat).
```

This defines natural numbers in Peano notation: 0, `s(0)`, `s(s(0))`, and so on.

Lists are the most common recursive type:

```
NumList ::= [] ; [Number | NumList].
```

A `NumList` is either empty or a cons cell where the head is a `Number` and the tail is a `NumList`.

Formal 5.1: Type Definition Syntax

A type definition has the form:

$$\textit{TypeName} ::= \textit{Alt}_1 ; \textit{Alt}_2 ; \dots ; \textit{Alt}_n .$$

Each alternative is one of:

- A constant: `0`, `[]`, `ready`
 - A structure: `s(Nat)`, `node(Tree, Any, Tree)`
 - A list cons: `[Number NumList]—`
 - A type reference: another type name
- Type names begin with an uppercase letter.

5.2 Built-in Types

GLP provides built-in types that need no definition:

Type	Description
<code>Number</code>	Numeric values (integers and floats)
<code>Any</code>	Any term (the universal type)
<code>Atom</code>	Atomic symbols

A generic list uses `Any` for elements:

`List ::= [] ; [Any | List].`

5.3 Moded Procedure Declarations

A procedure declaration specifies both the type and mode of each argument. Modes indicate data flow direction:

- **Consume** (Type?): callee consumes data provided by caller
- **Produce** (Type): callee produces data received by caller

`procedure merge(List?, List?, List).`

This declares that `merge/3`:

- Consumes arguments 1 and 2 (caller provides data via readers)
- Produces argument 3 (caller receives data via writer)

Formal 5.2: Mode Semantics

Modes describe data flow between caller and callee:

Mode	Data flows	Caller provides	Head expects
Consume (Type?)	caller $\rightarrow$ callee	reader	writer
Produce (Type)	callee $\rightarrow$ caller	writer	reader

At the call boundary, what the caller provides and what the head expects are complementary: for consume mode, the caller's reader pairs with the head's writer; for produce mode, the caller's writer pairs with the head's reader.

5.4 Mode Checking

The mode checker verifies that variable occurrences have the correct mode for their position:

- At consume positions: caller provides reader, head expects writer
- At produce positions: caller provides writer, head expects reader

Consider the merge program:

```
List ::= [] ; [Any | List].
```

```
procedure merge(List?, List?, List).
```

```
merge([X|Xs], Ys, [X?|Zs?]) :- merge(Ys?, Xs?, Zs).
```

```
merge(Xs, [Y|Ys], [Y?|Zs?]) :- merge(Xs?, Ys?, Zs).
```

```
merge([], [], []).
```

Head Checking First clause head `merge([XXs], Ys, [X?—Zs?])`—:

- Arg 1: consume mode $\rightarrow$ head expects writer. `X`, `Xs` are writers. ✓
- Arg 2: consume mode $\rightarrow$ head expects writer. `Ys` is writer. ✓
- Arg 3: produce mode $\rightarrow$ head expects reader. `X?`, `Zs?` are readers. ✓

Body Checking The recursive call `merge(Ys?, Xs?, Zs)`:

- Arg 1: consume mode $\rightarrow$ caller provides reader. `Ys?` is reader. ✓
- Arg 2: consume mode $\rightarrow$ caller provides reader. `Xs?` is reader. ✓
- Arg 3: produce mode $\rightarrow$ caller provides writer. `Zs` is writer. ✓

Each variable has exactly one writer occurrence and one reader occurrence, satisfying SRSW.

5.5 Embedded Modes: Response Slots

Type definitions may contain embedded mode annotations within structures. These create *response slots*—positions where the callee can send data back to the caller within a consumed message.

```
CounterMsg ::= clear ; up ; down ; show(Number?).
CounterStream ::= [] ; [CounterMsg | CounterStream].
```

```
procedure counter(CounterStream?, Number?).
```

The `show(Number?)` alternative contains an embedded consume annotation. When modes combine, they follow an involution rule:

Formal 5.3: Mode Involution

When a moded type appears inside another moded type, their modes combine:

Outer mode	Inner mode	Combined
produce	produce	produce
produce	consume	consume
consume	produce	consume
consume	consume	produce

This is involution: $\text{consume} \times \text{consume} = \text{produce}$, like double negation.

For the counter, argument 1 has consume mode (`CounterStream?`). Inside `show`, the `Number?` position also has consume mode. Combined: $\text{consume} \times \text{consume} = \text{produce}$. The caller provides a *writer* at this position to receive the counter’s response.

```
counter([show(State?)|S], State) :-
  number(State?) |
  counter(S?, State?).
```

When matching the `show` clause:

- Caller constructs `show(X)` where `X` is a writer (combined mode is produce)
- Head matches with `show(State?)` where `State?` is a reader
- Term matching connects `X` to `State?`
- When `State` is bound from the second argument, the value flows to `X`

The response slot allows bidirectional communication over a unidirectional stream: the caller sends requests and embeds writers to receive responses.

5.6 Complete Example: Typed Quicksort

```

NumList ::= [] ; [Number | NumList].

procedure quicksort(NumList?, NumList).
procedure qsort(NumList?, NumList?, NumList).
procedure partition(NumList?, Number?, NumList, NumList).

quicksort(Unsorted, Sorted?) :- qsort(Unsorted?, Sorted, []).

qsort([X|Unsorted], Sorted?, Rest) :-
    number(X?) |
    partition(Unsorted?, X?, Smaller, Larger),
    qsort(Smaller?, Sorted, [X?|Sorted1?]),
    qsort(Larger?, Sorted1, Rest?).
qsort([], Rest?, Rest).

partition([X|Xs], A, Smaller?, [X?|Larger?]) :-
    A? < X? | partition(Xs?, A?, Smaller, Larger).
partition([X|Xs], A, [X?|Smaller?], Larger?) :-
    A? >= X? | partition(Xs?, A?, Smaller, Larger).
partition([], A, [], []) :- number(A?) | true.

```

The declarations specify:

- **quicksort**: consumes unsorted list, produces sorted list
- **qsort**: consumes list, produces accumulator head, consumes accumulator tail
- **partition**: consumes list and pivot, produces smaller and larger sublists

5.7 Type Errors and Mode Errors

The checker reports two kinds of errors:

Type errors occur when a term cannot belong to its declared type:

```

procedure foo(NumList).
foo([a, b, c]). % Type error: 'a' is not a Number

```

Mode errors occur when a variable has the wrong mode for its position:

```

procedure bar(Number?, Number).
bar(X?, Y). % Mode error: reader X? where head expects writer,
            % writer Y where head expects reader

```

With declaration `bar(Number?, Number)`, argument 1 is consume mode (head expects writer) and argument 2 is produce mode (head expects reader). The correct clause:

```
bar(X, Y?) :- Y := X? + 1.
```

5.8 Summary

Moded types combine structural types with data flow annotations:

- **Type definitions** specify valid term structures using `:=` and `;`
- **Consume mode** (`Type?`): data flows from caller to callee
- **Produce mode** (`Type`): data flows from callee to caller
- **Mode checking** verifies correct reader/writer placement
- **Mode involution**: $\text{consume} \times \text{consume} = \text{produce}$, enabling response slots

Moded type declarations make the SRSW restriction statically checkable, catching errors before runtime.

Exercises

1. Write moded type declarations for `append/3` and `reverse/2`.
2. Define a `BinaryTree` type and write a moded declaration for `tree_sum/2` that computes the sum of values at nodes.
3. Trace through mode checking for the counter's `show` clause. Why does `show(Number?)` with embedded consume mode create a produce position for the caller?
4. Given procedure `foo(List?, List)`, what modes must the variables have in a clause head `foo([XXs], [Y—Ys])`—?

Typed Programming

This chapter presents advanced GLP programming techniques that build on the moded type system introduced in Chapter 5.

- 6.1 Difference Lists**
- 6.2 Quicksort**
- 6.3 Equators: Emergency Brake**
- 6.4 Bidirectional Communication**
- 6.5 Buffered Communication**

Module System

Programs of any size require modular structure. This chapter presents a module system for GLP that integrates with the moded type system, enabling type-safe composition of separately developed components.

7.1 Design Principles

The module system rests on four principles:

Hierarchy mirrors the file system A GLP project is a directory tree. Each `.glp` file is a module; each directory is a scope. A distinguished file `self.glp` in any directory defines types shared by all modules in that directory and its descendants, following FCP’s `self.cp` convention.

Implicit ancestor scoping A module sees every type and procedure definition from every ancestor scope—from its directory’s `self.glp` up through the project root—without any import directive. This eliminates type duplication: shared protocol types are defined once at the appropriate level.

Self-contained type checking Every module declares the full type signature of every procedure it calls. Its own procedures are declared with `procedure` or `exported procedure`; procedures from other modules are declared with `imported procedure`. The type checker needs only the module’s own text and its ancestor scope—it never reads another module’s source.

Structural type compatibility Type identity is structural: two types with the same automaton are compatible regardless of their names or the modules in which they are defined.

7.2 Module Structure

A typical project:

```
social/
  self.glp          -- shared types: Response, AgentContent, ...
  agent.glp         -- exported agent/4, private helpers
  ui/
    self.glp        -- (optional) UI-specific types
    mediator.glp    -- exported ui_mediator/5
    actors.glp      -- exported actor procedures
  boot.glp          -- imports everything, wires system together
```

`self.glp` files contain only type definitions. They have no procedures; their purpose is to establish the type vocabulary visible to all modules at their level and below.

7.3 Procedure Declarations

Every procedure declaration is exactly one of three kinds:

Kind	Syntax	Meaning
Private	<code>procedure p(...).</code>	Local to module and descendants
Exported	<code>exported procedure p(...).</code>	Reachable from other modules
Imported	<code>imported procedure M#p(...).</code>	Declares dependency on module M

An **exported** declaration serves as a module's public interface. An **imported** declaration records the caller's expectations about a remote procedure's type signature. Together they carry all the information needed for type checking without cross-module access.

Example: Social Agent

The social agent module exports a single procedure and keeps its helpers private:

```
-module(agent).

exported procedure agent(Constant?, UserInStream?,
                        NetInStream?, OutputsList?).

procedure merge(Stream?, Stream?, Stream).
procedure lookup_send(OutputKey?, OutputMsg?,
                    OutputsList?, OutputsList).
```

```

%% ... more private helpers ...

agent(Id, [msg('_user', Id1, connect(Target))|UserIn], NetIn, Outs) :-
    Id? == Id1?, ground(Target?) |
    lookup_send('_net', msg(Target?, intro(Id?, Resp)), Outs?, Outs1),
    %% ...

```

The private `merge/3` and `lookup_send/4` are invisible outside this module. A boot module that needs to call the agent declares:

```

imported procedure agent#agent(Constant?, UserInStream?,
                                NetInStream?, OutputsList?).

```

and calls it as:

```

agent # agent(alice, UserIn?, NetIn?,
              [output('_user', AgentToUser),
               output('_net', NetOut)])

```

The type checker verifies this call against the local `imported` declaration. It does not read `agent.glp`.

Ancestor Scoping

Types defined in `self.glp` are visible to all modules in that directory and below without any import:

```

%% self.glp
Response ::= accept(FriendChannel) ; no.
AgentContent ::= befriend(Constant, Response?)
               ; connected(Constant)
               ; rejected.

```

Both `agent.glp` and `ui/mediator.glp` can use `Response` and `AgentContent` directly. A child directory may shadow an ancestor type with a redefinition; compatibility is checked structurally at each use site.

Formal 7.1: Type Scope Assembly

The type scope for a module M at path p in a project rooted at r is constructed by:

1. Start with the prelude environment E_0 (built-in types: `Stream`, `Channel`, `Constant`, `Integer`, etc.).
2. For each directory $d_1, d_2, \dots, d_k$ on the path from r to the directory containing M , if d_i contains `self.glp` with type definitions T_i , com-

pute $E_i = E_{i-1} \cup T_i$ (later definitions shadow earlier ones).

3. Compute $E_M = E_k \cup T_M$ where T_M are the type definitions in M itself. The resulting E_M is the type environment used for type checking M .

7.4 Cross-Module Type Checking

When a module calls $M \# p(X?, Y)$, the type checker:

1. Finds the local `imported procedure M#p(...)` declaration.
2. Checks the call's arguments against the imported declaration's types, applying the standard well-typing rules including subtyping.

No access to module M is required. The imported declaration carries the full type signature, and types referenced in it are resolved against the caller's type scope (which includes ancestor types).

Formal 7.2: Cross-Module Well-Typing

A cross-module call $M \# p(a_1, \dots, a_n)$ in module N is well-typed if:

1. N contains a declaration `imported procedure M#p(T_1, ..., T_n)`.
2. For each argument a_i , the type of a_i in the clause context is a subtype of T_i (with appropriate mode variance).

This is identical to the well-typing rule for local calls, substituting the imported declaration for a local one.

7.5 Project Compilation

Overview

Project compilation transforms a hierarchy of modules into a single flat program. All inter-module calls become local calls. The output is indistinguishable from a hand-written single-file program.

The transformation proceeds in four steps: discovery, type checking, procedure renaming, and call resolution.

Discovery

The compiler walks the project directory tree, collecting every `.g1p` file. Each is parsed into a module AST. The ancestor scope chain is constructed for each module as described in Section 7.2.

Type Checking

Each module is type-checked independently against its ancestor scope, exactly as for single-file compilation. Imported declarations are verified against the caller's type scope. Type checking completes before any renaming occurs.

Procedure Renaming

After type checking, every procedure in every module is prefixed with its module name. This eliminates name collisions between modules that define procedures with the same name:

Module	Original	Renamed
agent.glp	merge/3	agent:merge/3
agent.glp	agent/4	agent:agent/4
ui/mediator.glp	ui_mediator/5	mediator:ui_mediator/5
boot.glp	merge/3	boot:merge/3
boot.glp	play1/0	boot:play1/0

Without renaming, `agent.glp`'s `merge/3` and `boot.glp`'s `merge/3` would collide in the combined program.

Call Resolution

Every goal in every clause body is resolved to its renamed target:

- **Local calls:** `merge(X, Y, Z)` in `agent.glp` becomes `agent:merge(X, Y, Z)`.
- **Cross-module calls:** `agent # agent(alice, ...)` in `boot.glp` becomes `agent:agent(alice, ...)`.
- **Prelude calls:** calls to built-in procedures (`send`, `receive`, `new_channel`, etc.) remain unprefixed.

Entry Points

The top-level module's exported procedures receive unprefixed aliases, allowing them to be invoked by name:

```
play1 :- boot:play1.
play2 :- boot:play2.
```

If no module exports procedures, all procedures in the top-level module receive aliases for backwards compatibility.

Output

The result is a single module containing all type definitions (deduplicated, inner scopes shadowing outer), all renamed procedures, and the entry-point aliases. This module is compiled through the standard pipeline: partial evaluation, code generation, and execution.

Formal 7.3: Correctness of Project Compilation

Project compilation preserves well-typing. If every module in the project is independently well-typed under its ancestor scope, then the flattened program is well-typed under the combined type environment.

The argument is straightforward: renaming is purely syntactic (replacing f/n with $m:f/n$), and call resolution maps each call to the unique procedure that type checking already verified it against. No new type relationships are introduced.

7.6 Dynamic Linking

Static project compilation is sufficient when all modules are available at compile time. For cases where modules are loaded at runtime—such as mobile agents or plugin architectures—a dynamic linking mechanism is needed.

Load-Time Verification

When a module N is loaded dynamically and calls a procedure from a separately compiled module M , the loader verifies that M 's **exported** declaration is subtype-compatible with N 's **imported** declaration.

Specifically, the loader checks that the callee's exported type accepts at least the inputs the caller may send (contravariance on consume arguments) and produces at most the outputs the caller expects (covariance on produce arguments).

The **imported** declaration records the caller's expectations; the **exported** declaration records the callee's guarantees. The loader checks that guarantees meet expectations via the subtyping relation on moded procedure types.

Type Automata as Runtime Artifacts

For dynamic linking, type automata must be available at load time, not just at compile time. A compiled module carries its type automata alongside its bytecode, enabling the subtype compatibility check without access to source code.

7.7 Validation: Child-Safe Social Graph

The module system was validated by restructuring the Child-Safe Social Graph (CSSG) application—a four-agent system with parent-mediated child befriending—from a monolithic program into a modular project.

The original CSSG consisted of three files totaling over 2,500 lines, with approximately 40 type definitions duplicated across all three. The modular version:

```
cssg_modules/
  self.glp          -- 40 type definitions, defined once
  agent.glp         -- exported agent/4, 13 private procedures
  ui/
    mediator.glp    -- exported ui_mediator/5, 3 private procedures
    actors.glp      -- 16 exported actor procedures
  boot.glp          -- untyped orchestration, 7 plays
```

Each typed module (`agent.glp`, `mediator.glp`, `actors.glp`) is checked independently using only its local declarations and the shared types from `self.glp`. No module reads another module’s source. All seven plays—covering cold-call befriending, friend-mediated introduction (accept and reject scenarios), and parent-mediated child introduction (with acceptance and rejection by each party)—execute correctly.

The `boot.glp` module remains untyped because the generic stream utilities (`tee`, `merge`, `sink`) that wire components together cannot express “this operation preserves the specific type of its input” without parameterized types. This motivates the extension presented in Chapter ??.

Exercises

1. Consider a project with modules `A.glp` and `B.glp` that both define a private `helper/2`. Explain why project compilation avoids the name collision and describe the renamed procedure names.
2. A module `client.glp` declares `imported procedure server#process(Request?, Response)`. The server module exports `exported procedure process(ExtendedRequest?, Response)`, where `ExtendedRequest` is a supertype of `Request`. Is this compatible? Explain using mode variance.
3. The CSSG boot module uses generic `tee/3` with type `Stream` to split streams that are actually `UserNotifyStream`. Describe informally what a parameterized `tee` would look like and why it would make the boot module typeable.

Part III

Simulating Multiagent
Systems

Introduction to Part III

The preceding parts developed GLP as a concurrent logic programming language. We now turn to its intended application: programming grassroots platforms where autonomous agents interact through authenticated channels. Before deploying such systems across distributed networks with cryptographic security, we must be able to develop and test the underlying protocol logic. This part presents a simulation framework that achieves exactly this.

The GLP Sequential Abstract Machine

All programs in this part execute on a single *GLP Sequential Abstract Machine* (GLPSAM)—the standard GLP runtime executing interleaved processes on one computer. When we write a program with goals `alice(...)` and `bob(...)` spawning concurrently, both execute as processes within the same GLPSAM, communicating through shared stream variables on a common heap.

This is simulation, not distribution. There is no network, no separate address spaces, no cryptographic signing. Yet the simulation faithfully captures the logical structure of multiagent interaction, allowing us to develop and verify protocol correctness before adding the complexity of deployment.

What the Simulation Captures

The GLPSAM simulation models the essential aspects of multiagent communication:

Stream-based messaging. Agents communicate by producing and consuming stream elements. A message sent by Alice appears as a cons cell on a stream that Bob reads. The suspension and reactivation mechanics of GLP—where a reader blocks until its paired writer provides a value—model the asynchrony of real network communication.

Channel abstractions. Bidirectional channels, represented as pairs of streams, provide the foundation for agent interaction. The channel notation—`send`,

`receive`, and `new_channel` (Chapter 8)—describes communication patterns that transfer directly to distributed implementation. In code, these operations are expressed through direct stream construction in clause heads.

Protocol sequencing. The befriending protocols—cold-call introduction, friend-mediated introduction, response handling—execute with their full logical structure. Message ordering, response variables, and the inject pattern for deferred message insertion all function exactly as they would in a distributed setting.

SRSW security properties. The single-reader/single-writer invariant provides unforkability: no agent can create two different continuations of the same stream. This property, enforced by GLP’s core semantics, holds equally in simulation and deployment.

Concurrent process structure. When Alice sends messages to Bob and Carol simultaneously, the GLPSAM spawns concurrent processes that interleave fairly. The causal structure of communication—which messages depend on which—emerges correctly from stream synchronization.

What the Simulation Abstracts Away

The simulation omits aspects that are orthogonal to protocol logic:

Cryptographic operations. In deployment, every message carries an attestation—a cryptographic signature binding the message to its sender’s public key. The `attestation/2` guard verifies these signatures. In simulation, we assume all agents are correct and all messages authentic. The attestation guard is simply omitted; protocol logic proceeds as if verification always succeeds.

Network transport. Deployed agents communicate over UDP or other network protocols, with attendant latency, packet loss, and reordering. In simulation, messages pass instantaneously through shared variables. The logical content of protocols is independent of transport mechanics.

Physical distribution. Real grassroots systems run on separate devices with no shared memory. The GLPSAM runs everything on one machine with a shared heap. This difference is invisible to protocol logic, which sees only streams and channels.

Agent identity. In deployment, agents are identified by public keys derived from cryptographic keypairs. In simulation, agents are simply strings: `alice`, `bob`, `carol`. Protocol logic treats identities as opaque values for routing and addressing.

Byzantine failures. The simulation assumes all agents follow the protocol correctly. Malicious agents who deviate from specified behavior, forge messages, or attempt to violate invariants are not modeled. Security against Byzantine behavior requires the cryptographic layer that simulation omits.

The Value of Simulation

Why simulate before deploying? Three reasons:

First, *correctness verification*. Protocol bugs are easier to find and fix when execution is deterministic and observable. The GLPSAM allows tracing message flow, inspecting stream contents, and verifying that protocols reach expected states.

Second, *separation of concerns*. Protocol logic and cryptographic implementation are independent. By developing protocols in simulation, we ensure the logical structure is sound before adding the complexity of key management, signature verification, and secure channels.

Third, *pedagogical clarity*. Understanding how agents coordinate through streams is simpler without the overhead of cryptographic concepts. This part teaches the protocol patterns; deployment adds the security layer.

From Simulation to Deployment

The transition from GLPSAM simulation to distributed deployment requires:

1. **Agent runtime:** Each agent runs its own GLP instance on its device.
2. **Network layer:** Stream messages are serialized, signed, and transmitted over the network.
3. **Attestation:** The `attestation/2` guard is implemented to verify cryptographic signatures.
4. **Module isolation:** Each agent's code runs in a separate module with its own identity.

The protocol logic—the GLP code presented in the following chapters—remains unchanged. This is the power of the abstraction: what works in simulation works in deployment, with security added by the runtime rather than the programmer.

Chapter Overview

Chapter 8 presents the grassroots social graph and befriending protocols. Agents establish authenticated friendships through cold-call and friend-mediated introduction, building a distributed graph of trust relationships.

Chapter 9 develops social networking applications atop the social graph: feed distribution with verified authorship, direct messaging channels, and group formation.

Chapter 10 introduces interlaced streams—the distributed data structure underlying group communication and the blocklace consensus mechanism.

Subsequent chapters extend these foundations to cryptocurrencies and constitutional consensus, demonstrating how grassroots platforms can provide the services of centralized systems without centralized control.

Throughout, remember: the code executes on GLPSAM, simulating multi-agent interaction. The protocols are real; the distribution is simulated. When you see Alice sending a message to Bob, both are processes in your GLP runtime, communicating through streams on a shared heap. This simulation captures the logic that will, in deployment, span devices and networks across the globe.

The Grassroots Social Graph

This chapter presents the grassroots social graph—the foundation for all grassroots platforms. We show how agents establish authenticated friendships through cold-call and friend-mediated introduction protocols.

8.1 Graph Structure

The grassroots social graph is a distributed graph where:

- Nodes represent cryptographically-identified agents
- Edges represent authenticated bidirectional channels
- Connected components arise spontaneously and merge through befriending

Each agent runs on a personal device and maintains its portion of the graph: its identity (keypair), its friend list (authenticated connections), and the GLP runtime.

8.2 Agent Initialization

Each agent begins with the goal `agent(Id, ChUser, ChNet)` where `Id` is the agent's unique identifier, `ChUser` provides bidirectional communication with the user interface, and `ChNet` connects to the network:

```
agent(Id, ch(UserIn, UserOut), ch(NetIn, NetOut)) :-
  merge(UserIn?, NetIn?, In),
  social_graph(Id?, In?, [(user, UserOut), (net, NetOut)]).
```

The initialization pattern-matches the channel structures directly in the head, avoiding guard-based unification that would violate SRSW. The input

streams are merged into a unified stream `In`, and output streams are stored in the initial friends list with special identifiers `user` and `net`.

The `social_graph` predicate is the main event loop, processing messages from the merged input stream. Its arguments are: the agent's identity, the input message stream, and the friends list (an association list mapping names to output streams).

8.3 Cold-Call Befriending Protocol

The cold-call protocol enables agents to establish friendship without prior shared variables. It proceeds in four phases: user initiation, offer transmission, user consultation, and channel establishment.

User Initiation

When the user requests connection to a target agent:

```
social_graph(Id, [msg(user, Id, connect(Target))|In], Fs) :-
    ground(Id), ground(Target) |
    lookup_send(net, msg(Id?, Target?, intro(Id?, Id?, Resp)), Fs?, Fs1),
    response_stream(Resp?, Target?, Id?, Rs),
    merge(In?, Rs?, In1),
    social_graph(Id?, In1?, Fs1?).
```

The agent sends an introduction message through the network channel containing its identity and a fresh response variable `Resp`. The `response_stream` predicate produces a single-element stream when the response arrives; this stream is merged into the input stream so the response can be processed by the main event loop.

Processing Received Introductions

When an agent receives a self-introduction:

```
social_graph(Id, [msg(From, Id?, intro(From?, From?, Resp))|In], Fs) :-
    ground(Id?), ground(From?) |
    lookup_send(user, msg(agent, user, befriend(From?, Resp)), Fs?, Fs1),
    social_graph(Id?, In?, Fs1?).
```

Upon receiving the introduction, the agent consults the user about whether to accept.

User Decision and Channel Establishment

When the user decides:

```
social_graph(Id, [msg(user, Id, decision(Dec, From, Resp))|In], Fs) :-
    ground(Id) |
    bind_response(Dec?, From?, Resp?, Fs?, Fs1, In?, In1),
    social_graph(Id?, In1?, Fs1?).
```

The `bind_response` predicate handles both acceptance and rejection:

```
bind_response(yes, From, accept(ch(FOut?, FIn)), Fs, Fs1?, In, In1?) :-
    constant(From?) |
    handle_accept(From?, ch(FIn?, FOut), Fs?, Fs1, In?, In1).
bind_response(no, _, no, Fs, Fs?, In, In?).
```

On acceptance, the predicate constructs a channel pair directly in the head: `ch(FOut?, FIn)` goes to the remote agent (via the response variable), while `ch(FIn?, FOut)` is kept locally. The reader/writer annotations ensure each stream has exactly one writer.

Establishing the Connection

The `handle_accept` predicate completes channel establishment for the accepting agent:

```
handle_accept(From, ch(FIn, FOut), Fs, Fs1?, In, In1?) :-
    constant(From?) |
    tag_stream(From?, FIn?, Tagged),
    merge(In?, Tagged?, In1),
    add_friend(From?, FOut?, Fs?, Fs1).
```

The `handle_response` predicate handles responses for the initiating agent:

```
handle_response(accept(ch(FIn, FOut)), From, Fs, Fs1?, In, In1?) :-
    constant(From?) |
    tag_stream(From?, FIn?, Tagged),
    merge(In?, Tagged?, In1),
    add_friend(From?, FOut?, Fs?, Fs1).
handle_response(no, _, Fs, Fs?, In, In?).
```

```
add_friend(Name, Out, Fs, [(Name?, Out?)|Fs?]).
```

On acceptance, the friend's output stream is added to the friends list via `add_friend`, and the friend's input stream (tagged with the friend's identity) is merged into the main input stream.

Processing Responses

When the initiating agent receives a response to their introduction:

```
social_graph(Id, [msg(From, Id, response(Resp))|In], Fs) :-
    ground(Id) |
    handle_response(Resp?, From?, Fs?, Fs1, In?, In1),
    social_graph(Id?, In1?, Fs1?).
```

The response message arrives via the merged input stream (from `response_stream`). The `handle_response` predicate establishes the connection on acceptance or does nothing on rejection.

Response Stream

The `response_stream` predicate produces a single-element stream when a trigger variable becomes bound:

```
response_stream(Resp, Target, Id, [msg(Target?, Id?, response(Resp?))]) :-
    known(Resp?) | true.
```

The `known(X?)` guard succeeds when `X` is bound to any value. Until the response arrives, the stream remains unbound. When the remote agent binds `Resp` (via acceptance or rejection), the guard succeeds and the stream produces a single response message that gets merged into the main input stream.

The Inject Predicate

An alternative to `response_stream` is the `inject` predicate, which defers message insertion into an existing stream:

```
inject(X, Y, Ys, [Y?|Ys?]) :- known(X) | true.
inject(X, Y, [Y1|Ys], [Y1?|Ys1?]) :- unknown(X) | inject(X?, Y?, Ys?, Ys1?).
```

The `known(X)` guard succeeds when `X` is bound; `unknown(X)` succeeds when `X` is unbound. When `X` is unbound, messages pass through unchanged. When `X` becomes bound, the deferred message `Y` is inserted. This enables non-blocking protocol operation where a response must be processed at a later time.

Note: `inject` works when the message `Y` is pre-constructed. For messages that must reference the trigger variable (like `response(Resp?)`), use `response_stream` with `merge` instead.

8.4 Friend-Mediated Introduction

When agent *r* is friends with both *p* and *q*, *r* can introduce them. This leverages existing trust relationships.

Initiating an Introduction

The introducer creates channel pairs and sends invitations:

```
social_graph(Id, [msg(user, Id?, introduce(P, Q))|In], Fs) :-
    ground(Id?), ground(P?), ground(Q?) |
    lookup_send(P?, msg(Id?, P?, intro(Q?, ch(QtoP?, PtoQ))), Fs?, Fs1),
    lookup_send(Q?, msg(Id?, Q?, intro(P?, ch(PtoQ?, QtoP))), Fs1?, Fs2),
    social_graph(Id?, In?, Fs2?).
```

The channel pair is constructed directly: *PtoQ* and *QtoP* are fresh variables that form the two halves. *P* receives *ch(QtoP?, PtoQ)* (reads from *Q*, writes to *Q*), and *Q* receives *ch(PtoQ?, QtoP)* (reads from *P*, writes to *P*).

Processing the Introduction

Recipients verify the introduction came from a trusted friend. In GLPSAM simulation, attestation guards are omitted:

```
social_graph(Id, [msg(From, Id?, intro(Other, Ch))|In], Fs) :-
    ground(Id?), ground(From?), ground(Other?) |
    lookup_send(user, msg(agent, user,
        befriending_intro(From?, Other?, Ch?)), Fs?, Fs1),
    social_graph(Id?, In?, Fs1?).
```

The user is consulted about whether to accept the introduction.

Accepting or Rejecting the Introduction

When the user accepts:

```
social_graph(Id, [msg(user, Id?, accept_intro(Other, ch(FIn, FOut)))]|In], Fs) :-
    ground(Id?), ground(Other?) |
    tag_stream(Other?, FIn?, Tagged),
    merge(In?, Tagged?, In1),
    add_friend(Other?, FOut?, Fs?, Fs1),
    social_graph(Id?, In1?, Fs1?).
```

When the user rejects:

```
social_graph(Id, [msg(user, Id?, reject_intro(_, _))|In], Fs) :-
    ground(Id?) |
    social_graph(Id?, In?, Fs?).
```

Upon acceptance, the provided channel is added to the friends list and its input merged into the main stream.

Catch-All Clause

Messages that don't match any specific clause are handled by a catch-all:

```
social_graph(Id, [msg(_, _, _)|In], Fs) :-
    ground(Id), otherwise |
    social_graph(Id?, In?, Fs?).
```

The `otherwise` guard ensures this clause only matches when no other clause applies. Unrecognized messages are silently dropped, allowing the event loop to continue processing.

8.5 Channel Notation

A *channel* `ch(In, Out)` pairs an input stream (which the holder reads) with an output stream (which the holder writes). The following notation describes channel operations semantically:

- **Send:** Adding message `X` to channel `ch(In, Out)` yields `ch(In, [XOut'])`—, extending the output stream.
- **Receive:** Reading from channel `ch([XIn'], Out)`— yields message `X` and continuation `ch(In', Out)`.
- **New channel:** A fresh channel pair consists of `ch(Xs, Ys)` and `ch(Ys, Xs)`— each channel's input is the other's output.

In code, these operations are expressed through direct stream manipulation in clause heads, as shown in the protocol clauses above.

8.6 Utility Predicates

The social graph protocol uses several utility predicates:

```
lookup(Key, [(K, Value)|_], Value?) :- Key? == K? | true.
lookup(Key, [_|Rest], Value?) :-
    otherwise | lookup(Key?, Rest?, Value?).
```

```
lookup_send(Key, Msg, Fs, Fs1?) :-
    ground(Key?), ground(Fs?) |
    lookup(Key?, Fs?, [Msg?|Out1]),
    update(Key?, Out1?, Fs?, Fs1).
```

```
tag_stream(Name, [M|In], [msg(Name?, M?)|Out?]) :-
    constant(Name?) | tag_stream(Name?, In?, Out?).
tag_stream(_, [], []).
```

The `lookup` predicate finds a value in an association list. The `lookup_send` predicate sends a message through the channel associated with a key and updates the friends list with the new stream tail. The `tag_stream` predicate wraps each message with its source identifier, enabling merged streams to preserve source information.

8.7 Testing with Multiagent Plays

The protocols presented above describe agent behavior in isolation. To verify that multiple agents interact correctly, we simulate multiagent systems. This section presents the *play* methodology: self-contained programs that spawn multiple agents, connect them through a network switch, and drive their behavior through scripted actors.

The Network Switch

In deployment, agents communicate through a physical network. In simulation, a `network` process routes messages between agents. Each agent maintains a single channel to the network; messages include a destination address, and the switch forwards them accordingly.

A channel `ch(In?, Out)` pairs an input stream (which the holder reads) with an output stream (which the holder writes). The network holds the inverse of each agent's channel: where Alice has `ch(NetIn?, NetOut)`, the network has `ch(NetOut?, NetIn)`.

For a two-agent network:

```
%% Alice sends to Bob
network2((alice, ch([msg(alice, bob, X)|AliceIn], AliceOut?)),
        (bob, ch(BobIn, [msg(alice, bob, X?)|BobOut?])) :-
    network2((alice, ch(AliceIn?, AliceOut)),
            (bob, ch(BobIn?, BobOut))).

%% Bob sends to Alice
network2((alice, ch(AliceIn, [msg(bob, alice, X?)|AliceOut?])),
        (bob, ch([msg(bob, alice, X)|BobIn], BobOut?))) :-
    network2((alice, ch(AliceIn?, AliceOut)),
            (bob, ch(BobIn?, BobOut))).
```

The structure merits careful study. In the first clause:

- The head reads from Alice's output: `[msg(alice, bob, X)|AliceIn]`— extracts the message
- The head writes to Bob's input: `[msg(alice, bob, X?)|BobOut?]`— delivers the message

- The body preserves the stream tails as writers for subsequent messages

The reader/writer annotations ensure SRSW compliance: each stream variable appears exactly once as a writer in the body. This enables the switch to route multiple messages—after forwarding one message, the switch recurses with fresh stream tails ready for the next.

For n agents, the switch requires $n(n - 1)$ clauses—one for each ordered pair of distinct agents. A four-agent switch has twelve clauses.

Actors and Scripts

A *play* simulates a scenario by spawning agents and driving them with *actors*. An actor represents a user following a deterministic script—responding to agent prompts with predetermined choices.

The simplest actor initiates a single action:

```
alice_actor(Out?, [msg(user, alice, connect(bob))]).
```

Alice's actor writes a `connect(bob)` request to her agent's user input stream, then terminates.

A responding actor waits for a prompt before acting:

```
bob_actor([msg(agent, user, befriend(From, Resp))|_],
          [msg(user, bob, decision(yes, From?, Resp?))|_]).
```

Bob's actor reads a `befriend` request from his agent, then writes an acceptance decision.

The Play Structure

A complete play wires together the network, agents, and actors:

```
play_alice_bob :-
    %% Network switch
    network2((alice, ch(AliceNetOut?, AliceNetIn)),
             (bob, ch(BobNetOut?, BobNetIn))),

    %% Agents
    agent(alice, ch(AliceUserIn?, AliceUserOut),
          ch(AliceNetIn?, AliceNetOut)),
    agent(bob, ch(BobUserIn?, BobUserOut),
          ch(BobNetIn?, BobNetOut)),

    %% Actors
    alice_actor(AliceUserOut?, AliceUserIn),
    bob_actor(BobUserOut?, BobUserIn).
```


The channel wiring requires attention. Each agent has two channels:

- User channel: `ch(UserIn?, UserOut)` connects to the actor
- Network channel: `ch(NetIn?, NetOut)` connects to the switch

The network holds inverse channels: `ch(NetOut?, NetIn)`. This ensures that when Alice writes to her `NetOut`, the network reads it; when the network writes to `NetIn`, Alice reads it.

Execution Trace

Running `play_alice_bob` produces a complete protocol execution:

1. Alice's actor sends `connect(bob)` to her agent
2. Alice's agent sends `intro(alice, alice, Resp)` through the network
3. The network routes the message to Bob
4. Bob's agent sends `befriend(alice, Resp)` to his actor
5. Bob's actor sends `decision(yes, alice, Resp)` to his agent
6. Bob's agent binds `Resp` to `accept(ch(FOut?, FIn))`
7. The `inject_msg` predicate delivers the response to Alice
8. Both agents add each other to their friends lists

The final state shows successful befriending: Alice's friends list contains Bob, and Bob's contains Alice. The protocol terminates with agents suspended on empty friend channels, ready for application-level messaging.

Scaling to More Agents

The methodology scales to larger plays. A four-agent scenario tests cross-network befriending:

```
play_4agents :-
    network4((alice, ch(AliceNetOut?, AliceNetIn)),
             (bob, ch(BobNetOut?, BobNetIn)),
             (carol, ch(CarolNetOut?, CarolNetIn)),
             (dave, ch(DaveNetOut?, DaveNetIn))),

    agent(alice, ch(AliceUserIn?, AliceUserOut),
           ch(AliceNetIn?, AliceNetOut)),
    agent(bob, ch(BobUserIn?, BobUserOut),
           ch(BobNetIn?, BobNetOut)),
    agent(carol, ch(CarolUserIn?, CarolUserOut),
           ch(CarolNetIn?, CarolNetOut)),
    agent(dave, ch(DaveUserIn?, DaveUserOut),
```

```

ch(DaveNetIn?, DaveNetOut)),

alice_actor(AliceUserOut?, AliceUserIn),
bob_actor(BobUserOut?, BobUserIn),
carol_actor(CarolUserOut?, CarolUserIn),
dave_actor(DaveUserOut?, DaveUserIn).

```

With appropriate actor scripts, this play can test scenarios such as: Alice befriends Bob, Carol befriends Dave, then Bob befriends Carol—demonstrating independent befriending operations and graph connectivity growth.

Testing Friend-Mediated Introduction

The cold-call plays test network-routed befriending. To test friend-mediated introduction, we need a scenario where an agent already has multiple friends.

The `play_introduction` scenario proceeds in three phases:

1. Alice cold-calls Bob through the network; Bob accepts
2. Alice cold-calls Carol through the network; Carol accepts
3. Alice introduces Bob to Carol through her friend channels

After phase 2, Alice has both Bob and Carol as friends. The introduction in phase 3 sends messages through Alice’s friend channels—not the network—to Bob and Carol. Each receives an `intro` message containing the other’s identity and one half of a fresh channel pair.

The actors must handle two protocol phases:

```

bob_actor([msg(agent, user, befriend(alice, Resp1))|Out1],
          [msg(user, bob, decision(yes, alice, Resp1?))|In1?]) :-
    bob_wait_intro(Out1?, In1).

bob_wait_intro([msg(agent, user, befriend_intro(alice, carol, Ch))|_],
               [msg(user, bob, accept_intro(carol, Ch?))|_]).

```

Bob’s actor first accepts Alice’s cold-call, then waits for the introduction. The `befriend_intro` message indicates a friend-mediated introduction (from Alice, introducing Carol). Upon acceptance, Bob and Carol establish a direct channel without network involvement.

The final state shows all three agents as mutual friends: Alice connected to both Bob and Carol through cold-calls, and Bob connected to Carol through the introduction.

8.8 Exercises

1. Trace the cold-call protocol when Alice initiates connection to Bob. Show the message flow and variable bindings at each step.

2. Extend the protocol to support connection rejection with a reason message.
3. Implement a `friends_of_friends` predicate that discovers potential connections through existing friends.
4. The current protocol trusts the network to deliver messages. Design an extension using challenge-response to verify liveness.

Social Networks

This chapter presents grassroots social networking applications built upon the social graph foundation (Chapter 8). We develop direct messaging between friends, feed distribution with followers, group messaging, and child-safe social networking.

Throughout this chapter, we assume agents are already connected via the social graph protocol—they have established bidirectional friend channels through cold-call or friend-mediated introduction. The applications here use these existing channels to build higher-level communication structures.

Channel Operations Channel operations introduced in Section 3.2 are used throughout:

```
send(X, ch(In, [X?|Out?]), ch(In?, Out)).
receive(X?, ch([X|In], Out?), ch(In?, Out)).
new_channel(ch(Xs?, Ys), ch(Ys?, Xs)).
```

These unit clauses can be used as defined guards for pattern matching and channel allocation. Complete play code for the examples in this chapter appears in Appendix 12.10.

9.1 Direct Messaging

Friends connected via the social graph share a *control channel* for protocol messages. For ongoing conversations, they establish a separate *direct messaging (DM) channel*—dedicated to private communication, independent of protocol traffic.

Channel Primitives

A DM channel is a bidirectional channel created with `new_channel`:

```
new_channel(ch(Xs?, Ys), ch(Ys?, Xs)).
```

This creates two channel endpoints with cross-connected streams. If Alice holds `ch(Xs?, Ys)` and Bob holds `ch(Ys?, Xs)`, then Alice writes to `Ys` and reads from `Xs`, while Bob writes to `Xs` and reads from `Ys`.

Sending and receiving on a channel:

```
dm_send(Msg, ch(In, [Msg?|Out?]), ch(In?, Out)).
```

```
dm_receive(Msg?, ch([Msg|In], Out?), ch(In?, Out)).
```

The `dm_send` predicate appends a message to the output stream, returning the updated channel. The `dm_receive` predicate extracts a message from the input stream.

Response Binding

When establishing a channel, one party creates both endpoints and sends one to the other through a response variable. Since the response variable extracted from an incoming message becomes a reader, we use explicit unification:

```
bind_resp(Resp, Ch) :- Resp = Ch?.
```

This binds the reader `Resp` to the channel `Ch`.

The DM Protocol

Alice initiates by sending a request containing a fresh response variable through her friend channel. Bob receives the request, creates the channel pair, binds one endpoint to the response variable, and keeps the other:

```
alice(FriendOut, FriendIn?, AliceResult?) :-
    FriendOut = [dm_request(Resp)|_],
    alice_continue(Resp?, FriendIn, AliceResult).

alice_continue(DMCh, _, result(sent, Reply?)) :-
    known(DMCh?) |
    dm_send(hello_bob, DMCh?, DMCh1),
    dm_receive(Reply, DMCh1?, _).
```

Alice sends `dm_request(Resp)` on the friend channel, then waits for `Resp` to become bound. The `known(DMCh?)` guard suspends until Bob responds. Once the DM channel arrives, Alice sends a greeting and waits for a reply.

```

bob([dm_request(Resp)|_], _, BobResult?) :-
    new_channel(BobDM, AliceDM),
    bind_resp(Resp?, AliceDM?),
    bob_continue(BobDM?, BobResult).

bob_continue(DMCh, result(Msg?, replied)) :-
    dm_receive(Msg, DMCh?, DMCh1),
    dm_send(hello_alice, DMCh1?, _).

```

Bob pattern-matches the request in his clause head, creates a channel pair, binds Alice's endpoint to her response variable, then continues with his own endpoint. He receives Alice's message and sends a reply.

The Play

The complete play connects Alice and Bob through a shared friend channel:

```

verify(result(sent, Reply), result(Msg, replied)) :-
    report(Msg?, Reply?).

report(hello_bob, hello_alice).

play_dm :-
    alice(FriendOut, FriendIn?, AliceResult),
    bob(FriendOut?, FriendIn, BobResult),
    verify(AliceResult?, BobResult?).

```

The friend channel connects Alice's output (`FriendOut`) to Bob's input (`FriendOut?`), and Bob's output (`FriendIn`) to Alice's input (`FriendIn?`). The `verify` predicate confirms successful message exchange.

Execution Trace

```

?- play_dm.
alice(FriendOut, FriendIn?, AliceResult) :-
    alice_continue(Resp?, FriendIn, AliceResult)
bob([dm_request(Resp)|_], FriendIn, BobResult) :-
    new_channel(BobDM, AliceDM),
    bind_resp(Resp?, AliceDM?),
    bob_continue(BobDM?, BobResult)
new_channel(BobDM, AliceDM) :- true
bind_resp(Resp?, ch(X1?, X2)) :- true
bob_continue(ch(X2?, X1), BobResult) :-
    dm_receive(Msg, ch(X2?, X1), DMCh1),
    dm_send(hello_alice, DMCh1?, _)

```

```

alice_continue(ch(X1?, X2), FriendIn, AliceResult) :-
    dm_send(hello_bob, ch(X1?, X2), AliceCh1),
    dm_receive(Reply, AliceCh1?, _)
dm_send(hello_bob, ch(X1?, X2), AliceCh1) :- true
dm_receive(Msg, ch([hello_bob|X3?], X1), DMCh1) :- true
dm_send(hello_alice, ch(X3?, X4), _) :- true
dm_receive(Reply, ch([hello_alice|X5?], X3), _) :- true
report(hello_bob, hello_alice) :- true
-> succeeds

```

The trace shows the complete protocol:

1. Alice sends `dm_request(Resp)` on the friend channel
2. Bob receives the request, creates a channel pair with `new_channel`
3. Bob binds Alice's endpoint to her response variable via `bind_resp`
4. Alice's `known` guard succeeds; she sends `hello_bob`
5. Bob receives `hello_bob` and replies with `hello_alice`
6. Alice receives the reply
7. `report(hello_bob, hello_alice)` confirms the exchange

9.2 Feeds and Followers

A *feed* is a stream of posts produced by an author and distributed to *followers*. This section develops the feed server—processing subscriptions and broadcasting posts to all subscribers.

Feed Structure

A feed is represented as `feed(Posts)` where `Posts` is a list of posts, newest first:

```
make_feed(feed([])).
```

```
extend_feed(Post, feed(Xs), feed([Post?|Xs?])).
```

The `make_feed` predicate creates an empty feed. The `extend_feed` predicate prepends a new post to an existing feed.

Broadcasting with Ground Guard

The `ground` guard enables distributing a single post to multiple followers. Without this guard, sending a post to multiple output streams would violate SRSW—each follower's stream would need its own writer occurrence of the post.


```

broadcast(_, [], []).
broadcast(Post, [(Name, Feed)|Fs], [(Name?, Feed1?)|Fs1?]) :-
    ground(Post?) |
    extend_feed(Post?, Feed?, Feed1),
    broadcast(Post?, Fs?, Fs1).

```

The `ground(Post?)` guard suspends until the post is fully instantiated, then permits reading `Post?` multiple times—once per follower. Each follower’s feed is extended with the post.

Follower Management

Followers are stored as an association list mapping names to feeds:

```

get_feed(Name, [(Name?, Feed)|_], Feed?).
get_feed(Name?, [_|Fs], Feed?) :- get_feed(Name, Fs?, Feed).

```

The `get_feed` predicate retrieves a follower’s feed by name.

Feed Server

The feed server processes a command stream, maintaining the follower list:

```

serve_feed([post(Content)|Cmds], Fs, Fs2?) :-
    ground(Content?) |
    broadcast(Content?, Fs?, Fs1),
    serve_feed(Cmds?, Fs1?, Fs2).
serve_feed([subscribe(Name)|Cmds], Fs, Fs2?) :-
    make_feed(Feed),
    serve_feed(Cmds?, [(Name?, Feed?)|Fs?], Fs2).
serve_feed([], Fs, Fs?).

```

The server handles two commands:

- `subscribe(Name)`: Creates a new feed for the subscriber and adds them to the follower list
- `post(Content)`: Broadcasts content to all current followers

The `ground(Content?)` guard ensures posts are fully formed before distribution.

The Play

The play demonstrates Alice running a feed with Bob and Carol as subscribers:

```

alice(Commands?, Followers, FinalFollowers?) :-
    serve_feed(Commands, Followers?, FinalFollowers).

bob_receive(feed([Post1, Post2|_]), result(Post1?, Post2?)).

carol_receive(feed([Post1, Post2|_]), result(Post1?, Post2?)).

verify(result(P1, P2), result(P3, P4)) :-
    check(P1?, P2?, P3?, P4?).

check(second_post, hello_world, second_post, hello_world).

play_feed :-
    alice([subscribe(bob), subscribe(carol),
          post(hello_world), post(second_post)], [], Followers),
    play_continue(Followers?).

play_continue(Followers) :-
    ground(Followers?) |
    get_feed(bob, Followers?, BobFeed),
    get_feed(carol, Followers?, CarolFeed),
    bob_receive(BobFeed?, BobResult),
    carol_receive(CarolFeed?, CarolResult),
    verify(BobResult?, CarolResult?).

```

The command stream subscribes Bob, then Carol, then posts two messages. The `play_continue` predicate uses `ground(Followers?)` to wait until all commands are processed before extracting feeds. Both Bob and Carol receive both posts.

Execution Trace

```

?- play_feed.
serve_feed([subscribe(bob), subscribe(carol),
           post(hello_world), post(second_post)], [], Fs) :-
    make_feed(Feed), serve_feed(...)
make_feed(feed([])) :- true
serve_feed([subscribe(carol), post(...), ...], [(bob, feed([]))|[]], Fs) :-
    make_feed(Feed), serve_feed(...)
make_feed(feed([])) :- true
serve_feed([post(hello_world), post(second_post)],
           [(carol, feed([]))|[(bob, feed([]))|[]]], Fs) :-
    broadcast(hello_world, ...), serve_feed(...)
broadcast(hello_world, ...) :- extend_feed(...), broadcast(...)

```

```

extend_feed(hello_world, feed([]), feed([hello_world])) :- true
...
broadcast(second_post, ...) :- extend_feed(...), broadcast(...)
extend_feed(second_post, feed([hello_world]),
             feed([second_post, hello_world])) :- true
...
play_continue([(carol, feed([second_post, hello_world]))|
              [(bob, feed([second_post, hello_world]))|[]]]) :- ...
get_feed(bob, ..., feed([second_post, hello_world])) :- true
get_feed(carol, ..., feed([second_post, hello_world])) :- true
bob_receive(feed([second_post, hello_world]),
            result(second_post, hello_world)) :- true
carol_receive(feed([second_post, hello_world]),
              result(second_post, hello_world)) :- true
check(second_post, hello_world, second_post, hello_world) :- true
-> succeeds

```

The trace shows:

1. Bob subscribes, receiving an empty feed
2. Carol subscribes, receiving an empty feed
3. `hello_world` is broadcast to both feeds
4. `second_post` is broadcast to both feeds
5. Both receive `feed([second_post, hello_world])`—newest first
6. Verification confirms identical results

9.3 Groups

A *group* enables multiple members to communicate. We present two designs: manager-based groups where a central manager serializes all messages, and groups via interlaced streams where members post independently without a bottleneck.

Manager-Based Groups

In manager-based groups, all messages flow through a manager who serializes and distributes them. This ensures all members see messages in the same order.

The Manager

The manager processes commands, maintaining a member list and message feed:

```

manager(Commands?, Feed?, Members?) :-
    manage(Commands, [], [], Feed, Members).

manage([invite(Name)|Cmds], Members, Feed, Feed1?, Members1?) :-
    manage(Cmds?, [(Name?, [])|Members?], Feed?, Feed1, Members1).

manage([post(From, Content)|Cmds], Members, Feed, Feed1?, Members1?) :-
    ground(From?), ground(Content?) |
    prepend_all(msg(From?, Content?), Members?, Members2),
    manage(Cmds?, Members2?, [msg(From?, Content?)|Feed?], Feed1, Members1).

manage([], Members, Feed, Feed?, Members?).

```

The manager handles two commands:

- `invite(Name)`: Adds a member with an empty message stream
- `post(From, Content)`: Broadcasts to all members, records in feed

The `ground` guards ensure messages are complete before distribution.

Broadcasting

The `prepend_all` predicate distributes a message to all member streams:

```

prepend_all(_, [], []).
prepend_all(Msg, [(Name, Stream)|Ms], [(Name?, [Msg?|Stream?])|Ms1?]) :-
    ground(Msg?) |
    prepend_all(Msg?, Ms?, Ms1).

```

Each member's stream receives the message prepended to their existing messages.

The Play

```

get_member_stream(Name, [(Name?, Stream)|_], Stream?).
get_member_stream(Name?, [_|Ms], Stream?) :-
    get_member_stream(Name, Ms?, Stream?).

play_group_manager :-
    manager([invite(bob), invite(carol),
            post(bob, hello_from_bob),
            post(carol, hello_from_carol)], Feed, Members),
    play_continue(Feed?, Members?).

play_continue(Feed, Members) :-
    ground(Feed?), ground(Members?) |

```

```

    get_member_stream(bob, Members?, BobStream),
    get_member_stream(carol, Members?, CarolStream),
    verify_group(Feed?, BobStream?, CarolStream?).

verify_group(Feed, BobRcvd, CarolRcvd) :-
    check_feed(Feed?),
    check_streams(BobRcvd?, CarolRcvd?).

check_feed([msg(carol, hello_from_carol), msg(bob, hello_from_bob)]).

check_streams([msg(carol, hello_from_carol), msg(bob, hello_from_bob)],
              [msg(carol, hello_from_carol), msg(bob, hello_from_bob)]).
```

The play invites Bob and Carol, then posts messages from each. The `play_continue` predicate waits for all commands to complete before verification.

Execution Trace

```

?- play_group_manager.
manage([invite(bob), ...], [], [], Feed, Members) :-
    manage([invite(carol), ...], [(bob, [])|[]], [], ...)
manage([invite(carol), ...], [(bob, [])], [], ...) :-
    manage([post(...), ...], [(carol, [])|[(bob, [])]], [], ...)
manage([post(bob, hello_from_bob), ...], [...], [], ...) :-
    prepend_all(msg(bob, hello_from_bob), [...], Members2), ...
prepend_all(msg(bob, hello_from_bob), [(carol, [])|(bob, [])], ...) :- ...
...
check_feed([msg(carol, hello_from_carol), msg(bob, hello_from_bob)]) :- true
check_streams([msg(carol, ...), msg(bob, ...)],
              [msg(carol, ...), msg(bob, ...)]) :- true
-> succeeds
```

Both Bob and Carol receive `[msg(carol, hello_from_carol), msg(bob, hello_from_bob)]`—identical message sequences. The manager serializes all posts, ensuring total ordering.

Limitations

The manager is a bottleneck: all messages must flow through a single point. For high-throughput groups or when the manager becomes unavailable, this design limits scalability and availability.

Groups via Interlaced Streams

In interlaced stream groups, each member maintains their own message stream. Members observe each other's streams and reference the current *tips*—the most recent unextended blocks. This creates a partial ordering without requiring a central sequencer.

Interlacing

Each member produces blocks containing their message and references to observed tips:

```
interlace([Payload|Payloads], [block(Payload?, Tips?)|Stream?], Others) :-
    collect_tips(Others?, Tips, Others1),
    interlace(Payloads?, Stream, Others1?).
interlace([], [], _).
```

For each payload, `interlace` collects tips from other members' streams, constructs a block, and continues with the remaining payloads.

Tip Collection

```
collect_tips([[Block|Bs]|Others], [Block?|Tips?], [Bs?|Others1?]) :-
    unknown(Bs?) |
    collect_tips(Others?, Tips, Others1).
collect_tips([[_|Bs]|Others], Tips?, [Bs?|Others1?]) :-
    otherwise |
    collect_tips(Others?, Tips, Others1).
collect_tips([], [], []).
```

The `unknown(Bs?)` guard detects unbound stream tails—positions not yet extended. Such blocks are current tips and get included in the reference list. Already-extended blocks are skipped.

Member Process

```
member(Name, Messages?, MyStream?, OtherStreams?) :-
    tag_messages(Name?, Messages, Tagged),
    interlace(Tagged?, MyStream, OtherStreams).

tag_messages(Name, [Msg|Msgs], [msg(Name?, Msg?)|Tagged?]) :-
    tag_messages(Name?, Msgs?, Tagged).
tag_messages(_, [], []).
```

Each member tags their messages with their name and interlaces with other members' streams.

The Play

```

play_group_interlaced :-
    member(alice, [hello, world], AliceStream, [BobStream?, CarolStream?]),
    member(bob, [hi, there], BobStream, [AliceStream?, CarolStream?]),
    member(carol, [hey, all], CarolStream, [AliceStream?, BobStream?]),
    verify_interlaced(AliceStream?, BobStream?, CarolStream?).

verify_interlaced(AliceStream, BobStream, CarolStream) :-
    check_has_blocks(AliceStream?),
    check_has_blocks(BobStream?),
    check_has_blocks(CarolStream?),
    report_success.

check_has_blocks([block(_, _) | _]).

report_success.

```

Alice, Bob, and Carol each produce two messages. Each member observes the other two members' streams and includes tips in their blocks.

Execution Trace

```

?- play_group_interlaced.
member(alice, [hello, world], AliceStream, [BobStream?, CarolStream?]) :- ...
member(bob, [hi, there], BobStream, [AliceStream?, CarolStream?]) :- ...
member(carol, [hey, all], CarolStream, [AliceStream?, BobStream?]) :- ...
...
AliceStream = [block(msg(alice, hello), []),
               block(msg(alice, world), [])],
BobStream = [block(msg(bob, hi), [block(msg(alice, hello), [])]),
             block(msg(bob, there), [block(msg(alice, world), [])])],
CarolStream = [block(msg(carol, hey), [block(msg(bob, hi), [...])]),
               block(msg(carol, hey), [...])],
-> succeeds

```

Alice produces first (no tips available). Bob's blocks reference Alice's tips. Carol's blocks reference Bob's (and transitively Alice's). The tip references create a partial ordering reflecting causal dependencies.

Properties

No Bottleneck Members produce blocks independently; no single point of failure.

Partial Ordering Concurrent posts remain unordered relative to each other. Causal order is captured by tip references.

Scalability Adding members does not increase load on any single member proportionally.

Consistency All members eventually observe all blocks, though potentially in different orders.

Comparison

Property	Manager-Based	Interlaced
Message ordering	Total	Partial
Bottleneck	Manager	None
Fault tolerance	Manager failure fatal	Resilient
Complexity	Simple	Higher

Manager-based groups suit small, synchronous communication. Interlaced stream groups suit large-scale, asynchronous scenarios where availability and scalability matter more than total ordering.

9.4 Child-Safe Social Networking

Child-Safe Social Networking (CSSN) empowers parents to control their children's online connections. Children can befriend only with mutual parental approval. This section presents the core protocol.

Architecture

CSSN operates on two tiers:

- **Parent tier:** Parents form a social network among themselves using the standard social graph protocol.
- **Child tier:** Children's apps are subordinate to their parents' apps. A child can connect only to children whose parents have approved the connection.

The key invariant: two children can befriend only if their parents are friends and both parents approve.

Approval Protocol

When Child-Smith wants to befriend Child-Jones:

1. Child-Smith requests befriending through their parent app
2. Parent-Smith sends an approval request to Parent-Jones (through their friend channel)
3. Parent-Jones decides whether to approve

4. If approved, the response flows back to Parent-Smith
5. Both parents establish the child channel

The approval request carries an embedded response variable, enabling the approving parent to communicate their decision directly.

Implementation

Response Binding

```
bind_resp(Resp?, Resp).
```

The `bind_resp` helper binds a reader (extracted from a received message) to a value. This enables the receiving parent to respond through the embedded variable.

Requesting Parent

```
parent_smith([approval_request(smith, child_smith, Resp)|_], Result?) :-
    smith_wait(Resp?, Result).
```

```
smith_wait(Resp, Result?) :-
    known(Resp?) |
    smith_done(Resp?, Result).
```

```
smith_done(approved, result(approved, channel_ready)).
```

Parent-Smith sends an `approval_request` containing their identity, the child's identity, and a fresh response variable `Resp`. The `known(Resp?)` guard suspends until Parent-Jones binds the response. On approval, the result indicates the channel is ready.

Approving Parent

```
parent_jones([approval_request(FromParent, Child, Resp)|_], Decision?) :-
    bind_resp(Resp?, approved),
    jones_done(FromParent?, Child?, Decision).
```

```
jones_done(FromParent, Child, approved(FromParent?, Child?)).
```

Parent-Jones receives the request, extracts the embedded response variable, and binds it to `approved`. In a complete system, this would involve consulting the user; here we auto-approve for demonstration.

The Play

```

play_child_safe :-
    parent_smith(StoJ, SmithResult),
    parent_jones(StoJ?, JonesDecision),
    verify_child_safe(SmithResult?, JonesDecision?).

verify_child_safe(result(approved, channel_ready),
    approved(smith, child_smith)) :-
    report_success.

report_success.

```

The play connects Parent-Smith's output to Parent-Jones's input. Parent-Smith sends the approval request; Parent-Jones approves by binding the embedded response variable; Parent-Smith's `known` guard succeeds; verification confirms success.

Execution Trace

```

?- play_child_safe.
parent_smith([approval_request(smith, child_smith, Resp)|_], Result) :-
    smith_wait(Resp?, Result)
parent_jones([approval_request(smith, child_smith, Resp)|_], Decision) :-
    bind_resp(Resp?, approved), jones_done(smith, child_smith, Decision)
smith_wait(Resp?, Result) -> suspended
bind_resp(Resp?, approved) :- true
jones_done(smith, child_smith, Decision) :- true
smith_wait(approved, Result) :- smith_done(approved, Result)
smith_done(approved, result(approved, channel_ready)) :- true
verify_child_safe(result(approved, channel_ready),
    approved(smith, child_smith)) :- report_success
report_success :- true
-> succeeds

```

The trace shows:

1. Parent-Smith sends the request with embedded `Resp`
2. `smith_wait` suspends on `known(Resp?)`
3. Parent-Jones binds `Resp` to `approved` via `bind_resp`
4. `smith_wait` resumes, producing the success result
5. Verification confirms both sides agree

Security Properties

Parental Control Children cannot connect without parental approval—the parent app mediates all connection requests.

Mutual Consent Both parents must approve; a single parent cannot force a connection.

No Third Party Communication occurs directly between parent apps over their friend channel; no platform intermediary.

Verifiable Identity Parents know each other through the social graph; they can verify children’s identities through trusted means.

Extensions

A complete CSSN implementation would include:

- User consultation before approval (not auto-approve)
- Rejection handling with reason messages
- Child channel establishment after mutual approval
- Parental monitoring capabilities
- Age-appropriate content filtering

The core protocol demonstrated here—embedded response variables for cross-agent communication with parental gatekeeping—forms the foundation for these extensions.

9.5 Exercises

1. **DM Message History.** Extend the DM channel to maintain a history of exchanged messages. Each endpoint should accumulate received messages in a list accessible to the holder.
2. **Unsubscribe.** Add an `unsubscribe(Name)` command to the feed server that removes a follower from the distribution list. Ensure subsequent posts do not reach the unsubscribed follower.
3. **Feed Filtering.** Implement a `filtered_feed` predicate that takes a feed and a predicate, returning only posts satisfying the predicate. Use this to implement topic-based filtering.
4. **Group Admin Delegation.** Extend the manager-based group to support multiple administrators. The founder can promote members to admin status; admins can invite new members.
5. **Leave Group.** Add a `leave` command to the manager-based group. When a member leaves, they should stop receiving new messages, and the manager should remove them from the distribution list.

6. **Interlaced Stream Merge.** Implement a `merge_blocklace` predicate that takes multiple interlaced streams and produces a single linearized stream respecting causal order (if block A is in block B 's tips, A appears before B).
7. **Tip Validation.** Add validation to `collect_tips` that verifies all referenced tips actually exist in the observed streams. Report an error if a tip reference is invalid.
8. **Child Rejection.** Extend the child-safe protocol to handle rejection. When Parent-Jones declines, Parent-Smith should receive a `rejected` response and report this to Child-Smith.
9. **Mutual Approval.** The current child-safe protocol shows one-way approval. Extend it so that Parent-Smith must also explicitly approve after receiving Parent-Jones's approval, making the protocol symmetric.
10. **Group Child-Safe.** Design a child-safe group protocol where a parent creates a group for children, and each child can join only with their parent's approval. The group founder's parent acts as group administrator.

Interlaced Streams

This chapter presents interlaced streams, a distributed data structure that generalizes blockchains to directed acyclic graphs (DAGs). Multiple agents maintain individual streams while observing and referencing each other's progress, forming a blocklace—a structure employed by modern consensus protocols.

10.1 Blocklace Structure

A blockchain is a sequence of blocks where each block references its predecessor. A *blocklace* generalizes this: each block may reference multiple predecessors from different agents, forming a DAG rather than a chain. This enables concurrent block creation without requiring consensus on ordering.

In GLP, each agent maintains its own stream of blocks. Each block contains:

- A *payload*: application-specific data
- *Tips*: references to the most recent blocks observed from other agents

The tips create cross-references between streams, weaving them into a unified DAG structure.

10.2 The Interlaced Streams Program

Entry Point

```
streams(stream(S?), Others) :-
    produce_payloads(Payloads),
    interlace(Payloads?, S, Others?).
```

The `streams` predicate takes a list of other agents' stream readers in `Others`. It generates payloads and produces a stream of blocks in `S`. The output is wrapped in `stream(S?)` to distinguish the result structure.

Block Production

```
interlace([Payload|Payloads], [block(Payload?, Tips?)|Stream?], Others) :-
    collect_tips(Others?, Tips, Others1),
    interlace(Payloads?, Stream, Others1?).
interlace([], [], _).
```

For each payload, `interlace` collects the current tips from other streams, constructs a block containing the payload and tips, and recurses with the remaining payloads. The `Others1` variable carries the updated stream positions after tip collection. The base case terminates with an empty stream.

Tip Collection

```
collect_tips([[Block|Bs]|Others], [Block?|Tips?], [Bs?|Others1?]) :-
    unknown(Bs?) |
    collect_tips(Others?, Tips, Others1).
collect_tips([[_|Bs]|Others], Tips?, [Bs?|Others1?]) :-
    otherwise |
    collect_tips(Others?, Tips, Others1).
collect_tips([], [], []).
```

The `collect_tips` predicate traverses the list of other agents' streams. For each stream `[BlockBs]`—

- If `Bs` is unbound (`unknown(Bs?)` succeeds), then `Block` is the current tip—the most recent block not yet extended. Include it in the tips list.
- Otherwise (`Bs` is bound), `Block` has already been extended by its owner. Skip it and continue with the tail.

The `unknown(X?)` guard succeeds when `X` is an unbound variable. This detects stream positions that have not yet been extended—the frontier where new elements will be added.

10.3 Execution Trace

Single Agent

With no other streams to observe:

```
?- streams(S, []).
S = stream([block(r1, []), block(r2, [])])
```

Each block has empty tips since there are no other streams.

Multiple Agents with Incomplete Streams

Testing with incomplete streams from agents P and Q:

```
?- streams(Result, [[block(p1, [])|Ptail], [block(q1, [])|Qtail]]).
Result = stream([block(r1, [block(p1, []), block(q1, [])]),
                 block(r2, X)])
Ptail = <unbound>
Qtail = <unbound>
-> suspended
```

The first block `r1` collects tips from both P and Q: `block(p1, [])` and `block(q1, [])`. The `unknown(Bs?)` guard detects that `Ptail` and `Qtail` are unbound, indicating these are the current tips.

The computation suspends on the second block because after consuming the first blocks, the streams need more blocks from P and Q (which haven't been produced yet). This is correct behavior—the interlacing naturally synchronizes with the progress of other agents.

10.4 Multiagent Deployment

For three agents *p*, *q*, *r* maintaining interlaced streams:

```
p(streams(stream(Ps?), [Qs?, Rs?])),
q(streams(stream(Qs?), [Ps?, Rs?])),
r(streams(stream(Rs?), [Ps?, Qs?]))
```

Each agent receives readers for the other agents' streams. As agents produce blocks concurrently, the tip collection mechanism captures cross-references at the moment each block is created, forming the DAG structure.

10.5 Security Properties

Interlaced streams inherit the security properties of GLP streams (Section ??):

Immutability Once a block is created, its contents cannot be changed.

Unforkability Each agent's stream has a single writer; forks are impossible.

Non-repudiation In authenticated GLP, each block carries attestation proving authorship.

Causal Ordering The tips in each block commit to observed state from other agents.

The blocklace structure provides eventual consistency: all agents eventually observe all blocks, though they may observe them in different orders. The DAG structure captures the causal relationships between blocks.

10.6 Applications

Interlaced streams form the foundation for:

- **Consensus protocols:** Cordial Miners, Morpheus, and Constitutional Consensus use blocklace structures for Byzantine fault-tolerant agreement.
- **Collaborative editing:** Multiple editors' changes form a DAG that can be merged.
- **Distributed ledgers:** Each participant maintains their transaction history while referencing others.

10.7 Exercises

1. Modify `interlaced_streams` to include a timestamp in each block.
2. Implement a `blocklace_merge` predicate that takes multiple agents' streams and produces a linearized view respecting causal order.
3. Add a `validate_tips` predicate that verifies all referenced tips exist in the observed streams.
4. Extend the program to detect when an agent has observed all blocks up to a certain point.
5. Implement a payload filter that only includes blocks matching a predicate in the tips.

Grassroots Cryptocurrencies

This chapter presents grassroots cryptocurrencies and their implementation in GLP. We implement the Grassroots Flash payment system [7], demonstrating how interlaced streams (Chapter 10) provide the blocklace data structure, and how the multiagent architecture enables agents to issue personal currencies and trade with one another.

11.1 Introduction

Grassroots cryptocurrencies [10] are a permissionless digital means for turning mutual trust into liquidity. Their coins are IOUs—units of debt that can be issued digitally by anyone and traded by anyone.

Personal Currencies

Each agent is the *sovereign* of their own currency. Alice issues Alice-coins; Bob issues Bob-coins. An Alice-coin is an IOU from Alice—a unit of debt that Alice owes to whoever holds it.

Alice can *mint* Alice-coins (create new ones) and *burn* them (destroy ones she holds). She can also price her goods and services in Alice-coins: if Alice sells bread, she might price a loaf at 5 Alice-coins.

Mutual Credit

Liquidity arises through *mutual credit*. When Alice and Bob trust each other, they exchange personal coins: Alice gives Bob some Alice-coins, Bob gives Alice some Bob-coins. Each now holds IOUs from the other—effectively, they have extended each other credit.

This exchange requires no external capital. The coins come into existence through the exchange itself, backed only by the trust between Alice and Bob.

Example Alice and Bob each issue 100 of their personal coins to the other. Now Alice holds 100 Bob-coins (claims against Bob) and Bob holds 100 Alice-coins (claims against Alice). If Carol also exchanges 100 coins with both Alice and Bob, then each of the three holds 200 coins from the other two—liquidity created entirely from mutual trust.

Trading

To buy from someone, a trader needs coins that the seller will accept. If Alice wants to buy bread from Bob, she can pay in Bob-coins (which Bob will certainly accept, as they reduce his debt). If Alice has run out of Bob-coins, she must obtain some—perhaps by selling something to Bob, or through coin redemption.

Coin Redemption

The *coin redemption principle* integrates personal currencies into a coherent economy: a sovereign must agree to redeem any domestic coin issued in return for any foreign coin held [10].

Example Bob holds 50 Alice-coins (IOUs from Alice). Bob can demand that Alice redeem these Alice-coins against any coins Alice holds. If Alice holds Bob-coins, Bob might request Bob-coins in return—reducing his exposure to Alice’s debt. If Alice holds Carol-coins and Bob needs to pay Carol, Bob might request Carol-coins instead.

This principle prevents sovereigns from issuing coins irresponsibly. If Alice mints too many Alice-coins and distributes them widely, holders will redeem them, draining Alice’s reserves of other currencies. Eventually Alice must provide goods or services directly to satisfy redemption claims.

Economic Integrity

An agent manifests lack of liquidity by not holding coins from other agents and not being able to obtain such coins by trade or redemption. Lack of liquidity may result in insolvency, making the agent’s coins ‘bad debt’—traded at a discount, if at all [10].

If Alice fails to maintain computational or economic integrity, the result would most probably be a bank run on Alice, with holders rushing to redeem their Alice-coins while Alice still has reserves. This causes financial harm to the family, friends, and colleagues who extended Alice credit. Hence, participants in the grassroots economy have strong incentives to maintain their integrity.

11.2 The Blocklace

The payment system for grassroots cryptocurrencies is implemented using the *blocklace* [7]—a distributed, partially-ordered counterpart of the replicated, totally-ordered blockchain data structure.

Personal Blockchains

Each agent maintains their own *personal blockchain*—a sequence of blocks where each block contains a *self-pointer* to the agent’s previous block. Alice’s blocks form a chain; Bob’s blocks form another chain; and so on.

Unlike a traditional blockchain where all participants extend a single shared chain, here each agent extends only their own chain. The single-writer property of GLP streams directly implements this: only Alice can write to Alice’s stream.

Cross-References

In addition to self-pointers, blocks contain *non-self pointers* to blocks in other agents’ chains. These cross-references weave the individual chains into a DAG—the blocklace.

When Alice creates a block, she includes pointers to blocks she has observed from other agents. These pointers serve specific purposes in the payment protocol:

- A payment block points to the payer’s previous self-payment (proving sufficient balance)
- An accept block points to the payment being accepted and its approval
- An approval block points to the payment being approved

Mapping to Interlaced Streams

The blocklace maps directly to the interlaced streams of Chapter 10:

Blocklace Concept	GLP Implementation
Personal blockchain	Agent’s stream
Self-pointer	Implicit in stream structure
Non-self pointer	Block reference in <code>Pointers</code> list
Block payload	<code>tx(Currency, Payments, Comment)</code>

The key difference from generic interlaced streams: payment blocks use *specific* references (to THE payment block, THE approval block) rather than generic tip collection. The protocol dictates which blocks must be referenced.

Transaction Blocks

A transaction block records a payment in a particular currency. In GLP:

```
block(tx(Currency, Payments, Comment), Pointers)
```

Where:

- **Currency** identifies the sovereign (whose coins are being transacted)
- **Payments** is a list of `payment(Recipient, Amount)` terms
- **Comment** indicates the transaction type (`issue`, `accept`, `approve`, etc.)
- **Pointers** contains references to other blocks

Self-Payments and Balances

A *self-payment* is a payment where the recipient is the block's author. When Alice creates a block containing `payment(alice, 50)` in Bob-coins, this records that Alice holds 50 Bob-coins after the transaction.

Self-payments create an audit trail. The most recent self-payment in a currency equals the agent's current balance in that currency. To verify Alice's Bob-coin balance, find her most recent block containing a Bob-coin self-payment.

Consuming Blocks

A block *consumes* another block when it spends coins received in that block. If Bob's block pays Alice 10 Bob-coins, Alice's accepting block must point to (consume) Bob's block. This pointer proves the payment occurred and prevents Alice from accepting the same payment twice.

A block is *balanced* if the sum of its outgoing payments equals the sum of payments to the author in the blocks it consumes. Only minting is unbalanced—the sovereign creates value by fiat.

Sovereign Approval

Payments in a currency require the sovereign's approval before the recipient can accept them. If Alice pays Bob 10 Carol-coins:

1. Alice creates an Issue block with the payment, pointing to her previous Carol-coin self-payment
2. Carol observes Alice's block and creates an Approve block pointing to it
3. Bob observes both blocks and creates an Accept block pointing to Alice's Issue block and Carol's Approve block

The sovereign's approval confirms the payment is valid: the payer's block is balanced and the payer has not equivocated (created conflicting blocks that spend the same coins twice).

Payments in one's own currency need no external approval—Alice approving her own Alice-coin payments would be redundant.

Equivocation

Two blocks by the same agent that do not observe each other constitute an *equivocation*. If both blocks spend the same coins, this is a *double-spend* attempt.

The blocklace structure makes equivocation detectable: if Alice creates two blocks that both consume the same previous self-payment, neither block observes the other. Any agent who receives both blocks can detect the equivocation.

A sovereign who detects equivocation creates a Disapprove block pointing to the fraudulent payment and the evidence (the equivocating block).

11.3 The Grassroots Flash Protocol

Grassroots Flash [7] defines four types of transaction blocks.

Trader Blocks

Any agent can create these blocks. Both must be balanced.

Issue. Alice pays Bob 10 Carol-coins. Her block contains:

- Payment `payment(bob, 10)` in Carol-coins
- Self-payment `payment(alice, Z)` recording her new balance
- Pointer to her previous Carol-coin self-payment (showing she had at least 10)

Accept. Bob accepts Alice's payment (after Carol approves it). His block contains:

- Self-payment `payment(bob, Z)` recording his new balance
- Pointer to Alice's Issue block (the payment being accepted)
- Pointer to Carol's Approve block (proving sovereign approval)
- Pointer to his previous Carol-coin self-payment, if any

Sovereign Blocks

Only the currency's sovereign creates these blocks.

Mint. Carol creates Carol-coins. Her block contains:

- Self-payment `payment(carol, Z)` recording her new balance
- Pointer to her previous Carol-coin self-payment, if any

Minting is the only unbalanced operation—the sovereign creates value by fiat.

Approve. Carol validates Alice’s payment to Bob in Carol-coins. Her block contains:

- No payments (empty list)
- Comment `approve`
- Pointer to Alice’s Issue block

Carol approves if Alice’s block is balanced and Carol does not observe Alice equivocating (double-spending).

Protocol Safety

The protocol ensures:

Balance Safety Self-payments accurately record balances.

No Double-Spend Each payment block is consumed by at most one Accept block.

Sovereign Authority Payments require sovereign approval before acceptance.

These properties follow from the single-writer constraint: only Alice can extend Alice’s stream, so she cannot be forced to accept payments or have her balance modified by others.

11.4 Implementation in GLP

We implement Grassroots Flash using the agent-and-request-stream pattern established in earlier chapters. Each agent processes a stream of requests, maintains balances as an accumulator, and produces a stream of blocks.

Agent Process

An agent processes requests, updating balances and extending their block stream:

```
agent([], _, []).
agent([Request|Requests], Balances, Stream) :-
    handle(Request?, Balances?, Balances1, Stream?, Stream1),
    agent(Requests?, Balances1?, Stream1?).
```

The agent consumes requests one at a time. Each request may update balances and append blocks to the stream. The stream produced is the agent's personal blockchain.

Balance Management

Balances are tracked per currency as an association list:

```
get_balance(Currency, [(Currency?, Bal)|_], Bal?).
get_balance(Currency?, [_|Balances], Bal?) :-
    get_balance(Currency, Balances?, Bal).
get_balance(_, [], 0).

set_balance(Currency, NewBal, [], [(Currency?, NewBal?)]).
set_balance(Currency, NewBal, [(Currency?, _)|Bs], [(Currency?, NewBal?)|Bs?]).
set_balance(Currency?, NewBal, [B|Bs], [B?|Bs1?]) :-
    set_balance(Currency, NewBal?, Bs?, Bs1?).
```

Request Handlers

Issue. Pay someone in a given currency. The block points to the payer's previous self-payment (implicit in stream position) and records the new balance:

```
handle(issue(To, Amount, Currency), Balances, Balances1, Stream, Stream1) :-
    get_balance(Currency?, Balances?, OldBal),
    Amount? =< OldBal? |
    NewBal := OldBal? - Amount?,
    set_balance(Currency?, NewBal?, Balances?, Balances1),
    Stream = [block(tx(Currency?,
        [payment(To?, Amount?), payment(self, NewBal?)]),
        issue, [])
    | Stream1?].
```

Accept. Accept an incoming payment. The block points to the payment being consumed and its approval:

```
handle(accept(Amount, Currency, PaymentBlock, ApprovalBlock),
    Balances, Balances1, Stream, Stream1) :-
    get_balance(Currency?, Balances?, OldBal),
    NewBal := OldBal? + Amount?,
    set_balance(Currency?, NewBal?, Balances?, Balances1),
    Stream = [block(tx(Currency?, [payment(self, NewBal?)]),
        accept, [PaymentBlock?, ApprovalBlock?])
    | Stream1?].
```

Approve. Sovereign approves a payment in their currency. The block points to the payment being approved:

```
handle(approve(PaymentBlock), Balances, Balances?, Stream, Stream1) :-
    Stream = [block(tx(self, []), approve, [PaymentBlock?]) | Stream1?].
```

Mint. Sovereign creates new coins. This is the only unbalanced operation:

```
handle(mint(Amount), Balances, Balances1, Stream, Stream1) :-
    get_balance(self, Balances?, OldBal),
    NewBal := OldBal? + Amount?,
    set_balance(self, NewBal?, Balances?, Balances1),
    Stream = [block(tx(self, [payment(self, NewBal?)]), mint, []) | Stream1?].
```

Mutual Credit

Two agents establish mutual credit by each issuing payment to the other in their own currency, then accepting:

```
alice_mutual_credit(Bob, Amount, BobStream) :-
    [issue(Bob?, Amount?, alice),
     accept(Amount?, bob, BobPayment?, _)] :-
    find_payment_to(alice, BobStream?, BobPayment).
```

```
bob_mutual_credit(Alice, Amount, AliceStream) :-
    [issue(Alice?, Amount?, bob),
     accept(Amount?, alice, AlicePayment?, _)] :-
    find_payment_to(bob, AliceStream?, AlicePayment).
```

Each pays in their own currency (so approval is implicit), then accepts the other's payment. The result: Alice holds Bob-coins, Bob holds Alice-coins.

Redemption

Bob holds Alice-coins and wants to redeem them. He creates a redemption claim specifying his preference list of currencies:

```
handle(redeem(Sovereign, Amount, Preferences),
       Balances, Balances1, Stream, Stream1) :-
    get_balance(Sovereign?, Balances?, OldBal),
    Amount? =< OldBal? |
    NewBal := OldBal? - Amount?,
    set_balance(Sovereign?, NewBal?, Balances?, Balances1),
    Stream = [block(tx(Sovereign?, [payment(Sovereign?, Amount?)]),
                  (redeem, Preferences?), [])
              | Stream1?].
```


The sovereign receives the claim and must repay. She pays from the first preferred currency until exhausted, then the second, and so on. Any remainder is paid in her own coins:

```
handle(repay(Claimer, Amount, Preferences, ClaimHash),
      Balances, Balances1, Stream, Stream1) :-
  compute_repayments(Amount?, Preferences?, Balances?, Repayments, Balances1),
  emit_repayments(Claimer?, Repayments?, ClaimHash?, Stream?, Stream1).
```

Computing repayments. Walk the preference list, taking what's available:

```
compute_repayments(0, _, Balances, [], Balances?).
compute_repayments(Amount, [], Balances, [(self, Amount?)], Balances?) :-
  Amount? > 0 | .
compute_repayments(Amount, [Currency|Rest], Balances, Repayments?, Balances1) :-
  Amount? > 0 |
  get_balance(Currency?, Balances?, Available?),
  take_from_currency(Amount?, Currency?, Available?, Rest?,
                    Balances?, Repayments, Balances1).

take_from_currency(Amount, _, 0, Rest, Balances, Repayments?, Balances1) :-
  compute_repayments(Amount?, Rest?, Balances?, Repayments, Balances1).
take_from_currency(Amount, Currency, Available, _, Balances,
                  [(Currency?, Amount?)], Balances1) :-
  Available? >= Amount? |
  NewBal := Available? - Amount?,
  set_balance(Currency?, NewBal?, Balances?, Balances1).
take_from_currency(Amount, Currency, Available, Rest, Balances,
                  [(Currency?, Available?)|Repayments?], Balances1) :-
  Available? > 0, Available? < Amount? |
  Remainder := Amount? - Available?,
  set_balance(Currency?, 0, Balances?, Balances2?),
  compute_repayments(Remainder?, Rest?, Balances2?, Repayments, Balances1).
```

Emitting repayment blocks. Each repayment becomes a block referencing the claim:

```
emit_repayments(_, [], _, Stream, Stream?).
emit_repayments(Claimer, [(self, Amount)|Rest], ClaimHash, Stream, Stream1) :-
  Stream = [block(tx(self, [payment(Claimer?, Amount?)]),
                  (repay, ClaimHash?), [])) | Stream2?],
  emit_repayments(Claimer?, Rest?, ClaimHash?, Stream2?, Stream1).
emit_repayments(Claimer, [(Currency, Amount)|Rest], ClaimHash, Stream, Stream1) :-
  Currency? \= self |
```

```
Stream = [block(tx(Currency?, [payment(claimer?, Amount?)]),
               (repay, ClaimHash?), []) | Stream2?],
emit_repayments(claimer?, Rest?, ClaimHash?, Stream2?, Stream1).
```

11.5 A Complete Example

Three agents: Alice, Bob, and Carol. Alice and Bob establish mutual credit, then Alice pays Carol in Bob-coins.

The Scenario

1. Alice and Bob exchange 100 personal coins (mutual credit)
2. Alice pays Carol 20 Bob-coins
3. Bob approves Alice's payment
4. Carol accepts the payment

The Play

```
play_cryptocurrency :-
    %% Alice: credit with Bob, then pay Carol
    agent([issue(bob, 100, alice),
           accept(100, bob, BobPayment?, _),
           issue(carol, 20, bob)],
          [], AliceStream),

    %% Bob: credit with Alice, then approve Alice's payment to Carol
    agent([issue(alice, 100, bob),
           accept(100, alice, AlicePayment?, _),
           approve(AliceToCarol?)],
          [], BobStream),

    %% Carol: accept payment from Alice
    agent([accept(20, bob, AliceToCarol?, BobApproval?)],
          [], CarolStream),

    %% Wire up the cross-references
    find_payment_to(bob, AliceStream?, AlicePayment),
    find_payment_to(alice, BobStream?, BobPayment),
    find_payment_to(carol, AliceStream?, AliceToCarol),
    find_approval(BobStream?, BobApproval),

    verify(AliceStream?, BobStream?, CarolStream?).
```

Helper Predicates

```

find_payment_to(Recipient, [Block|_], Block?) :-
    Block = block(tx(_, Payments), issue, _),
    has_payment_to(Recipient?, Payments?) | .
find_payment_to(Recipient?, [_|Blocks], Block?) :-
    find_payment_to(Recipient, Blocks?, Block).

has_payment_to(Recipient, [payment(Recipient?, _)|_]).
has_payment_to(Recipient?, [_|Payments]) :-
    has_payment_to(Recipient, Payments?).

find_approval([Block|_], Block?) :-
    Block = block(_, approve, _) | .
find_approval([_|Blocks], Block?) :-
    find_approval(Blocks?, Block).

```

Execution Trace

The execution proceeds as follows:

1. Alice issues 100 Alice-coins to Bob (her stream: [block(tx(alice, ...), issue, [])])
2. Bob issues 100 Bob-coins to Alice (his stream: [block(tx(bob, ...), issue, [])])
3. Bob accepts Alice's payment—his block points to Alice's Issue block
4. Alice accepts Bob's payment—her block points to Bob's Issue block
5. Alice issues 20 Bob-coins to Carol—her block records new balance of 80 Bob-coins
6. Bob approves Alice's payment—his block points to Alice's Issue-to-Carol block
7. Carol accepts the payment—her block points to Alice's Issue block and Bob's Approve block

Final balances:

- Alice: 80 Bob-coins, 0 Alice-coins (gave 100 to Bob)
- Bob: 100 Alice-coins, 0 Bob-coins (gave 100 to Alice)
- Carol: 20 Bob-coins

The blocklace structure captures causal dependencies: Carol's Accept block points to both Alice's Issue block and Bob's Approve block, proving the payment was properly authorised before acceptance.

Redemption Example

Bob holds 50 Alice-coins and wants Carol-coins or Dave-coins in return:

```
play_redemption :-
    %% Bob has 50 Alice-coins, redeems with preference [carol, dave]
    agent([redeem(alice, 50, [carol, dave])],
          [(alice, 50)], BobStream),

    %% Alice has 20 Carol-coins, 15 Dave-coins
    %% She repays: 20 Carol + 15 Dave + 15 Alice = 50
    agent([repay(bob, 50, [carol, dave], ClaimHash?)],
          [(carol, 20), (dave, 15)], AliceStream),

    find_claim_hash(BobStream?, ClaimHash),
    verify_redemption(AliceStream?, BobStream?).
```

After redemption:

- Alice: 0 Carol-coins, 0 Dave-coins, owes 15 more Alice-coins
- Bob: 0 Alice-coins, 20 Carol-coins, 15 Dave-coins, 15 Alice-coins (remainder)

Bob wanted to fully exit Alice-coins but Alice didn't have enough preferred currencies, so Bob still holds 15 Alice-coins.

11.6 Security Properties

The implementation inherits security properties from GLP streams and the Grassroots Flash protocol.

From Single-Writer Streams

Immutability Once a block is added to an agent's stream, its contents cannot be changed. The stream is append-only.

Unforkability Each agent's stream has a single writer. An agent cannot create two different extensions of the same stream position—this would require two writer occurrences of the same variable.

Non-repudiation In authenticated GLP, each block carries the agent's signature. Authorship cannot be denied.

No Forced Transactions Only Alice can extend Alice's stream. No one can force Alice to accept a payment or modify her recorded balance.

From the Protocol

No Double-Spending Each payment block can be consumed by at most one Accept block. If Alice tries to spend the same coins twice (creating two Issue blocks that consume the same self-payment), the sovereign will observe the equivocation and refuse to approve.

Balance Integrity Self-payments create an audit trail. The most recent self-payment in a currency equals the agent’s actual balance. Any discrepancy indicates protocol violation.

Sovereign Authority Payments in a currency require that currency’s sovereign to approve before the recipient can accept. The sovereign verifies that payments are balanced and that the payer has not equivocated.

Redemption Obligation Sovereigns must repay redemption claims according to the specified preference order. Failure to repay damages the sovereign’s reputation, potentially triggering a bank run.

Equivocation Detection

Two blocks by agent p that do not observe each other constitute an equivocation. Detection is straightforward: if an agent receives two p -blocks where neither points (directly or transitively) to the other, p has equivocated.

In GLP terms, equivocation would require Alice to create two blocks at the same stream position—assigning to the same writer variable twice. The SRSW constraint makes this syntactically impossible within a single agent process. Equivocation can only occur through Byzantine behaviour (running modified code), which the protocol detects through the blocklace structure.

Limitations

This implementation assumes all-to-all dissemination: every agent eventually receives every block. The full Grassroots Flash protocol [7] uses *cordial dissemination* over a social graph, where friends relay blocks based on need. This grassroots dissemination mechanism could be implemented using the social graph protocols from Chapter 8.

11.7 Exercises

1. **Disapproval.** Extend the sovereign operations to include a **disapprove** request that rejects unbalanced payments or equivocations. The disapproval block should point to the evidence.
2. **Balance Query.** Implement a **query_balance/3** predicate that traverses an agent’s stream and computes their current balance in a given currency by finding the most recent self-payment.

3. **Equivocation Detection.** Implement a `detect_equivocation/2` predicate that takes two blocks from the same agent and determines whether they constitute an equivocation (neither observes the other).
4. **Block Validation.** Implement a `validate_block/2` predicate that checks whether a payment block is balanced: the sum of outgoing payments equals the sum of payments in consumed blocks.
5. **Community Bank.** Model a community bank that establishes mutual credit lines with villagers. The bank can shift risk by redeeming coins from financially unstable villagers against coins of more stable villagers.
6. **Cordial Dissemination.** Extend the implementation to use friend-mediated dissemination. Agents only see blocks relayed by friends. A payment from Alice to Carol in Bob-coins requires paths in the social graph connecting all parties.
7. **Transaction History.** Implement a `payment_history/3` predicate that extracts all payments to a given agent from a blocklace, returning a list of `received(From, Amount, Currency)` records.
8. **Credit Limit.** Extend mutual credit to enforce a maximum credit line. If Alice and Bob agree to a limit of 100 coins, neither can hold more than 100 of the other's coins at any time.
9. **Multi-Currency Redemption.** Extend the redemption protocol to allow Bob to redeem coins from multiple sovereigns in a single transaction, specifying a global preference order across all currencies.
10. **Blocklace Visualisation.** Write a predicate that outputs a blocklace in a format suitable for graph visualisation, showing each agent's chain and the cross-references between chains.

Constitutional Consensus

This chapter presents Constitutional Consensus and its implementation in GLP. We show how interlaced streams (Chapter 10) provide the blocklace structure for consensus, enabling communities to reach agreement on transaction ordering while maintaining complete self-governance.

12.1 Introduction

Some grassroots applications require no consensus. Grassroots social networks (Chapter ??) and grassroots cryptocurrencies (Chapter 11) operate without global agreement on transaction ordering—their safety depends only on agents following the protocol, which can be ensured through attestation mechanisms.

However, applications requiring collective agreement—community banks with multiple signatories, democratic governance with voting, digital cooperatives—need consensus to ensure all participants agree on the order of transactions and decisions.

Digital Social Contracts

Constitutional Consensus enables *digital social contracts* [2]—deterministic state machines encoding agreements among people who know each other. Unlike smart contracts that operate among anonymous accounts and run on third-party servers, social contracts are among identified people and run on the participants' own devices.

	Smart Contract	Digital Social Contract
Among	Accounts	People
Executed by	Third-party servers	Participants' smartphones
Governance	Token-weighted (plutocracy)	One person, one vote (democracy)
Amendments	External tokens/DAOs	Participants via supermajority

Technical Requirements

Communities implementing digital social contracts face unique technical challenges:

Quiescent operation No activity when idle. Small communities may have transaction rates much lower than network latency.

Dual-mode efficiency Seamless transition between low-throughput (isolated transactions) and high-throughput (many concurrent transactions) modes.

Responsiveness Network-speed operation, achieving finality in 3δ where δ is the message delay.

Self-governance Participants control membership and protocol parameters through the consensus protocol itself.

Attestation and Simplification

Throughout this chapter, we assume agents are *attested*—running verified code, possibly within Trusted Execution Environments (TEEs), or mutually attesting correct behaviour. This assumption eliminates Byzantine faults, allowing significant simplification.

With no Byzantine agents:

- Simple majority ($> n/2$) suffices instead of supermajority ($> 2n/3$)
- Equivocation is impossible—the single-writer property of GLP streams prevents any agent from creating conflicting blocks
- Reliable dissemination can be assumed—no need for complex protocol mechanisms to handle partial Byzantine dissemination

What remains unchanged is the fundamental structure needed for consensus in asynchronous networks. The three-round wave structure addresses agreement despite message delays—a problem that exists even among perfectly correct agents.

12.2 The Consensus Problem

Agreement in Asynchronous Networks

The consensus problem requires a set of agents to agree on a sequence of values (transactions), satisfying:

Safety No two correct agents finalize different transactions at the same sequence position.

Liveness Every transaction submitted by a correct agent is eventually finalized.

Totality All correct agents eventually output the same sequence.

The FLP impossibility result [?] proves that deterministic consensus is impossible in purely asynchronous systems, even with just one crash failure. Any consensus protocol must therefore make timing assumptions.

Eventual Synchrony

Constitutional Consensus assumes *eventual synchrony*: message latency is unbounded, but after some unknown *Global Stabilization Time (GST)*, all messages arrive within a bound δ .

This model captures real networks: they may experience arbitrary delays (network partitions, congestion), but eventually stabilize. The protocol guarantees safety always (even before GST) and liveness after GST.

Constitution

A *constitution* defines the protocol's parameters:

Definition 12.1 (Constitution). *A constitution is a triple (P, σ, Δ) where:*

- $P \subset \Pi$ is the set of participants
- σ is the supermajority threshold (fraction required for decisions)
- Δ is the presumed message delay bound after GST

With attestation (no Byzantine agents), we set $\sigma = \frac{1}{2}$, requiring simple majority. A *majority* among participants is any subset $Q \subseteq P$ with $|Q| > |P|/2$.

Why Majority Suffices

With Byzantine agents, the protocol requires $\sigma > \frac{2}{3}$ to ensure that:

1. Any two supermajorities overlap in at least one correct agent
2. A supermajority contains a majority of correct agents

With no Byzantine agents, simple majority ($\sigma = \frac{1}{2}$) suffices: any two majorities overlap in at least one agent (by pigeonhole), and that agent is necessarily correct.

12.3 The Blocklace for Consensus

Constitutional Consensus uses the blocklace structure introduced in Chapter 11, but for a different purpose: ordering transactions rather than tracking payments.

Mapping to Interlaced Streams

Each participant maintains a personal blockchain—their stream of blocks. Cross-references between blocks create a DAG (the blocklace). In GLP:

Consensus Concept	GLP Implementation
Personal blockchain	Agent's stream
Block	Stream element
Cross-reference	Block field pointing to other blocks
Blocklace	Interlaced streams

Block Structure

A consensus block contains:

`block(Round, Payload, Pointers)`

Where:

- **Round** is the block's round number (determines its role in the wave)
- **Payload** is the transaction (or empty for endorsement/ratification blocks)
- **Pointers** contains references to observed blocks from other participants

Constitutional Genesis Block

Each consensus instance begins with a *constitutional genesis block* specifying the initial constitution (P, σ, Δ) . All participants must observe this block before participating.

`genesis(P, Sigma, Delta)`

Depth and Rounds

The *depth* of a block is its distance from the genesis block along any path. Blocks at the same depth are in the same *round*. Rounds are grouped into *waves* of three rounds each.

12.4 Wave Structure

The protocol operates in waves of three rounds. Each wave attempts to finalize one block.

The Three Rounds

Round 1 (Candidates) Participants with transactions create blocks containing their payload. These are candidates for finality.

Round 2 (Endorsements) Participants observe round 1 and create endorsement blocks. An endorsement points to the round 1 blocks the participant has seen.

Round 3 (Ratifications) Participants observe round 2 and create ratification blocks. A ratification confirms that a majority endorsed the same candidate.

Finality

A round 1 block b is *finalized* when:

1. A majority of participants created round 2 blocks endorsing b (and only b)
2. A majority of participants created round 3 blocks ratifying those endorsements

Once finalized, the block's payload is added to the consensus output sequence.

Why Three Rounds?

The three-round structure solves agreement despite asynchrony—not Byzantine behaviour. Consider:

When Alice sees a single round 1 block from Bob, she cannot know if:

- Bob's block is truly the only candidate (low-throughput case)
- Carol also sent a round 1 block that Alice hasn't received yet

The endorsement round creates *evidence* that a majority saw the same candidates. The ratification round creates *evidence* that a majority saw the same endorsements. This evidence is what enables agreement despite arbitrary message delays.

Quiescent Waves

A wave is *quiescent* if all its blocks (rounds 1, 2, and 3) are empty—no participant had a transaction to propose. During quiescence, the protocol is idle, consuming no resources.

12.5 Dual-Mode Operation

Constitutional Consensus seamlessly handles both low and high transaction rates.

Low-Throughput Mode

When transactions arrive infrequently:

1. The network is quiescent (no activity)
2. A participant with a transaction creates a round 1 block
3. If no conflicts arise, others immediately endorse and ratify
4. The transaction is finalized in one wave (3δ)

Any participant can initiate a wave—there is no designated leader. This is called *spontaneous leadership*.

High-Throughput Mode

When multiple participants have transactions simultaneously:

1. Multiple round 1 blocks appear (conflict)
2. A *formal leader* is selected (round-robin by round number)
3. Participants endorse the leader’s block
4. The leader’s transaction is finalized; others retry in subsequent waves

Mode Transitions

The protocol transitions automatically based on observed activity:

Low → **High** When a participant observes conflicting round 1 blocks, they switch to high-throughput mode and follow formal leadership.

High → **Low** When a wave completes with only empty blocks (quiescent), participants return to low-throughput mode.

Timeout Handling

During high-throughput mode, if the formal leader fails to produce a block, participants timeout after Δ and proceed to the next round. This ensures liveness even if some participants are slow or offline.

In GLP, timeouts use the `wait` primitive:

```
wait_for_leader(Timeout, Block) :-
    wait(Timeout?),
    Block = timeout.
```

```
wait_for_leader(_, Block) :-
    known(Block?) | .
```

The first clause suspends for the timeout duration; the second succeeds immediately if a block arrives. Whichever happens first proceeds.

12.6 The Ordering Function

Each participant computes the same finalized sequence from their local blocklace view using the ordering function τ .

Definition of τ

Given a blocklace B , $\tau(B)$ returns the sequence of finalized payloads:

1. Identify all finalized blocks (those with majority endorsement and ratification)
2. Order them by wave number
3. Within a wave, order deterministically (e.g., by block hash)
4. Output non-empty payloads in this order

Consistency

With reliable dissemination and no Byzantine agents, all correct participants eventually observe the same blocklace and compute the same $\tau(B)$. This is the consensus output.

Incremental Computation

Participants compute τ incrementally as new blocks arrive:

```
compute_tau(Blocklace, FinalizedSoFar, NewFinalized) :-
    find_new_finalized(Blocklace?, FinalizedSoFar?, Candidates),
    order_candidates(Candidates?, Ordered),
    append(FinalizedSoFar?, Ordered?, NewFinalized).
```

12.7 Implementation in GLP

We implement Constitutional Consensus using the agent-and-stream pattern. Each participant runs an agent process that maintains state, processes incoming blocks, and produces outgoing blocks.

Agent State

Each participant maintains:

```
state(Blocklace, Mode, CurrentRound, Finalized, Pending)
```

Where:

- `Blocklace` is the local view of all observed blocks
- `Mode` is low or high
- `CurrentRound` is the latest advanced round
- `Finalized` is the τ output so far
- `Pending` is the queue of transactions awaiting proposal

Agent Process

The main agent loop processes events (incoming blocks or timeouts):

```
agent([], State, _, Finalized) :-
    State = state(_, _, _, Finalized?, _) | .
agent([Event|Events], State, OutStream, Finalized) :-
    handle_event(Event?, State?, State1, OutStream?, OutStream1),
    update_finalized(State1?, Finalized?, Finalized1),
    agent(Events?, State1?, OutStream1?, Finalized1?).
```

Handling Incoming Blocks

When a block arrives, add it to the blocklace and check for mode transitions:

```
handle_event(block(R, Payload, Ptrs), State, State1, Out, Out1) :-
    State = state(B, Mode, Round, Fin, Pend),
    add_block(block(R?, Payload?, Ptrs?), B?, B1),
    check_mode(B1?, Mode?, Mode1),
    advance_round(B1?, Round?, Round1),
    maybe_issue(Mode1?, Round1?, Pend?, B1?, Out?, Out1, Pend1),
    State1 = state(B1?, Mode1?, Round1?, Fin?, Pend1?).
```

Issuing Blocks

Block issuance depends on mode and round:

```
%% Low-throughput: issue if we have payload and round is first of wave
maybe_issue(low, Round, [Tx|Pend], B, Out, Out1, Pend?) :-
    Round? mod 3 == 1 |
    tips(B?, Tips),
```

```

    Out = [block(Round?, Tx?, Tips?) | Out1?].

%% Low-throughput: endorse single candidate
maybe_issue(low, Round, Pend, B, Out, Out1, Pend?) :-
    Round? mod 3 == 2,
    single_candidate(B?, Round?, Candidate?) |
    tips(B?, Tips),
    Out = [block(Round?, empty, Tips?) | Out1?].

%% Low-throughput: ratify if majority endorsed same candidate
maybe_issue(low, Round, Pend, B, Out, Out1, Pend?) :-
    Round? mod 3 == 0,
    majority_endorsed(B?, Round?, _) |
    tips(B?, Tips),
    Out = [block(Round?, empty, Tips?) | Out1?].

%% High-throughput: issue if leader or timeout
maybe_issue(high, Round, Pend, B, Out, Out1, Pend1) :-
    Round? mod 3 == 1 |
    issue_high_throughput(Round?, Pend?, B?, Out?, Out1, Pend1).

%% No action needed
maybe_issue(_, _, Pend, _, Out, Out?, Pend?).

```

Leader Selection

In high-throughput mode, the formal leader for round r is determined by:

```

leader(Round, Participants, Leader) :-
    length(Participants?, N),
    WaveNum := Round? // 3,
    Index := WaveNum? mod N?,
    nth(Index?, Participants?, Leader).

```

The leader rotates round-robin through participants by wave number.

Finality Detection

A block is finalized when it has majority endorsement and ratification:

```

is_finalized(Block, Blocklace, Participants) :-
    endorsements_for(Block?, Blocklace?, Endorsers),
    is_majority(Endorsers?, Participants?),
    ratifications_for(Endorsers?, Blocklace?, Ratifiers),
    is_majority(Ratifiers?, Participants?).

```

```
is_majority(Set, Participants) :-
    length(Set?, M),
    length(Participants?, N),
    M? > N? // 2 | .
```

Computing the Finalized Stream

Each participant maintains their finalized output stream:

```
update_finalized(State, OldFin, NewFin) :-
    State = state(B?, _, _, _, _),
    find_newly_finalized(B?, OldFin?, New),
    append(OldFin?, New?, NewFin).

find_newly_finalized(Blocklace, AlreadyFin, New) :-
    all_finalized(Blocklace?, AllFin),
    subtract(AllFin?, AlreadyFin?, New).
```

12.8 A Complete Example

Three participants—Alice, Bob, and Carol—form a consensus instance. We trace through both low and high-throughput scenarios.

Setup

The genesis block establishes:

```
genesis([alice, bob, carol], 0.5, 1000)
```

Three participants, majority threshold, 1000ms timeout.

Low-Throughput Scenario

Alice has a transaction `tx_a` to propose. The network is quiescent.

Wave 1, Round 1. Alice creates:

```
block(1, tx_a, [genesis])
```

Bob and Carol receive Alice’s block. No conflicts—the wave is non-contentious.

Wave 1, Round 2. Bob and Carol each endorse:

```
block(2, empty, [alice_block_1]) % Bob’s endorsement
block(2, empty, [alice_block_1]) % Carol’s endorsement
```

Alice also endorses her own block (participants endorse what they see).

Wave 1, Round 3. All three ratify:

```
block(3, empty, [bob_endorse, carol_endorse, alice_endorse])
```

Finality. Each participant observes majority endorsement (3/3) and majority ratification (3/3). Alice's block is finalized. All three output:

```
Finalized = [tx_a]
```

High-Throughput Scenario

Now Alice and Bob both have transactions simultaneously.

Wave 2, Round 4. Alice creates:

```
block(4, tx_a2, [ratify_blocks...])
```

Bob creates:

```
block(4, tx_b, [ratify_blocks...])
```

Carol observes both—conflict detected. All switch to high-throughput mode.

Formal Leader. Wave 2 has leader index $2 \bmod 3 = 2$, so Carol is the formal leader. But Carol has no transaction, so Alice's and Bob's blocks compete.

The protocol specifies: endorse the *leader block* if present, otherwise the *first block received* (deterministic tie-breaker needed). Suppose all endorse Alice's block (she was leader for wave 2 by the round-robin).

Rounds 5 and 6. Endorsements and ratifications proceed. Alice's `tx_a2` is finalized.

Wave 3. Bob retries with `tx_b`. If no conflicts, it finalizes. If Carol also has a transaction now, they compete again under formal leadership.

Execution Trace

Wave 1 (low-throughput):

R1: Alice proposes `tx_a`

R2: All endorse Alice

R3: All ratify

Finalized: `[tx_a]`

Wave 2 (high-throughput, conflict):

R4: Alice proposes tx_a2, Bob proposes tx_b (conflict!)
 R5: All endorse Alice (formal leader)
 R6: All ratify
 Finalized: [tx_a, tx_a2]

Wave 3 (back to low if Bob alone):

R7: Bob proposes tx_b
 R8: All endorse
 R9: All ratify
 Finalized: [tx_a, tx_a2, tx_b]

12.9 Correctness Properties

The implementation satisfies the required consensus properties.

Safety

Claim No two correct participants finalize different transactions at the same sequence position.

Argument A transaction is finalized only when it has majority endorsement and majority ratification. Any two majorities overlap. If two different transactions were both finalized at position k , both would need majority endorsement in the same wave—but a participant endorses at most one candidate per wave. Contradiction.

Liveness

Claim After GST, every transaction submitted by a correct participant is eventually finalized.

Argument After GST, messages arrive within Δ . In low-throughput mode, a transaction with no conflicts finalizes in one wave (3Δ). In high-throughput mode, the round-robin leader selection ensures every participant eventually becomes leader. A correct leader's transaction is endorsed and ratified within one wave. Timeouts ensure progress even if some participants are slow.

Totality

Claim All correct participants eventually output the same finalized sequence.

Argument With reliable dissemination, all participants eventually observe the same blocklace. The τ function is deterministic. Therefore, all participants compute the same finalized sequence.

What Attestation Provides

The correctness argument relies on:

- No equivocation (each participant creates at most one block per round)
- Reliable dissemination (all blocks eventually reach all participants)
- Protocol compliance (participants follow the protocol)

Attestation mechanisms ensure these properties. Without attestation, Byzantine agents could equivocate (creating conflicting blocks) or selectively disseminate (sending different blocks to different participants). The protocol would then require $\sigma > 2/3$ and additional mechanisms (nack blocks, equivocation detection) to maintain safety.

12.10 Exercises

1. **Quiescence Detection.** Implement a predicate `is_quiescent/2` that determines whether a wave in the blocklace is quiescent (all blocks empty).
2. **Conflict Detection.** Implement `has_conflict/2` that detects whether a round contains multiple non-empty blocks from different participants.
3. **Leader Verification.** Implement `is_leader_block/3` that verifies whether a block is from the formal leader for its wave.
4. **Endorsement Counting.** Implement `count_endorsements/3` that counts how many participants endorsed a given candidate block.
5. **Finality Checker.** Implement `check_finality/3` that takes a blocklace and returns the list of all finalized blocks in wave order.
6. **Mode Transition.** Extend the agent to explicitly track mode transitions, logging when it switches between low and high-throughput modes.
7. **Timeout Integration.** Implement the timeout mechanism using `wait/1`, ensuring the agent proceeds to the next round if the leader block doesn't arrive within Δ .
8. **Transaction Queue.** Extend the agent to maintain a queue of pending transactions, proposing them one at a time as waves complete.
9. **Blocklace Visualisation.** Write a predicate that outputs the blocklace structure showing waves, rounds, and finalized blocks.
10. **Byzantine Simulation.** Modify the implementation to simulate a Byzantine participant who equivocates (creates two conflicting blocks). Verify that with $\sigma = 1/2$, safety can be violated, demonstrating why attestation is necessary.

GLP Syntax Reference

This appendix provides a complete formal syntax definition for GLP.

Lexical Elements

Identifiers

- **Variables:** Start with uppercase letter or underscore
- **Constants:** Start with lowercase letter
- **Numbers:** Integer or floating-point literals

Reader/Writer Notation

- **Writer:** plain variable name, e.g., `X`
- **Reader:** variable name with `?` suffix, e.g., `X?`

Terms

```
term ::= variable
      | constant
      | number
      | functor(term, ..., term)
      | [term | term]           % list
      | []                     % empty list
```

Atoms and Goals

```
atom ::= predicate(term, ..., term)
      | term = term             % unification
      | term is expression      % arithmetic

goal ::= atom
      | atom, goal              % conjunction
      | (goal)                  % grouping
```

Clauses

```

clause ::= atom.                % fact
        | atom :- goal.         % rule
        | atom :- guard | goal. % guarded rule

```

Guards

```

guard ::= guard_atom
        | guard_atom, guard      % conjunction

guard_atom ::= ground(term)
            | unknown(term)
            | known(term)
            | term == term        % identity
            | term \== term       % non-identity
            | arithmetic_comparison

```

Programs

```

program ::= clause
         | clause program

```

SRSW Requirements

For a clause to satisfy SRSW:

1. Each writer variable occurs at most once
2. Each reader variable occurs at most once
3. Variables occur in reader/writer pairs

GLP Reference Manual

This appendix provides a complete reference for GLP syntax, guards, system predicates, and built-in operations.

Lexical Syntax

Variables A *writer* is an alphanumeric string beginning with an uppercase letter: `X`, `X1`, `Xs`, `MyVar`. A *reader* is a writer followed by `?`: `X?`, `Xs?`. The underscore `_` is an anonymous variable (don't-care), matching any value without binding.

Strings A *string* begins with a lowercase letter: `a`, `foo`, `myAtom`. Quoted strings are also strings: `'Hello World'`, `'`, `'`.

Numbers Integers: `0`, `42`, `-17`. Floats: `3.14`, `-0.001`, `2.5e10`.

Compound Terms A *structure* has a functor (string) and arguments: `point(X,Y)`, `node(Left,Value,Right)`. The functor's *arity* is its argument count.

Lists The empty list is `[]`. A non-empty list is `[HeadTail]`— where `Head` is the first element and `Tail` is the rest. Syntactic sugar: `[1,2,3]` means `[1[2—[3—[]]]]`—. Mixed notation: `[1,2Xs]`— means `[1[2—Xs]]`—.

Clause Syntax

A *clause* has the form:

Head `:-` **Guard** `|` **Body**.

where **Guard** and **Body** are optional. A *unit clause* (fact) is just **Head**.

The guard separator is `|`. The body is a comma-separated conjunction of goals.

SRSW Requirement Each variable in a clause must occur exactly twice: once as a writer, once as a reader. Exception: anonymous variables `_` occur once.

Programming Style: Fold Unifications into the Head Avoid explicit unification `X = Pattern` in guards. Instead, fold the pattern directly into the clause head. This ensures correct SRSW analysis and produces cleaner code.

```
% Avoid:
process(Xs, Out?) :- Xs = [H,T|Rest] | handle(H?, T?, Rest?, Out).

% Prefer:
process([H,T|Rest], Out?) :- handle(H?, T?, Rest?, Out).
```

Guard Predicates

Guards appear between the head and `|`, testing conditions without side effects. A clause is applicable only if its guard succeeds.

Built-in Guards

ground(X) Succeeds if `X` contains no unbound variables. With this guard, the clause body may contain multiple occurrences of `X?` without violating SRSW, enabling safe replication of ground terms.

known(X) Succeeds if `X` is not an unbound variable (though it may contain unbound variables in subterms).

integer(X) Succeeds if `X` is bound to an integer.

number(X) Succeeds if `X` is bound to a number (integer or float).

constant(X) Succeeds if `X` is bound to a constant (string or number).

compound(X) Succeeds if `X` is bound to a compound term.

is_list(X) Succeeds if `X` is bound to a proper list.

unknown(X) Succeeds if `X` is bound to an unbound variable. **Warning: This guard is non-monotonic**—its result can change from true to false as computation proceeds. Use only when testing whether a stream has been extended, as in the interlaced streams pattern. Example:

```
% Test if stream tail Bs is still unextended
collect_tips([[Block|Bs]|Others], [Block?|Tips?], ...) :-
    unknown(Bs?) | % Bs unbound means Block is the current tip
    ...
```


otherwise Succeeds if all previous clauses for this predicate failed (not suspended). Enables default case handling.

X =?= Y Ground equality test. Succeeds if **X** and **Y** are both ground and structurally equal. Suspends if either contains unbound readers. Fails if both ground but different, or if either contains unbound writers.

X := Y Succeeds if **X** and **Y** evaluate to equal numbers.

X \= Y Arithmetic inequality. Succeeds if **X** and **Y** evaluate to different numbers.

X < Y Succeeds if **X** evaluates to less than **Y**.

X > Y Succeeds if **X** evaluates to greater than **Y**.

X =< Y Succeeds if **X** evaluates to less than or equal to **Y**.

X >= Y Succeeds if **X** evaluates to greater than or equal to **Y**.

wait(Duration) Suspends for **Duration** milliseconds, then succeeds. Uses standard GLP suspension mechanism: creates a fresh reader/writer pair, starts a timer, suspends on the reader. When the timer fires, it binds the writer, reactivating the goal via the normal ROQ mechanism.

wait_until(Timestamp) Succeeds if the current time (milliseconds since epoch) is greater than or equal to **Timestamp**. **Note:** This is a test, not a blocking wait. If the time has not yet passed, it *fails* (not suspends), allowing the next clause to be tried.

Guard Negation

Any negatable guard **G** can be negated using the syntax **~G**. The negated guard succeeds when **G** fails, fails when **G** succeeds, and suspends when **G** suspends.

Negatable guards Type guards: **ground**, **known**, **unknown**, **integer**, **number**, **constant**, **compound**, **tuple**, **list**, **is_list**, **writer**, **reader**. Equality guard: **==**.

Non-negatable guards Arithmetic comparisons (**<**, **>**, **=<**, **>=**, **:=**, **\=**) cannot be negated. Control guards (**otherwise**) and time guards (**wait**, **wait_until**) also cannot be negated.

Restrictions Only built-in guards can be negated. Defined guards (unit clauses) cannot be negated. Double negation **~~G** is syntactically forbidden.

Examples

```
% Handle non-integers
process(X, Y) :- ~integer(X?) | handle_non_int(X?, Y).
process(X, Y) :- integer(X?) | handle_int(X?, Y).

% Structural inequality via negated equality
lookup(Key, [(K,V)|_], V?) :- Key? == K? | true.
lookup(Key, [(K,_)|Rest], V?) :- ~(Key? == K?) | lookup(Key?, Rest?, V?).
```

System Predicates

System predicates execute atomically with goal reduction, providing access to runtime services.

X? := Expr Evaluate arithmetic expression **Expr** and bind result to **X**. The expression must be ground. See Section 12.10 for operators.

current_time(T) Bind **T** to the current system timestamp (milliseconds since epoch).

get_time(T) Synonym for **current_time(T)**. Binds **T** to current timestamp.

variable_name(X, Name) Bind **Name** to a unique string identifier for variable **X**.

functor(Term, F, N) Unify **F** with the functor and **N** with the arity of **Term**.

arg(N, Term, Arg) Unify **Arg** with the **N**th argument of **Term** (1-indexed).

copy_term(Term, Copy) Create **Copy** as a renamed copy of **Term** with fresh variables.

abort(Message) Terminate execution with error message.

Arithmetic

Arithmetic expressions are evaluated using the **:=** operator:

```
Result? := Expression
```

The expression must be ground at evaluation time. The definition:

```
X? := E :- ground(E?) | evaluate(E?, X).
```

Arithmetic Operators

Operator	Description	Example
$X + Y$	Addition	$R := 3 + 4$
$X - Y$	Subtraction	$R := 7 - 2$
$X * Y$	Multiplication	$R := 3 * 4$
X / Y	Division (float result)	$R := 7 / 2$
$X // Y$	Integer division	$R := 7 // 2$
$X \bmod Y$	Modulo	$R := 7 \bmod 3$
$-X$	Unary negation	$R := -5$
$\text{abs}(X)$	Absolute value	$R := \text{abs}(-5)$
$\text{sqrt}(X)$	Square root	$R := \text{sqrt}(16)$
$\sin(X)$	Sine (radians)	$R := \sin(3.14)$
$\cos(X)$	Cosine (radians)	$R := \cos(0)$
$\tan(X)$	Tangent (radians)	$R := \tan(0.5)$
$\exp(X)$	Exponential (e^X)	$R := \exp(1)$
$\ln(X)$	Natural logarithm	$R := \ln(2.718)$
$\log(X)$	Base-10 logarithm	$R := \log(100)$
$\min(X, Y)$	Minimum	$R := \min(3, 5)$
$\max(X, Y)$	Maximum	$R := \max(3, 5)$
$X ** Y$	Exponentiation	$R := 2 ** 10$

Bitwise Operators

Operator	Description	Example
$X /\!\!/\ Y$	Bitwise AND	$R := 5 /\!\!/\ 3$
$X \backslash\!\!\backslash Y$	Bitwise OR	$R := 5 \backslash\!\!\backslash 3$
$X \text{ xor } Y$	Bitwise XOR	$R := 5 \text{ xor } 3$
$\backslash\!\!\backslash X$	Bitwise complement	$R := \backslash\!\!\backslash 5$
$X \ll N$	Left shift	$R := 1 \ll 4$
$X \gg N$	Right shift	$R := 16 \gg 2$

Channel Abstractions

Channels provide bidirectional communication. A channel is represented as $\text{ch}(\text{In?}, \text{Out})$ where In? is the input stream reader and Out is the output stream writer.

```
% Send message X on channel, return updated channel
send(X, ch(In, [X?|Out?]), ch(In?, Out)).
```

```
% Receive message X from channel, return updated channel
receive(X?, ch([X|In], Out?), ch(In?, Out)).
```

```
% Create paired channels for bidirectional communication
```

```
new_channel(ch(Xs?, Ys), ch(Ys?, Xs)).
```

These are typically defined as guard predicates for use in clause heads:

```
relay(In, Out?, Ch) :-
    In? = [X|In1], send(X?, Ch?, Ch1) |
    relay(In1?, Out, Ch1?).
```

Standard Library Predicates

List Operations

```
% Append two lists
append([], Ys?, Ys).
append([X|Xs], Ys, [X?|Zs?]) :- append(Xs?, Ys?, Zs).

% List length
length([], 0?).
length([_|Xs], N?) :- length(Xs?, N1), N := N1? + 1.

% List membership (suspends until found or list exhausted)
member(X?, [X|_]).
member(X, [_|Xs]) :- member(X?, Xs?).

% Reverse a list
reverse(Xs, Ys?) :- reverse_acc(Xs?, [], Ys).
reverse_acc([], Acc?, Acc).
reverse_acc([X|Xs], Acc, Ys?) :- reverse_acc(Xs?, [X?|Acc?], Ys).
```

Stream Operations

```
% Merge two streams fairly
merge([X|Xs], Ys, [X?|Zs?]) :- merge(Ys?, Xs?, Zs).
merge(Xs, [Y|Ys], [Y?|Zs?]) :- merge(Xs?, Ys?, Zs).
merge([], [], []).

% Distribute stream to multiple consumers (requires ground elements)
distribute([X|Xs], [X|Ys1?], [X|Ys2?]) :-
    ground(X?) | distribute(Xs?, Ys1, Ys2).
distribute([], [], []).

% Tag stream elements with source identifier
tag_stream(Name, [M|In], [msg(Name?, M?)|Out]) :-
    tag_stream(Name?, In?, Out?).
tag_stream(_, [], []).
```

Stream Security Properties

Authenticated GLP streams achieve blockchain-like security through language-level guarantees:

Immutability Once stream element $[XXs]$ — is created with X bound to value T , single-assignment semantics prevents any subsequent assignment to X .

Unforkability The SRSW invariant ensures each writer Xs has exactly one occurrence. Attempting two continuations $Xs=[YYs]$ — and $Xs=[ZZs]$ — would require two occurrences of writer Xs , violating SRSW.

Non-repudiation In authenticated GLP, stream extensions carry attestations $(Xs := [Y|Ys])_{M,p,q}$. The signature by agent p provides cryptographic proof of authorship.

Acyclicity The occurs check prevents a writer from being bound to a term containing its paired reader, ensuring strict temporal ordering.

These properties emerge from GLP’s core semantics rather than requiring additional cryptographic protocols.

Execution Model Summary

1. Goals are selected from the active queue (FIFO).
2. Clauses are tried top-to-bottom until one succeeds.
3. Unification proceeds left-to-right, depth-first.
4. If unification requires binding a reader, the goal suspends.
5. When a writer is bound, goals suspended on its reader are reactivated.
6. Failed goals (no clause matches, no suspension possible) are recorded.
7. Computation terminates when the active queue is empty.

Writer Unification Catalog

This appendix provides a comprehensive catalog of writer unification outcomes, derived systematically from Definition ???. Each entry shows the outcome when unifying a goal term with a clause head term, following the three-case priority: succeed, suspend, fail.

The “First Use” column indicates where each case first appears with explanation in the main text. Subsequent occurrences reference back to that section.

Notation

Throughout this catalog:

- c, d denote distinct constants
- X, Y denote goal writers (from the goal being reduced)
- $X?, Y?$ denote goal readers
- V, W denote head writers (fresh, from the renamed clause)
- $V?, W?$ denote head readers (fresh)
- $\leftarrow$ denotes assignment direction in the resulting writer assignment

Head variables are always fresh (renamed apart), so they never conflict with goal variables.

Atomic Term Unification

These cases cover unification at a single argument position with atomic terms (constants and variables).

Constant vs. Constant

Goal	Head	Outcome	First Use
c	c	succeeds with $\emptyset$	§??
c	d	fails	§??

Identical constants unify with the empty assignment. Distinct constants have no unifier.

Constant vs. Variable

Goal	Head	Outcome	Assignment	First Use
c	V	succeeds	$\{V \leftarrow c\}$	§??
c	$V?$	succeeds	$\{V \leftarrow c\}$	§??

A constant against a head variable (writer or reader) succeeds. Head readers $V?$ behave like head writers for unification because V is fresh and can be assigned. The induced reader assignment then propagates the value to $V?$.

Goal Writer vs. Term

Goal	Head	Outcome	Assignment	First Use
X	c	succeeds	$\{X \leftarrow c\}$	§??
X	V	fails (WxW)	—	§??
X	$V?$	succeeds	$\{X \leftarrow V?\}$	§??

Goal writers can receive values (constants or reader references). Writer-to-writer (X vs V) fails because it would abandon both paired readers $X?$ and $V?$.

The case X vs $V?$ succeeds: the goal writer X is assigned the head reader $V?$. This connects two communication channels—when V is later bound, the value flows to both $V?$ and (via X) to $X?$.

Goal Reader vs. Term

Goal	Head	Outcome	Suspension Set	First Use
$X?$	c	suspends	$\{X?\}$	§??
$X?$	V	succeeds	$\{V \leftarrow X?\}$	§??
$X?$	$V?$	succeeds	$\{V \leftarrow X?\}$	§??

Goal readers cannot be assigned. When a goal reader meets a constant, the goal must wait for its paired writer X to provide a value—hence suspension on $\{X?\}$.

When a goal reader meets a head variable (writer or reader), the head writer V is assigned the goal reader $X?$. This succeeds because only head writers are being assigned.

Compound Term Unification

Compound terms unify by matching functors and recursively unifying arguments.

Functor Matching

Goal	Head	Outcome	First Use
$f(\dots)$	$f(\dots)$	recursive on arguments	§??
$f(\dots)$	$g(\dots)$	fails	§??
f/n	f/m	fails (if $n \neq m$)	§??

Terms with different functors or arities cannot unify.

Variable vs. Compound

Goal	Head	Outcome	Assignment	First Use
X	$f(\dots)$	succeeds	$\{X \leftarrow f(\dots)\}$	§??
$X?$	$f(\dots)$	suspends	$\{X?\}$	§??
$f(\dots)$	V	succeeds	$\{V \leftarrow f(\dots)\}$	§??
$f(\dots)$	$V?$	succeeds	$\{V \leftarrow f(\dots)\}$	§??

Goal writers can be assigned compound terms. Goal readers suspend waiting for their paired writer.

List Structure Cases

Lists are the primary data structure for stream communication. These cases are essential for understanding merge, split, and pipeline programs.

Cons Cell Matching

Goal	Head	Outcome	Notes	First Use
$[X Xs]$	$[H Hs]$	succeeds	Both writers, recursive	§??
$[X Xs]$	$[H Hs?]$	succeeds	Goal tail writer, head tail reader	§??
$[X? Xs?]$	$[H Hs]$	suspends on $\{X?\}$	Consumer waiting for element	§??
$[X Xs?]$	$[H Hs]$	succeeds	Element ready, tail waiting	§??
$[X? Xs]$	$[H Hs]$	suspends on $\{X?\}$	Element not ready	§??

Writer/Reader Tail Discrimination

This is the key mechanism for clause selection in concurrent observers:

Goal	Head	Outcome	Why	First Use
$[X Xs]$	$[H Hs]$	fails (WxW on tail)	Xs vs Hs both writers	§??
$[X Xs]$	$[H Hs?]$	succeeds	Xs assigned $Hs?$	§??
$[X Xs?]$	$[H Hs]$	succeeds	Hs assigned $Xs?$	§??
$[X Xs?]$	$[H Hs?]$	succeeds	Hs assigned $Xs?$	§??

Programs use WxW failure on tails to discriminate which stream is producing. A clause with writer tail matches goals with reader tail, and vice versa.

Empty List Cases

Goal	Head	Outcome	First Use
$[]$	$[]$	succeeds	§??
$[]$	$[H Hs]$	fails	§??
$[X Xs]$	$[]$	fails	§??
Xs (writer)	$[]$	succeeds, $\{Xs \leftarrow []\}$	§??
$Xs?$ (reader)	$[]$	suspends on $\{Xs?\}$	§??

Variable vs. List

Goal	Head	Outcome	Assignment	First Use
Xs	$[H Hs]$	succeeds	$\{Xs \leftarrow [H Hs]\}$	§??
$Xs?$	$[H Hs]$	suspends	$\{Xs?\}$	§??
$[X Xs]$	Ys	succeeds	$\{Ys \leftarrow [X Xs]\}$	§??
$[X Xs]$	$Ys?$	succeeds	$\{Ys \leftarrow [X Xs]\}$	§??

Difference List Cases

Difference lists use paired hole/tail variables for efficient append. The threading mechanism depends on specific unification outcomes.

Difference List Argument Pairs

A difference list is represented as two arguments $(List, Hole)$ where $Hole$ is the uninstantiated tail of $List$.

Goal DL	Head DL	Outcome	Threading	First Use
(L, H)	(L', H')	succeeds	L unified with L' , H with H'	§??
$(L?, H)$	(L', H')	suspends	Waiting for input list	§??
$(L, H?)$	(L', H')	succeeds	Hole provided by caller	§??

Accumulator Threading

The standard difference list accumulator uses:

```
flatten(Tree, Xs, Xs0?) :- ... body uses Xs?, Xs0 ...
```

Position	Mode	Role	First Use
Xs (head writer)	output	Receives constructed list	§??
$Xs?$ (body reader)	input to body	Body reads accumulator	§??
$Xs0?$ (head reader)	input	Hole from caller	§??
$Xs0$ (body writer)	output from body	Body extends list	§??

Guard Evaluation Cases

Guards use the same writer unification but with different operational intent: testing rather than binding.

Equality Guards

Guard	Outcome	Notes	First Use
$X =? = Y$	succeeds if both ground and equal	No binding, pure test	§??
$\sim (X =? = Y)$	succeeds if both ground and different	Negated equality	§??

Note: Unification $X = Y$ in guard position is not supported; use head unification instead.

Type-Testing Guards

Guard	Behavior	On Reader	First Use
<code>ground(X)</code>	succeeds if X ground	suspends if $X?$ unbound	§??
<code>integer(X)</code> / <code>number(X)</code>	succeeds if numeric	suspends if $X?$ unbound	§6.2
<code>is_list(X)</code>	succeeds if proper list	suspends if $X?$ unbound	§??
<code>known(X)</code>	succeeds if principal functor known	suspends if $X?$ unbound	§??
<code>unknown(X)</code>	succeeds if X unbound	fails if functor known	§??
<code>atomic(X)</code>	succeeds if atomic	suspends if $X?$ unbound	§??
<code>compound(X)</code>	succeeds if compound	suspends if $X?$ unbound	§??

Arithmetic Comparison Guards

Guard	Behavior	On Reader	First Use
$X \geq Y$	succeeds if $X \geq Y$	suspends if either unbound	§??
$X > Y$	succeeds if $X > Y$	suspends if either unbound	§??
$X < Y$	succeeds if $X < Y$	suspends if either unbound	§??
$X \leq Y$	succeeds if $X \leq Y$	suspends if either unbound	§??

Arithmetic comparison guards compare numeric values. When applied to readers, they suspend until the readers are bound to ground numeric values.

Guard Negation

Guard	Negated Form	Behavior
<code>integer(X?)</code>	<code>~integer(X?)</code>	succeeds if X not an integer
<code>ground(X?)</code>	<code>~ground(X?)</code>	succeeds if X not ground
<code>X? == Y?</code>	<code>~(X? == Y?)</code>	succeeds if X and Y different

Negation inverts success/fail while preserving suspension. Arithmetic guards cannot be negated.

Otherwise Guard

Guard	Behavior	First Use
<code>otherwise</code>	succeeds if all prior clauses fail or suspend	§??

The `otherwise` guard commits only when no prior clause can possibly succeed—either they failed or they suspended and this clause would not conflict.

Clause Selection Mechanisms

These cases show how writer unification enables clause selection based on argument structure and mode.

Selection by Structure

Goal	Clause 1 Head	Clause 2 Head	Selected	First Use
<code>p([a Xs],...)</code>	<code>p([H T],...)</code>	<code>p([],...)</code>	Clause 1	§??
<code>p([],...)</code>	<code>p([H T],...)</code>	<code>p([],...)</code>	Clause 2	§??
<code>p(X,...)</code>	<code>p([H T],...)</code>	<code>p([],...)</code>	Suspends	§??
<code>p(X?,...)</code>	<code>p([H T],...)</code>	<code>p([],...)</code>	Suspends	§??

Selection by Tail Mode (Cooperative Streams)

For observing two streams that cooperatively produce to a shared output:

Goal Tails	C1: writer/reader	C2: reader/writer	Selected	First Use
writer, reader	fails C1, succeeds C2	C2	§??	
reader, writer	succeeds C1	C1	§??	
writer, writer	fails C1, fails C2	deadlock	§??	
reader, reader	suspends both	suspends	§??	

This mechanism enables fair merge without explicit scheduling.

Selection by Ground Guard

Goal	Guard	Outcome	First Use
$p(a, \dots)$	$\text{ground}(X)$	succeeds	§??
$p(X, \dots)$	$\text{ground}(X)$	fails (X unbound)	§??
$p(X?, \dots)$	$\text{ground}(X?)$	suspends	§??
$p(f(a, b), \dots)$	$\text{ground}(X)$	succeeds	§??
$p(f(a, Y), \dots)$	$\text{ground}(X)$	fails (Y unbound)	§??

Suspension and Reactivation

Suspension Set Computation

Given a regular mgu σ , the suspension set is:

$$W_\sigma = \{X? \in V? : X?\sigma \notin \mathcal{V}\}$$

That is, goal readers that would be mapped to non-variables.

For compound terms, the operational algorithm uses two phases:

1. **Collection:** Process arguments left-to-right, accumulating tentative writer bindings $\hat{\sigma}_w$ and a preliminary suspension set S (readers matched against constants or structures whose paired writers are not yet in $\hat{\sigma}_w$).
2. **Resolution:** Compute $S' = \{X? \in S : X? \notin \text{dom}(\hat{\sigma}_w)\}$. If $S' = \emptyset$, succeed with $\hat{\sigma}_w$. Otherwise, suspend on S' .

This two-phase approach ensures that a reader encountered before its paired writer is bound does not cause spurious suspension. For example, goal $p(X?, X)$ against head $p(a, a)$ succeeds: phase 1 adds $X?$ to S and $X := a$ to $\hat{\sigma}_w$; phase 2 finds $X \in \text{dom}(\hat{\sigma}_w)$, so $S' = \emptyset$.

Goal	Head	Suspension Set	First Use
$f(X?, a)$	$f(b, Y)$	$\{X?\}$	§??
$f(X?, Y?)$	$f(a, b)$	$\{X?, Y?\}$	§??
$f(X?, Y)$	$f(a, b)$	$\{X?\}$	§??
$f(X, Y?)$	$f(a, b)$	$\{Y?\}$	§??

Reactivation Conditions

A suspended goal reactivates when *any* reader in its suspension set receives a value from its paired writer:

Suspended On	Reactivates When	First Use
$\{X?\}$	X is assigned	§??
$\{X?, Y?\}$	X or Y is assigned	§??

Upon reactivation, clause matching restarts from the first clause.

Cross-Reference Index

To be completed: alphabetical index of cases by program construct.

append §??: list threading, accumulator

bounded buffer §??: difference lists, suspension for flow control

flatten §??: difference list threading, type-testing guards

merge §??: stream interleaving, tail mode selection

meta-interpreter §??: known/unknown guards, program encoding

observer §??: writer/reader tail discrimination

producer-consumer §??: reader suspension, writer assignment

quicksort §6.2: partition by comparison, type-testing guards

reverse §??: accumulator, empty list matching

split §??: alternating element distribution

Implementation-Ready GLP

This appendix presents a deterministic, implementation-ready specification for GLP, suitable for workstation and smartphone implementations.

Overview

The nondeterministic transition system of Chapter 3 is a *specification*. For implementation, we need deterministic choices for:

1. Goal selection: which unit goal to reduce next
2. Message delivery: when to apply pending assignments
3. Suspension handling: how to track and reactivate suspended goals

Goal Scheduling

Goals are maintained in a ready queue. Selection is round-robin with priority for activated goals.

Suspension and Activation

When a goal suspends on reader $X?$:

1. The goal is moved to a wait set indexed by $X?$
2. When $X?$ receives a value, all goals in its wait set are activated
3. Activated goals are added to the front of the ready queue

Message Delivery

Messages are delivered when:

1. A goal requiring the message becomes active
2. The message is in the pending set

Deterministic Transition System

[Formal specification to be developed]

Smartphone Implementation

For smartphone deployment, additional considerations:

1. Network communication via message passing
2. Dynamic shared-variable tables
3. Pull-based rather than push-based communication

From Prolog to GLP

This appendix guides Prolog programmers in transitioning to GLP.

Key Differences

Aspect	Prolog	GLP
Variables	Shared, any can write	Reader/writer pairs
Unification	Bidirectional	Writer-only
Clause selection	Backtracking	Committed choice
Execution	Sequential	Concurrent

Converting Programs

Variable Usage

In Prolog:

```
swap(pair(X, Y), pair(Y, X)).
```

In GLP (requires restructuring):

```
swap(pair(X?, Y?), Result) :-  
    Result = pair(Y?, X?).
```

Removing Backtracking

Prolog's generate-and-test patterns must be restructured for committed choice.

Adding Concurrency

Sequential Prolog programs become concurrent GLP programs through stream-based communication.

Common Patterns

[Pattern translations to be developed]

Pitfalls

[Common mistakes to be documented]

Historical Notes

This appendix traces the historical development of ideas leading to GLP.

Logic Programming Origins

Logic programming emerged from the intersection of automated theorem proving and programming language design in the early 1970s.

Kowalski and Colmerauer

Robert Kowalski’s procedural interpretation of Horn clauses and Alain Colmerauer’s Prolog implementation established the field.

The Art of Prolog

Sterling and Shapiro’s “The Art of Prolog” (1986, 1994) codified logic programming techniques and inspired this book’s structure.

Concurrent Logic Programming

Concurrent Prolog

Shapiro’s Concurrent Prolog (1983) introduced committed choice and shared variables for concurrent execution.

The FGCS Project

Japan’s Fifth Generation Computer Systems project (1982–1992) developed parallel logic programming languages including GHC (Ueda) and KL1.

Flat Concurrent Prolog

FCP simplified Concurrent Prolog for practical implementation, leading to the Logix system.

The Grassroots Vision

The grassroots vision emerged from recognizing that:

1. Centralized platforms create single points of failure and control
2. Cryptographic identity enables trustless authentication
3. Personal devices are powerful enough for distributed computation

From FCP to GLP

GLP's key innovation—the SRSW discipline—emerged from recognizing that distributed unification was the fundamental obstacle to grassroots deployment.

Acknowledgments

[To be written]

Social Networks Code

This appendix contains the complete play code for the social networks examples in Chapter 9.

Direct Messaging Play

```
%% play_dm.glp - Direct Messaging Play

%% Channel primitives
new_channel(ch(Xs?, Ys), ch(Ys?, Xs)).

dm_send(Msg, ch(In, [Msg?|Out?]), ch(In?, Out)).

dm_receive(Msg?, ch([Msg|In], Out?), ch(In?, Out)).

bind_resp(Resp?, Resp).

%% Alice: initiates DM request
alice(FriendOut, FriendIn?, AliceResult?) :-
    FriendOut = [dm_request(Resp)|_],
    alice_continue(Resp?, FriendIn, AliceResult).

alice_continue(DMCh, _, result(sent, Reply?)) :-
    known(DMCh?) |
    dm_send(hello_bob, DMCh?, DMCh1),
    dm_receive(Reply, DMCh1?, _).

%% Bob: receives request, creates channel
bob([dm_request(Resp)|_], _, BobResult?) :-
    new_channel(BobDM, AliceDM),
    bind_resp(Resp?, AliceDM?),
    bob_continue(BobDM?, BobResult).

bob_continue(DMCh, result(Msg?, replied)) :-
```

```

    dm_receive(Msg, DMCh?, DMCh1),
    dm_send(hello_alice, DMCh1?, _).

%% Verification
verify(result(sent, Reply), result(Msg, replied)) :-
    report(Msg?, Reply?).

report(hello_bob, hello_alice).

%% The play
play_dm :-
    alice(FriendOut, FriendIn?, AliceResult),
    bob(FriendOut?, FriendIn, BobResult),
    verify(AliceResult?, BobResult?).

```

Feeds and Followers Play

```

%% play_feed.glp - Feed and Followers Play

%% Broadcast with ground guard
broadcast(_, [], []).
broadcast(Post, [(Name, Feed)|Fs], [(Name?, Feed1?)|Fs1?]) :-
    ground(Post?) |
    extend_feed(Post?, Feed?, Feed1),
    broadcast(Post?, Fs?, Fs1).

%% Feed operations
make_feed(feed([])).
extend_feed(Post, feed(Xs), feed([Post?|Xs?])).
get_feed(Name, [(Name?, Feed)|_], Feed?).
get_feed(Name?, [_|Fs], Feed?) :- get_feed(Name, Fs?, Feed).

%% Feed server
serve_feed([post(Content)|Cmds], Fs, Fs2?) :-
    ground(Content?) |
    broadcast(Content?, Fs?, Fs1),
    serve_feed(Cmds?, Fs1?, Fs2).
serve_feed([subscribe(Name)|Cmds], Fs, Fs2?) :-
    make_feed(Feed),
    serve_feed(Cmds?, [(Name?, Feed?)|Fs?], Fs2).
serve_feed([], Fs, Fs?).

%% Alice: Feed author

```

```

alice(Commands?, Followers, FinalFollowers?) :-
    serve_feed(Commands, Followers?, FinalFollowers).

bob_receive(feed([Post1, Post2|_]), result(Post1?, Post2?)).

carol_receive(feed([Post1, Post2|_]), result(Post1?, Post2?)).

verify(result(P1, P2), result(P3, P4)) :-
    check(P1?, P2?, P3?, P4?).

check(second_post, hello_world, second_post, hello_world).

%% The play
play_feed :-
    alice([subscribe(bob), subscribe(carol),
          post(hello_world), post(second_post)], [], Followers),
    play_continue(Followers?).

play_continue(Followers) :-
    ground(Followers?) |
    get_feed(bob, Followers?, BobFeed),
    get_feed(carol, Followers?, CarolFeed),
    bob_receive(BobFeed?, BobResult),
    carol_receive(CarolFeed?, CarolResult),
    verify(BobResult?, CarolResult?).

```

Manager-Based Group Play

```

%% play_group_manager.glp - Manager-Based Group Play

%% Manager: processes commands
manager(Commands?, Feed?, Members?) :-
    manage(Commands, [], [], Feed, Members).

manage([invite(Name)|Cmds], Members, Feed, Feed1?, Members1?) :-
    manage(Cmds?, [(Name?, [])|Members?], Feed?, Feed1, Members1).

manage([post(From, Content)|Cmds], Members, Feed, Feed1?, Members1?) :-
    ground(From?), ground(Content?) |
    prepend_all(msg(From?, Content?), Members?, Members2),
    manage(Cmds?, Members2?, [msg(From?, Content?)|Feed?], Feed1, Members1).

manage([], Members, Feed, Feed?, Members?).

```

```

%% Prepend message to all member streams
prepend_all(_, [], []).
prepend_all(Msg, [(Name, Stream)|Ms], [(Name?, [Msg?|Stream?])|Ms1?]) :-
    ground(Msg?) |
    prepend_all(Msg?, Ms?, Ms1).

%% Get member stream
get_member_stream(Name, [(Name?, Stream)|_], Stream?).
get_member_stream(Name?, [_|Ms], Stream?) :-
    get_member_stream(Name, Ms?, Stream?).

%% Verification
verify_group(Feed, BobRcvd, CarolRcvd) :-
    check_feed(Feed?),
    check_streams(BobRcvd?, CarolRcvd?).

check_feed([msg(carol, hello_from_carol), msg(bob, hello_from_bob)]).

check_streams([msg(carol, hello_from_carol), msg(bob, hello_from_bob)],
               [msg(carol, hello_from_carol), msg(bob, hello_from_bob)]).

%% The play
play_group_manager :-
    manager([invite(bob), invite(carol),
             post(bob, hello_from_bob),
             post(carol, hello_from_carol)], Feed, Members),
    play_continue(Feed?, Members?).

play_continue(Feed, Members) :-
    ground(Feed?), ground(Members?) |
    get_member_stream(bob, Members?, BobStream),
    get_member_stream(carol, Members?, CarolStream),
    verify_group(Feed?, BobStream?, CarolStream?).

```

Interlaced Streams Group Play

```

%% play_group_interlaced.glp - Groups via Interlaced Streams

%% Interlacing: each member produces blocks referencing others' tips
interlace([Payload|Payloads], [block(Payload?, Tips?)|Stream?], Others) :-
    collect_tips(Others?, Tips, Others1),
    interlace(Payloads?, Stream, Others1?).

```



```

interlace([], [], _).

%% Collect current tips from other streams
collect_tips([[Block|Bs]|Others], [Block?|Tips?], [Bs?|Others1?]) :-
    unknown(Bs?) |
    collect_tips(Others?, Tips, Others1).
collect_tips([[_|Bs]|Others], Tips?, [Bs?|Others1?]) :-
    otherwise |
    collect_tips(Others?, Tips, Others1).
collect_tips([], [], []).

%% Member: produces stream of messages interlaced with others
member(Name, Messages?, MyStream?, OtherStreams?) :-
    tag_messages(Name?, Messages, Tagged),
    interlace(Tagged?, MyStream, OtherStreams).

tag_messages(Name, [Msg|Msgs], [msg(Name?, Msg?)|Tagged?]) :-
    tag_messages(Name?, Msgs?, Tagged).
tag_messages(_, [], []).

%% Verification
verify_interlaced(AliceStream, BobStream, CarolStream) :-
    check_has_blocks(AliceStream?),
    check_has_blocks(BobStream?),
    check_has_blocks(CarolStream?),
    report_success.

check_has_blocks([block(_, _)|_]).

report_success.

%% The play
play_group_interlaced :-
    member(alice, [hello, world], AliceStream, [BobStream?, CarolStream?]),
    member(bob, [hi, there], BobStream, [AliceStream?, CarolStream?]),
    member(carol, [hey, all], CarolStream, [AliceStream?, BobStream?]),
    verify_interlaced(AliceStream?, BobStream?, CarolStream?).

```

Child-Safe Social Networking Play

```
%% play_child_safe.glp - Child-Safe Social Networking
```

```
%% Response binding
```

```

bind_resp(Resp?, Resp).

%% Parent-Smith: sends approval request
parent_smith([approval_request(smith, child_smith, Resp)|_], Result?) :-
    smith_wait(Resp?, Result).

smith_wait(Resp, Result?) :-
    known(Resp?) |
    smith_done(Resp?, Result).

smith_done(approved, result(approved, channel_ready)).

%% Parent-Jones: receives and approves
parent_jones([approval_request(FromParent, Child, Resp)|_], Decision?) :-
    bind_resp(Resp?, approved),
    jones_done(FromParent?, Child?, Decision).

jones_done(FromParent, Child, approved(FromParent?, Child?)).

%% Verification
verify_child_safe(result(approved, channel_ready),
    approved(smith, child_smith)) :-
    report_success.

report_success.

%% The play
play_child_safe :-
    parent_smith(StoJ, SmithResult),
    parent_jones(StoJ?, JonesDecision),
    verify_child_safe(SmithResult?, JonesDecision?).

```

Selected Exercise Solutions

This appendix provides solutions to selected exercises from the main text.

Chapter ??: Streams

Exercise ??: Biased Merge

Implement a biased merge operator that maintains fairness in dynamically-constructed trees by tracking subtree weights.

```
% Biased merge with weight tracking
% Bx, By: weights of left and right subtrees
% N: counter for current priority stream

bmerge(X, Y, [started?|Z?]) :- bmerge(1, 1, 1, X?, Y?, Z).

% Counter exhausted: switch priorities
bmerge(Bx, By, 0, X, Y, Z?) :- bmerge(By?, Bx?, By?, Y?, X?, Z).

% Forward from high-priority stream
bmerge(Bx, By, N, [M|X], Y, Out?) :- N? > 0 |
    forward_msg(M?, Bx?, By?, N?, X?, Y?, Out).

% Forward from low-priority stream (only if high-priority has no message)
bmerge(Bx, By, N, X, [M|Y], Out?) :- N? > 0, unknown(X?) |
    forward_msg(M?, By?, Bx?, N?, Y?, X?, Out).

% End of stream
bmerge(_, _, _, [], Y, [halted?|Y?]).
bmerge(_, _, _, X, [], [halted?|X?]).

% Handle control messages
forward_msg(started, Bx, By, N, X, Y, [started?|Z?]) :-
    Bx1 := Bx? + 1,
    bmerge(Bx1?, By?, N?, X?, Y?, Z).
```

```

forward_msg(halted, Bx, By, N, X, Y, [halted?|Z?]) :-
    Bx1 := Bx? - 1,
    bmerge(Bx1?, By?, N?, X?, Y?, Z).
forward_msg(M, Bx, By, N, X, Y, [M?|Z?]) :-
    M =\= started, M =\= halted |
    N1 := N? - 1,
    bmerge(Bx?, By?, N1?, X?, Y?, Z).

```

The key ideas:

- Each merge node tracks **Bx** and **By**, the weights of its subtrees
- When **started** arrives, increment the weight; when **halted** arrives, decrement
- The counter **N** determines how many messages to forward from the high-priority side before switching
- When **N** reaches 0, swap priorities: the other stream becomes high-priority with counter reset to its weight

This achieves linear bounded-waiting: in a tree with n leaves, a message waits at most $O(n)$ messages before exiting the root.

Exercise ??: Stream Length

Compute the length of a stream using an accumulator for tail recursion.

```

length(Xs, N?) :- length_acc(Xs?, 0, N).

length_acc([], Acc, Acc?).
length_acc(_|Xs, Acc, N?) :-
    Acc1 := Acc? + 1,
    length_acc(Xs?, Acc1?, N).

```

The accumulator **Acc** tracks the count seen so far. When the stream ends, the accumulated value is assigned to the output. This is $O(n)$ in both time and space (for the recursive calls, though GLP can optimize tail recursion).

Exercise ??: Increment Stream

Apply increment to each element of a stream.

```

map_inc([], []).
map_inc([X|Xs], [Y?|Ys?]) :-
    Y := X? + 1,
    map_inc(Xs?, Ys).

```

Each element X from the input stream is incremented and placed in the output stream as Y . The recursive call processes the tail concurrently as elements arrive.

Exercise ??: Filter Even Numbers

Select only even numbers from a stream.

```
filter_even([], []).
filter_even([X|Xs], [X?|Ys?]) :-
    X? mod 2 == 0 |
    filter_even(Xs?, Ys?).
filter_even([X|Xs], Ys?) :-
    X? mod 2 \= 0 |
    filter_even(Xs?, Ys?).
```

The guard $X? \text{ mod } 2 == 0$ tests whether X is even. If so, X is included in the output; otherwise, it is skipped. Both clauses have mutually exclusive guards, ensuring deterministic clause selection.

Library Utilities

This appendix documents utility predicates used throughout the book. These predicates provide common functionality for stream processing, channel operations, and protocol support.

Channel Operations

Bidirectional channels are represented as `ch(In?,Out)` where `In?` is the input stream reader and `Out` is the output stream writer.

```
send(X, ch(In,[X?|Out?]), ch(In?,Out)).
receive(X?, ch([X|In],Out?), ch(In?,Out)).
new_channel(ch(Xs?,Ys), ch(Ys?,Xs)).
```

send/3 Adds message `X` to the channel's output stream, returning the updated channel.

receive/3 Removes message `X` from the channel's input stream, returning the updated channel.

new_channel/2 Creates a pair of connected channels where each channel's input is paired with the other's output.

Stream-Channel Relay

Bidirectional relay between a stream pair and a channel:

```
relay([X|In], Out?, Ch) :-
    send(X?, Ch?, Ch1) |
    relay(In?, Out, Ch1?).
relay(In, [X?|Out?], Ch) :-
    receive(X, Ch?, Ch1) |
    relay(In?, Out, Ch1?).
```

The first clause reads from the input stream and sends to the channel. The second clause receives from the channel and writes to the output stream. The two clauses alternate based on message availability.

Broadcast

Sends a message to all recipients in a friends list:

```
broadcast(_, [], []).
broadcast(Msg, [(Name, Out)|Fs], [(Name?, [Msg?|Out1?])|Fs1?]) :-
    broadcast(Msg?, Fs?, Fs1).
```

Recursively traverses the friends list, adding the message to each output stream. Returns the updated friends list with new stream tails for subsequent broadcasts.

Lookup Utilities

Association list operations for friends list management:

```
lookup(Key, [(K, Value)|_], Value?) :- Key? == K? | true.
lookup(Key, [_|Rest], Value?) :-
    otherwise | lookup(Key?, Rest?, Value?).

lookup_send(Key, Msg, Fs, Fs1?) :-
    ground(Key?), ground(Fs?) |
    lookup(Key?, Fs?, [Msg?|Out1]),
    update(Key?, Out1?, Fs?, Fs1).

update(Key, NewOut, [(K, _)|Rest], [(K?, NewOut?)|Rest?]) :-
    ground(K?), Key? == K? | true.
update(Key, NewOut, [Pair|Rest], [Pair?|Rest1?]) :-
    otherwise | update(Key?, NewOut?, Rest?, Rest1).
```

`lookup/3` Finds the value associated with a key in an association list.

`lookup_send/4` Sends a message through the channel associated with a key, returning the updated friends list.

`update/4` Updates the value associated with a key in an association list.

Stream Tagging

When multiple streams merge, source identity is lost. Stream tagging preserves source information:

```
tag_stream(Name, [M|In], [msg(Name?, M?)|Out?]) :-
    tag_stream(Name?, In?, Out).
tag_stream(_, [], []).
```


Each message *M* is wrapped in `msg(Name, M)`, allowing tagged streams to be merged while preserving source identification.

Deferred Injection

Defers insertion of a message until a trigger variable becomes bound:

```
inject(X, Y, Ys, [Y?|Ys?]) :- known(X) | true.
inject(X, Y, [Y1|Ys], [Y1?|Ys1?]) :- unknown(X) | inject(X?, Y?, Ys?, Ys1).
```

When *X* becomes bound, inserts *Y* at the output head. Until then, passes input messages to output while waiting. This enables non-blocking protocol operation where a response must be injected at a later time (Section 8.3).

Time Utilities

Timestamp access for tracing and scheduling:

```
current_time(T) :- time(T).
```

The `time(T)` guard binds *T* to the current timestamp. Used in tracing metainterpreters (Section ??) and absolute time scheduling (Section ??).

Guard Utilities

Commonly used guard predicates (built-in):

`ground(X)` Succeeds when *X* contains no unbound variables.

`known(X)` Succeeds when *X* is bound to a value.

`unknown(X)` Succeeds when *X* is unbound.

`otherwise` Succeeds when no other clause in the procedure matches (as opposed to suspending).

See Appendix 12.10 for the complete guard specification.

Bibliography

- [1] Erik Barendsen and Sjaak Smetsers. Uniqueness typing for functional languages with graph rewriting semantics. *Mathematical Structures in Computer Science*, 6(6):579–612, 1996.
- [2] Luca Cardelli, Liav Orgad, Gal Shahaf, Ehud Shapiro, and Nimrod Talmon. Digital social contracts: A foundation for an egalitarian and just digital society. In *CEUR Proceedings of the First International Forum on Digital and Democracy*, volume 2781, pages 51–60. CEUR-WS, 2020.
- [3] Haim Gaifman and Ehud Shapiro. Fully abstract compositional semantics for logic programs. In *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 134–142, 1989.
- [4] Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50(1):1–101, 1987.
- [5] Jean-Yves Girard. Linear logic: Its syntax and semantics. In Jean-Yves Girard, Yves Lafont, and Laurent Regnier, editors, *Advances in Linear Logic*, volume 222 of *London Mathematical Society Lecture Note Series*, pages 1–42. Cambridge University Press, 1995.
- [6] Alon Kleinman, Yoram Moses, and Ehud Shapiro. Distributed variable server for atomic unification. In *Proceedings of the ninth annual ACM symposium on Principles of distributed computing*, pages 59–74, 1990.
- [7] Andrew Lewis-Pye, Oded Naor, and Ehud Shapiro. Grassroots flash: A payment system for grassroots cryptocurrencies. *arXiv preprint arXiv:2309.13191*, 2023.
- [8] John Alan Robinson. A machine-oriented logic based on the resolution principle. *Journal of the ACM (JACM)*, 12(1):23–41, 1965.
- [9] Ehud Shapiro. Alternation and the computational complexity of logic programs. *The Journal of Logic Programming*, 1(1):19–33, 1984.
- [10] Ehud Shapiro. Grassroots currencies: Foundations for grassroots digital economies. *arXiv preprint arXiv:2202.05619*, 2024.

- [11] Philip Wadler. Linear types can change the world. *Programming concepts and methods*, 2:347–359, 1990.
- [12] Philip Wadler. A taste of linear logic. *Mathematical Structures in Computer Science*, 3(4):367–392, 1993.